Studio II

FujiNet Network Cartridge

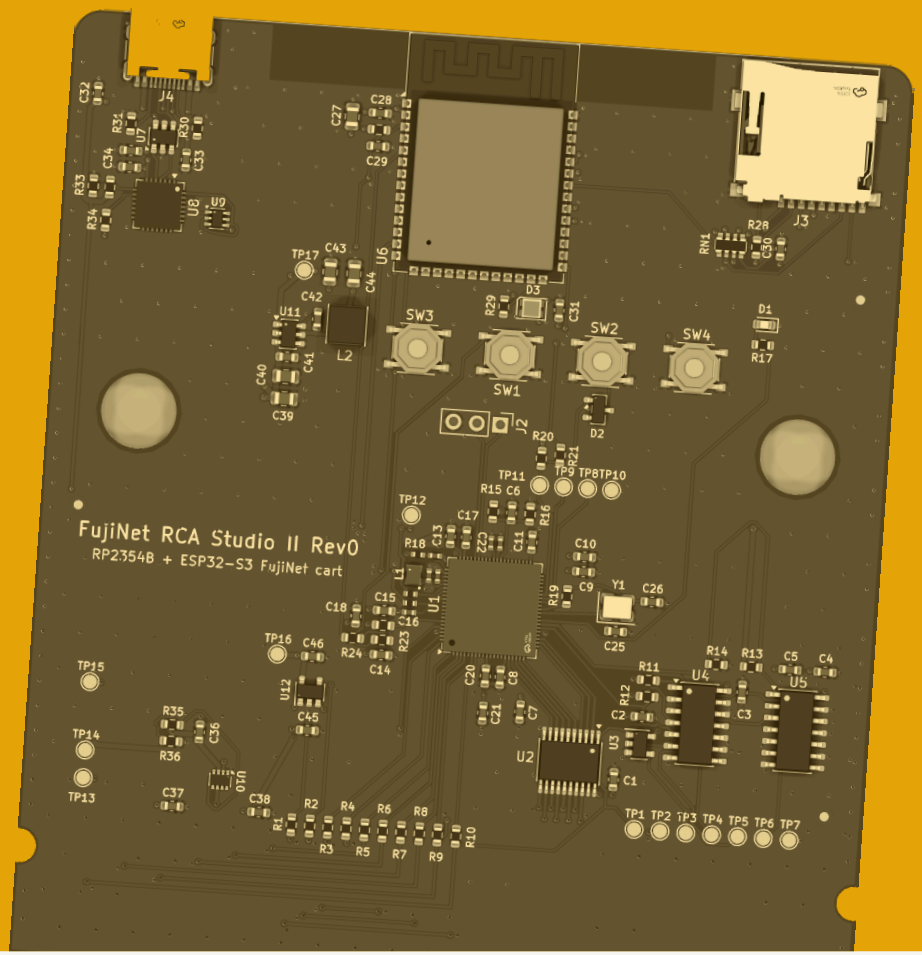
FN-S2

Programmer's Handbook

Assembly Network







FujiNet *Studio II* **Programmer's Handbook.** First edition, 2026.

Typeset after the 1977 *RCA Studio II Owner's Manual* as a tribute. FujiNet is an independent project and is not affiliated with, endorsed by, or sponsored by RCA or its successors. "RCA" and "*Studio II*" are trademarks of their owners and are used here to name the console this handbook is about.

Every address, register, command and code fragment in this handbook is transcribed from the project sources and verified in the FujiNet MAME cartridge model against a running FujiNet adapter. No FujiNet *Studio II* cartridge has been built yet: where a timing or a behaviour matters, the handbook says what was measured in emulation. The sources and their commits are listed in the Parts List at the back.

Text and diagrams © 2026 the FujiNet Project, CC BY-SA 4.0. Program listings carry the licence of their repositories.

Contents

Page

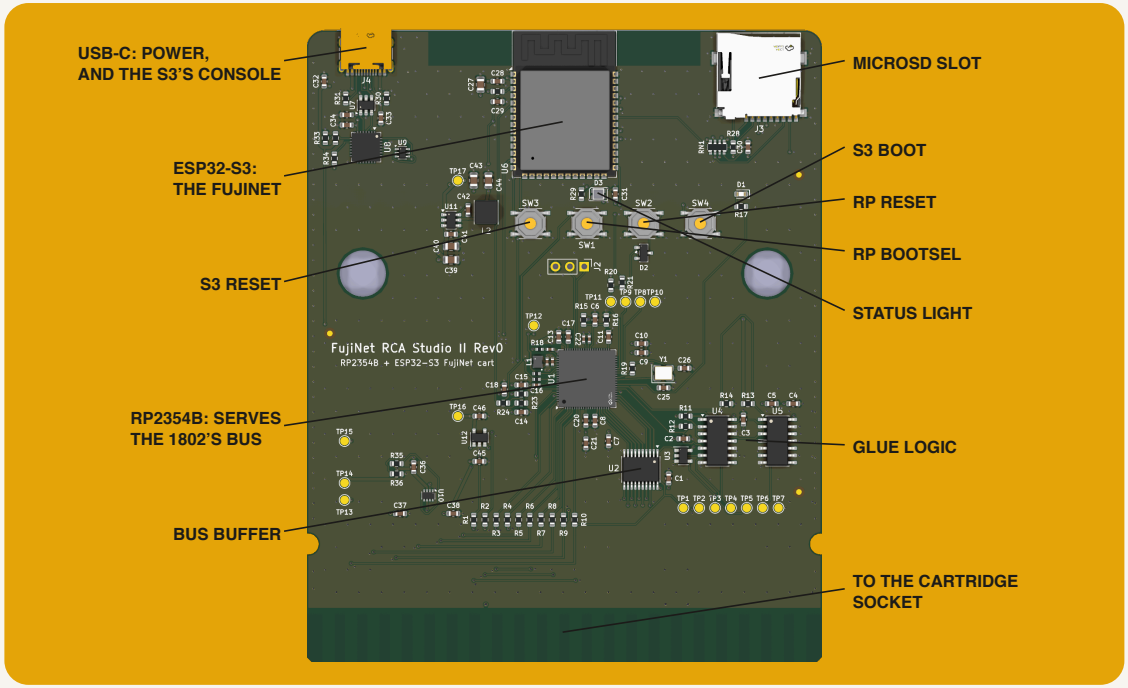
Introduction to FujiNet for *Studio II* 1
Install the Tools 3
The Memory Map 5
How the Mailbox Works 7
The Text Display 10
Living in 512 Bytes 12
The 1802 Runtime 13
Pages and Far Calls 15
Keypads and Sound 16
First Contact 18
The Network Device 20
netcat 22
Directories, Mount and Boot 24
App Keys, Clock and Utilities 26
Glyphs, Tiles and Mosaics 28
Command Reference: The Fuji Device 30
Command Reference: The Network Device 39
Command Reference: Disk, Printer, Clock and Modem 44
Command Reference: The Cartridge Itself 47
Battleship 51
5 Card Stud 53
CONFIG 55
Things That Cost Real Time 57
Troubleshooting Checklist 59
Parts List 62
A. Mailbox Quick Reference 64
B. Error and Status Codes 66
C. Program Listings 67
D. Game and CONFIG Listings 78

Important

This handbook describes the FujiNet cartridge for the RCA *Studio II* as it is built in the FujiNet firmware today. **No cartridge has been made yet.** Every program here runs in the FujiNet model of the cartridge in MAME, talking to a real FujiNet adapter; a real console may differ where timing matters, and the handbook says where.

- Turning the console off is the only way back to CONFIG from a game.
- Test programs against your own servers, never against a public game server.
- Never read the cartridge's raster pages from a program: the cartridge serves them by position, and a stray read moves the picture.

Introduction to FujiNet for *Studio II*



The FujiNet cartridge brings the network to the RCA *Studio II*. Plug it into the cartridge socket and the console that shipped in 1977 with five built-in games can browse a file server, load programs from an SD card or the internet, keep the time, and play against other machines across the world.

The heart of the cartridge is two small computers. An RP2354B microcontroller answers every read the CDP1802 makes, faster than the console can ask: it serves your program, the screen the 1861 draws, and a *mailbox* through which your program talks to the second computer. That one, an ESP32-S3, is the FujiNet itself, the same adapter that serves the Atari, the Apple II, the Coleco ADAM and a dozen other machines, with its WiFi, its SD card and its network devices.

This handbook is for the programmer. It shows how a *Studio II* program written in CDP1802 assembly language reaches the network through that mailbox, one read at a time, and it carries every command the FujiNet answers.

What you should have

- A FujiNet *Studio II* cartridge, or the FujiNet model of it in MAME (Install the Tools, page 2).
- A computer with Python 3 and the Macroassembler AS.
- A FujiNet adapter to talk to: the cartridge's own ESP32-S3, or fujinet-pc running on your computer.
- The listings of this handbook, which build every program it shows.

NOTE:
No FujiNet *Studio II* cartridge has been built yet. Everything in this handbook was run in the cartridge's model in MAME, which runs the cartridge's own firmware sources. Where a real console may differ, the text says so.

How this handbook is organised

The first chapters explain the machine: the tools, the memory map, the mailbox, the text display, the console's 512 bytes of RAM, the 1802 runtime every program is built on, far calls across the cartridge's pages, and the keypads and the beeper.

The middle chapters are programs, each built and run: first contact with the adapter, the network device, a netcat, directories and booting, app keys and the clock, and glyphs, tiles and mosaics.

Four reference chapters list every command the FujiNet answers, with the parameters it wants, the reply it sends and an 1802 example. Three games follow, Battleship, 5 Card Stud and CONFIG, and the back of the book has the troubleshooting checklist, the parts list and the full listings.

Install the Tools

A *Studio II* program for the FujiNet is a file of CDP1802 machine code in the ST2 format, the one the console's emulators and cartridge copiers have used for years. Four tools make and run one: an assembler, a few Python scripts, MAME with the FujiNet cartridge grafted in, and a FujiNet to talk to.

The assembler

Every program in this handbook is written for the Macroassembler AS by Alfred Arnold, which knows the 1802 as `CPU 1802`. Build it from its source, or take a package; the handbook was checked with version 1.42, build 306.

1. Unpack AS and run its `make`. Put `asl` and `p2bin` where you like; the build scripts look in `~/asl/` first, then on your `PATH`, and `ASL=` names any other place.
2. Run AS with `-U`. It makes names case-sensitive, and the runtime depends on it: `fn_frames` is a routine, `FN_FRAMES` a status byte.

IMPORTANT:
AS takes one include list. Give it one `-i` with the folders separated by colons, and keep the paths relative: a long list of absolute paths is cut short without a word, and AS then reports only that it cannot open a file.

The Python tools

Three scripts from the cartridge's firmware come with the handbook's listings, in `listings/runtime/tools/`, and every build runs them:

- `checkrom.py` reads AS's listing and refuses an instruction the cartridge cannot live with (The 1802 Runtime, page 13).
- `mkst2.py` turns AS's `.p` file into an ST2 image, one 256-byte block for every page the program touched, and refuses a page a FujiNet program may not use.
- `st2.py` holds the format and the page rules for both.

They need Python 3 and nothing else.

MAME and the cartridge

No FujiNet *Studio II* cartridge has been built yet, so the cartridge you will use is MAME's. The firmware carries a model of it, `pico/studio2/emu/`, which runs the cartridge's own C sources inside MAME's `studio2` driver: the same mailbox, the same text engine, the same loader.

1. Check out MAME and the FujiNet firmware.
2. Graft the cartridge in: `pico/studio2/emu/apply.sh ~/Workspace/mame`. It copies the sources into `src/mame/rca/` and adds the device to the console.
3. Build only the *Studio II*: `make SUBTARGET=studio2 SOURCES=src/mame/rca/studio2.cpp REGENIE=1 NOWERROR=1`. It takes a minute; afterwards an edit to the cartridge's sources rebuilds in seconds.
4. Put the console's BIOS, `studio2.zip`, in MAME's `roms` folder.

The device is asleep unless the environment wakes it: `FUJINET=1` serves CONFIG from power-on, as a real cartridge would, and `FUJINET_BOOT=` names an ST2 image to serve in its place. `FUJINET_TCP` says where the FujiNet is.

The FujiNet

The cartridge's ESP32-S3 runs the FujiNet firmware's RS-232 build. On your computer the same build runs as `fujinet-pc`, and MAME reaches it over *bus-over-IP* (BoIP), a TCP connection in place of the USB cable inside the cartridge.

1. Build it from the firmware: `./build.sh -p RS232`. The result is in `build/dist/`.
2. Copy `fujinet`, `run-fujinet` and `data/` to a folder of their own, with an `SD/` folder beside them for the SD card.
3. Give it an `fnconfig.ini` with BoIP on: `[BOIP]`, `enabled=1`, `host=localhost` and a `port=` that nothing else on your computer uses.
4. Start it: `./run-fujinet -u http://127.0.0.1:8073 -c fnconfig.ini -s SD`. The web page at that address is the adapter's, as on any FujiNet.

IMPORTANT:
Always set `FUJINET_TCP` when you start MAME. Unset, the cartridge model tries `127.0.0.1:9995`, and on a computer with other FujiNet rigs that port may belong to one of them. Only one MAME may talk to a `fujinet-pc` at a time.

The handbook's bench

The handbook keeps its whole bench in one place. `bench/ports.env` names its ports, and `bench/up.sh` starts `fujinet-pc` on BoIP 9973, a small web server for the network chapter's test file, and the line server the netcat talks to; `bench/up.sh games` adds the Battleship and 5 Card Stud servers. `bench/down.sh` stops them all.

Command	Does
`make listings`	assembles every program into `listings/asm/build/`
`make verify`	runs each one in MAME and checks its screen
`make games`	builds CONFIG, 5 Card Stud and Battleship
`make check`	checks the handbook against its sources

Running a program

`emu/run.sh` starts MAME with a program served from power-on:

```
emu/run.sh listings/asm/build/netget.st2
```

Named with a second argument, a Lua script drives MAME instead of you, with no window: `emu/run.sh netget.st2 screen` is how `make verify` checks a program's screen.

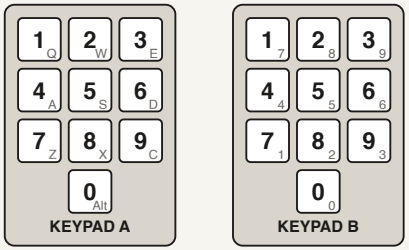


Figure 1. MAME's keys: Q W E, A S D, Z X C and left Alt are keypad A's 1 to 9 and 0; the numeric pad's 7 8 9, 4 5 6, 1 2 3 and 0 are keypad B's. F3 is CLEAR.

On a real console

When the cartridge exists, a program will travel the way every FujiNet program does. Copy the `.st2` file to the FujiNet's SD card or a TNFS server, choose it in CONFIG, and CONFIG mounts it and hands the console over (Directories, Mount and Boot, page 24).

The Memory Map

The CDP1802 has sixteen address lines, but the *Studio II* was built with very little behind them: 2K of BIOS in two ROMs, 512 bytes of RAM, and a cartridge socket that was meant for a 1K or 2K ROM. The FujiNet cartridge fills most of the rest.

It can, because the socket carries every read the console makes. The 1802 puts the high byte of an address on its eight address lines, then the low byte, and the socket carries both, with the strobe that says a read is happening. The cartridge sees the whole address of every read, and when the page is one of its own it pulls the socket's CS line, which switches the console's RAM off the bus, and answers the read itself.

The pages

A FujiNet program lives on the cartridge's pages and the console's RAM. The low four pages of every 4K block, `$x000` to `$x3FF`, belong to no one: the BIOS ROMs decode only some of their address lines and may answer there too, so the cartridge never claims them. That leaves, for a program:

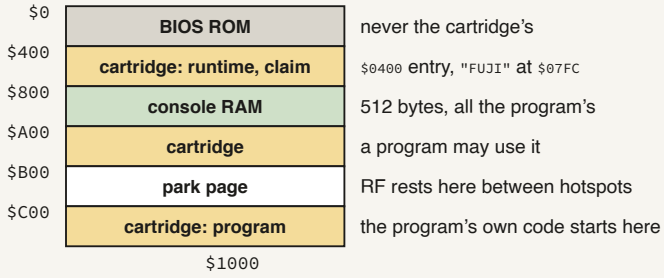
- `$0400-$07FF`, where the BIOS starts it. The runtime fills it.
- `$0A00`, and `$0C00-$0FFF`, where the program's own code begins.
- `$x400-$xFFF` for x from 1 to D: twelve pages in each of thirteen 4K blocks.

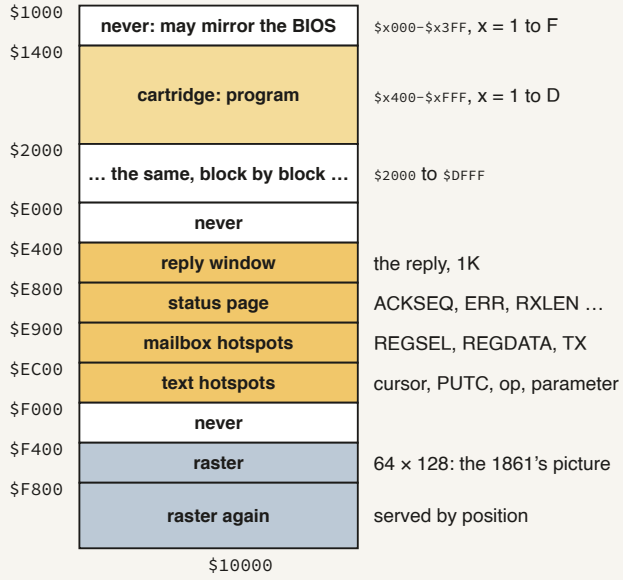
A program may use any 64 of those pages; an ST2 image holds no more.

NOTE:
The first program in this handbook, `fujitest.asm`, checks the mirror on its last row: an `M` for every 4K block whose `$x200` reads the same as the BIOS's `$0200`. In MAME every block shows a dot; on a real console, nobody knows yet.

The arena

Pages `$E4` to `$EF` are the cartridge's *arena*, where a program talks to it. The reply window and the status page are ordinary memory the cartridge paints; the six hotspot pages are not memory at all, and a read there is a message (How the Mailbox Works, page 7). Pages `$F4` to `$FF` are the picture the 1861 shows, which the cartridge serves in the





order the 1861 asks for it, whatever address it is read at. A program never reads them (The Text Display, page 10).

The claim

What decides between a FujiNet program and an old cartridge is four bytes: `"FUJI"` at `$07FC`. An image that carries them is a FujiNet program, and the cartridge serves the arena and the picture with it. An image without them is a *game*: the cartridge serves its pages and nothing else, as a plain ROM cartridge would, and the mailbox is gone until the console is turned off.

Pages	A FujiNet program	A game
`$x0`–`$x3`, `$08`, `$09`	never	never
`$0B`	never: the park page	yes
`$E4` and up	never: the arena and the picture	yes
the others	yes	yes

The runtime's `S2CLAIM` macro puts the four bytes in place; it is the last line of every program in this handbook.

The ST2 image

An ST2 file is a 256-byte header and then up to 64 blocks of 256 bytes, each block one page of the console's memory:

Offset	Holds
0–3	`"RCA2"`
4	the number of blocks, the header included: 2 to 65
5	the version, 1
16–24	the catalogue number, 9 characters
32–63	the title, 31 characters and a NUL
64 + *i*	the console page block *i* goes to
256 × (*i* + 1)	block *i*'s 256 bytes

`mkst2.py` writes one from AS's output: a block for every page the program touched, and `$04`–`$07` always, so the runtime is there. It refuses a page the claim does not allow, and the cartridge refuses it again when the image arrives. The largest image is 16,640 bytes.

NOTE:
When an image has page `$0C`, the cartridge maps all of `$0C00`–`$0FFF`, with zeros where the image has nothing, because MAME's own cartridge does.

How the Mailbox Works

The *Studio II*'s cartridge socket was built for ROM. It carries the address, the data and the strobe for a read, and nothing for a write: there is no write line on the edge at all. A program cannot store a byte into the cartridge.

So the cartridge listens to **where** the console reads. Every read puts an address on the bus, and the cartridge sees all sixteen bits of it. A read of `$EB41` is a perfectly good way of saying `$41`: the high byte names what the byte is for, the low byte carries it. The FujiNet mailbox is built on that one idea.

Writing by reading

The mailbox has three pages of *hotspots*. A read from any of them sends its low byte to the cartridge, and gives back `$FF`, which the program ignores.

Page	A read of `$pp`*vv* sends vv as
`$E9`	REGSEL: the number of the register the next byte goes to
`$EA`	REGDATA: the value for the register REGSEL named
`$EB`	one byte of the TX stream: parameters and payload

Setting a register takes two reads, one of `$E900` plus the register number, then one of `$EA00` plus the value. A REGDATA read with no REGSEL read before it is ignored, and a REGSEL is good for one REGDATA. The runtime's `fn_rw` makes the pair, with RF as the pointer, and leaves RF parked on page `$0B` so a stray read through it touches nothing:

CDP1802 ASSEMBLY (AS) · FUJILIB.INC

```
fn_rw:  PHI     RD  ; RD.1 is free between calls
        LDI     HI(FN_REGSEL)
        PHI     RF
        GLO     R8
        PLO     RF
        LDN     RF
        LDI     HI(FN_REGDATA)
        PHI     RF
        GHI     RD
        PLO     RF
        LDN     RF
        LDI     FN_PARK
        PHI     RF
        FRET
```

IMPORTANT:
Read the hotspots with LDN only. An LDA through RF would step RF to the next address, and the next read through it would send a byte you never meant.

The registers

Name	No.	A value written
DEVICE	`$00`	the FujiNet device: `$70` the FUJI device, `$71` a network channel …
CMD	`$01`	the command, such as `$C4`
NPARAM	`$02`	how many parameters lead the TX stream; at most 8
DATA_RST	`$05`	any value: the TX stream starts over
SEQ	`$10`	a new sequence number: **launch**
BOOTLOCK	`$11`	`$B5`: arm a staged image
CONFIG	`$14`	`$C5`: stage CONFIG and arm it

The whole table, with the two registers no program needs, is in The Cartridge Itself (page 47).

The TX stream

After the registers, the program streams what the command takes. Each parameter is its size, 1, 2 or 4, then its value, low byte first: a one-byte 7 goes as `$01 $07`, a two-byte 512 as `$02 $00 $02`. After the parameters comes the payload, the command's free-form bytes, such as a URL. The stream holds 320 bytes; the cartridge drops any more.

Runtime call	Streams
`fn_pb`	a one-byte parameter, D
`fn_pw`	a two-byte parameter, R8
`fn_txb`	one byte, D, as it is
`fn_path`	a string at R8, padded with NULs to exactly 256 bytes
`fn_str`	a string at R8, as long as it is (`booklib.inc`)

The transaction

A command is a short conversation. The program names the device and the command, says how many parameters follow, streams them, and then writes a new sequence number. That last write launches the transaction. The cartridge sends the request to the ESP32-S3 over USB, waits for its reply, and paints it into the reply window, with a status page above it. The very last thing it paints is ACKSEQ, the sequence number the program wrote: when ACKSEQ matches, the whole reply is in place.

The status page

The cartridge paints the status page, `$E800`, with every reply. The bytes a program reads after a transaction:

At	Name	Holds
+$00	ACKSEQ	the sequence number answered; painted last
+$01	STAT	bit 0: the link to the ESP32-S3 is up
+$02	ERR	0 when the transaction went through (below)
+$03	REPLYCMD	$06 ACK or $15 NAK: the FujiNet's verdict
+$04	RXLEN	how long the reply is, two bytes, low first
+$09	MAGIC	'F', 'N': a FujiNet cartridge

Boot progress, the mode and a few counters live further up; The Cartridge Itself has them all.

The reply window

The reply sits at $E400, up to 1024 bytes, RXLEN of them; past RXLEN the window reads 0. It stays until the next launch, so a program can read it as often as it likes, and the text engine can print from it directly, without the bytes crossing into the console at all. Launching the next transaction paints over it.

The sequence number

The next sequence number is always the cartridge's ACKSEQ plus one, skipping zero. Never keep a counter of your own: the cartridge keeps ACKSEQ across a press of CLEAR, so the first transaction after CLEAR is number 2, while turning the console off restarts the cartridge, and the first is number 1 again. Writing the number the cartridge already holds does nothing at all.

How long to wait

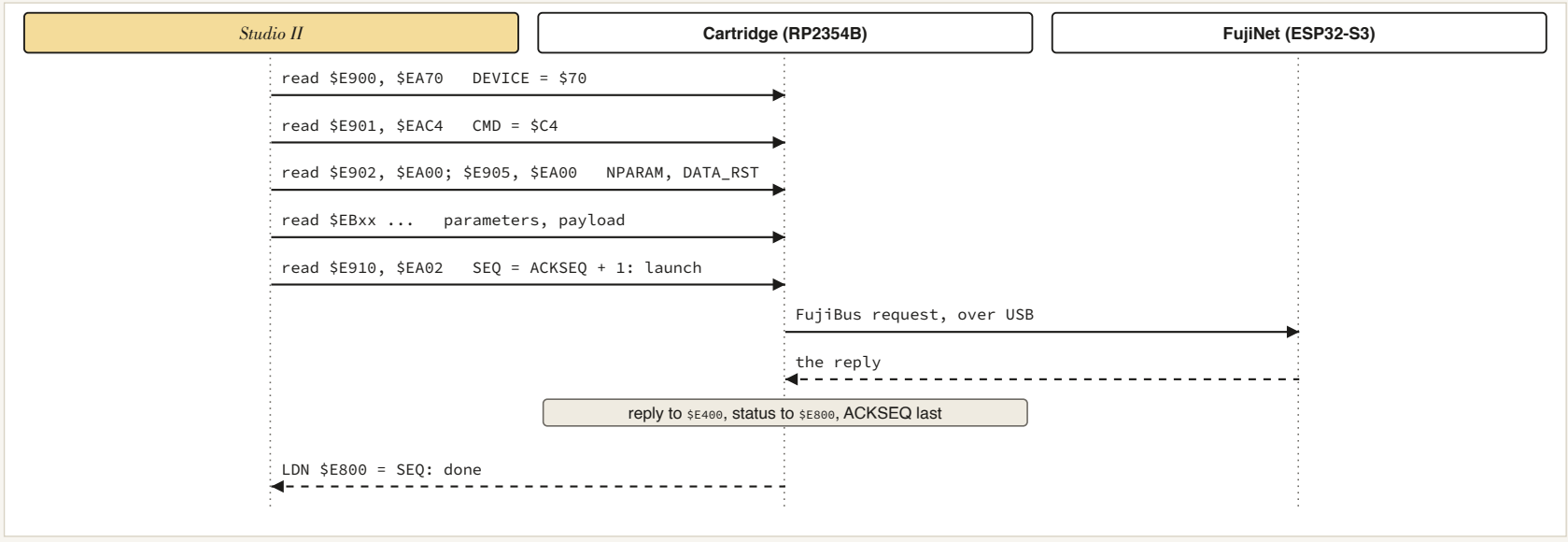
The cartridge gives the FujiNet 5 seconds to answer a command, 60 for MOUNT_IMAGE and COPY_FILE, and 90 for a network channel's OPEN, STATUS, READ and CLOSE. When the time runs out, ERR says so (2, a timeout) and ACKSEQ is painted as usual.

The runtime waits for ACKSEQ for 12 seconds by default; fn_beg sets it, in V_TMO. A slow command must raise it **above** the cartridge's own limit, after fn_beg, as the SLOW macro does:

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN NETDEV, NCOPEN, 2
        LDI     NMREAD
        FCALL   fn_pb
        LDI     NTRNONE
        FCALL   fn_pb
        LDR     R8, url
        FCALL   fn_str
        SLOW    95  ; past the cart's 90
        FCALL   fn_go
        BNZ     failed
```

IMPORTANT:
If the runtime gives up first, it returns FNEWAIT ($FF) while the transaction is still running. Launching again with the same sequence number does nothing, and the reply that arrives later would look like the answer to the retry. Wait longer than the cartridge does, and the question never comes up.



The recipe

1. `fn_chk` once at start: DF = 1 if a FujiNet cartridge answers.
2. `FNBEGIN device, command, nparam`: the three registers and DATA_RST.
3. Stream the parameters, then the payload.
4. `SLOW` if the command can take longer than 12 seconds.
5. `fn_go`: it launches with ACKSEQ + 1 and waits. D = 0 when the transaction went through, else the error.
6. `fn_ack`: D = 0 if the FujiNet said ACK, `FNENAK` if it said no.
7. The reply is at `$E400`, RXLEN bytes.

`go_ack` in `booklib.inc` does steps 5 and 6, and the `GOACK` macro jumps to the failure screen if either says no; every program in this handbook is written with it.

The Text Display





16 columns · 10 rows · 32 glyphs of your own

The *Studio II* draws its picture with the CDP1861: sixty-four dots across, fetched by DMA straight out of memory, one byte for every eight dots. The console's own games keep a 64 × 32 picture in 256 bytes of RAM. A FujiNet program does better: the cartridge serves a picture of 64 × 128 dots from its own memory, and draws text into it for you.

You never store a dot. The cartridge has a *text engine*, a screen of 16 columns by 10 rows, and you drive it the same way you drive the mailbox, with reads of hotspot pages.

The four text pages

Read	Does
`$EC`*rc*	moves the cursor to row r, column c
`$ED`*cc*	draws character cc at the cursor and moves on
`$EE`*op*	runs operation op with the parameters pushed so far
`$EF`*vv*	pushes the parameter vv

A character cell is four dots wide and twelve scanlines tall: a 3 × 5 letter with every row doubled, a blank line above and below, and a blank column between letters. Lower case is drawn as small capitals, so the case shows by height. PUTC wraps from column 15 to the next row, and past row 9 it draws nothing: the screen never scrolls by itself.

NOTE:
On a television a dot is about three scanlines wide, so a cell is a little wider than it is tall and the picture fills the screen from side to side.

Hello

The first program of every FujiNet book prints a greeting. On the *Studio II* it is a cursor move and a string, through the runtime's `d_at` and `d_puts`:

CDP1802 ASSEMBLY (AS) · HELLO.ASM

```
main:   FCALL   d_cls
        LDI     01H
        FCALL   d_at
        LDR     R8, t_hello
        FCALL   d_puts
        LDI     21H
```

`d_at` takes the row and column in one byte, the row in the high nibble: `$21` is row 2, column 1. `d_puts` reads the string from console memory and sends every character as one read of `$ED`. The text engine draws while your program runs: every call is a few reads and returns at once. There is no frame to wait for and no buffer to flip.

The characters

Code	Draws
`$20-$7F`	the font; lower case as small capitals
`$01-$04`	the suits: ♠ ♥ ♣ ♦
`$05`	a solid block ■
`$06-$09`	arrows: right, left, up, down ▶ ◀ ▲ ▼
`$0A`	"10" in one cell, for playing cards
`$0B`	a card's back
`$80-$9F`	your own glyphs (Glyphs, Tiles and Mosaics, page 28)

Every other code draws a blank.

The operations

Push the parameters with `$EF` first, then read the operation's `$EE` address; every operation clears the parameters after it.

Op	Code	Does
CLS	`$00`	clears rows 0-9, cursor home, inverse off, marquee off
INV_ON / INV_OFF	`$01` `$02`	inverse video for what is drawn next
CLREOL	`$03`	blanks from the cursor to the end of the row
SCROLL	`$04`	moves rows 1-9 up one; row 9 comes up blank
PRINT	`$10`	prints reply text: offset (two bytes), then the most to print (0: 256)
PRINT_PAD	`$11`	the same, then spaces out to the most
MARQUEE	`$20`	a later PRINT on this row that is too long scrolls across it
MQ_OFF	`$21`	the marquee stops
REPEAT	`$30`	a character, a count: draws it that many times
GLYPH	`$40`	defines one of your 32 glyphs
TILE / MOSAIC	`$41` `$42`	small tiles, and cells built from them

PRINT is the one to remember. It prints text straight out of the reply window: the cartridge reads its own memory, and the bytes never cross into the console. `d_rpl` and `d_rplpad` wrap it: R8 is the offset into the reply, D the most to print. This is

how `fujitest` shows the adapter's IP address, which starts at byte 140 of GET_ADAPTERCONFIG_EXTENDED's reply:

CDP1802 ASSEMBLY (AS) · FUJITEST.ASM

```
        PUTS    30H, t_ip
        LDI     40H
        FCALL   d_at
        LDR     R8, AC_SIP
        LDI     15
```

The status strip

The text rows take scanlines 0 to 119. Scanlines 120 to 127 belong to the cartridge: while an image is coming over the network it draws a progress bar there, and the program cannot write over it.

Where the picture comes from

The 1861 asks for its 1024 bytes a frame by DMA through R0, which the display interrupt points at `$F400`. The cartridge does not look at the address: it hands out the picture in the order the 1861 asks for it, a line of eight bytes at a time, starting over each frame. That is why the runtime's interrupt routine, in `s2call.inc`, must be what it is (The 1802 Runtime, page 13).

IMPORTANT:
Never read `$F400-$FFFF` from a program. The cartridge would count the read as one of the 1861's and the picture would slip by a byte.

Living in 512 Bytes

The *Studio II* has 512 bytes of RAM, `$0800` to `$09FF`, and a FujiNet program has all of them. The console's own games lose the top 256 to their picture; a FujiNet program's picture lives in the cartridge, so the whole of it is yours.

Everything else a program can reach is read-only: its code, its strings and tables on the cartridge's pages, and the reply window. A variable, the stack and anything the program builds must live in those 512 bytes.

The ledger

Bytes	Belong to
`$0800-$080F`	the runtime's variables, `V_TMO` to `V_T`
`$0810` up	the program's variables, from `V_APP`
… down to R2	the stack, from `$09FF` down

Variable	Holds
`V_TMO`	`fn_go`'s timeout, in seconds
`V_SEQ`	the sequence number it launched
`V_CNT`	`fn_frames`'s countdown, two bytes
`V_LAST`	the frame `fn_tick` last saw
`V_KTMP`	the key number `OUT 2` sends to the keypads
`V_KLAST`	the key held at the last scan
`V_KTIME`	when a held key repeats next
`V_T`	scratch for one routine at a time

Write the program's own ledger at the top of its source, as a comment and as equates, and let the assembler check that the pieces do not overlap. The netcat's is typical: its variables, the line being typed, a copy of the screen, and the stack.

CDP1802 ASSEMBLY (AS) · NETCAT.ASM

```
; ---- RAM ledger: $0800-$09FF ----
; $0800-$080F  the runtime's V_*
; $0810-$081A  the state below
; $0870-$08B0  V_EDIT, the line being typed
; $0900-$098F  SHADOW, rows 0-8 as shown
; $0990-$09FF  the stack (112 bytes)
V_ROW    EQU     V_APP+00H  ; receive row, 0-8
V_COL    EQU     V_APP+01H  ; its column, 0-16
V_EDLEN  EQU     V_APP+02H  ; keyboard: length
V_EDMAX  EQU     V_APP+03H  ;   the most
V_OSK    EQU     V_APP+04H  ;   its cell, 0-95
V_KOLD   EQU     V_APP+05H  ; key_wait's
V_KTO    EQU     V_APP+06H  ;   own two
V_EDWHAT EQU     V_APP+07H  ; 0: URL, 1: line
V_CONN   EQU     V_APP+08H  ; 1: connected
V_N      EQU     V_APP+09H  ; reply bytes left
V_TS     EQU     V_APP+0AH  ; RE.0 at STATUS
V_EDIT   EQU     0870H  ; 65
SHADOW   EQU     0900H  ; 9 rows of 16
```

The stack

R2 is the stack pointer, from `$09FF` downwards, and X is 2 between calls. Each `FCALL` pushes the caller's link, two bytes, and `FRET` takes them back. The byte R2 points at is free: a routine may use it as scratch with `STR R2` and arithmetic on M(R(X)), until its next call.

IMPORTANT:
The display interrupt saves two bytes **below** R2, sixty times a second, at any moment. Never keep anything below the stack pointer.

The reply window is memory too

The reply to the last transaction stays at `$E400` until the next one is launched: a kilobyte of read-only memory a program did not have to find room for. `netget` reads its text straight out of it with LDA, and the text engine prints from it without the bytes crossing into the console at all. Copy out only what must outlive the next transaction.

What survives what

Event	The program	The cartridge
CLEAR	starts over from the BIOS; RAM is the BIOS's to clear	keeps the program and ACKSEQ: the next sequence number goes on
a hand-over	the new program starts; `$08A0-$08FF` are zeroed	ACKSEQ starts again at 0
power off	gone	back to CONFIG

Turning the console off is the only way back from a game, the image that does not claim the mailbox; a FujiNet program can go back to CONFIG by itself, with `fn_config`.

The 1802 Runtime

Every program in this handbook starts with the same few hundred bytes: the runtime the cartridge's own bring-up programs were written with, in the firmware's `pico/studio2/testrom/`. The handbook carries it unchanged, in `listings/runtime/`, and adds two small files of its own.

File	Holds
`fujinet.inc`	names: registers, the arena, the status page, the text engine
`s2macro.inc`	the macros and the runtime's RAM
`s2call.inc`	the entry, the display interrupt, calls, the hand-over
`fujilib.inc`	the mailbox
`fujidisp.inc`	the text engine
`input.inc`	the keypads
`s2defs.inc`	the handbook's: every device and command, and its macros
`booklib.inc`	the handbook's: `go_ack`, `fn_str`, `pause`, `fail`

A program's shape

CDP1802 ASSEMBLY (AS) · CLOCK.ASM

```
        CPU     1802
        INCLUDE "fujinet.inc"
        INCLUDE "s2macro.inc"
        INCLUDE "s2defs.inc"

CKSETZA EQU     74H  ; the adapter's zone

        ORG     0400H
        INCLUDE "s2call.inc"
        INCLUDE "fujilib.inc"
        INCLUDE "fujidisp.inc"

        ORG     0C00H
        INCLUDE "booklib.inc"
```

The equates come first, before any code. The runtime goes at `$0400`, where the BIOS starts a cartridge, and fills it to `$07FB`. The program's own code starts at `$0C00`, and `S2CLAIM` puts `"FUJI"` at `$07FC` last of all.

The registers

Reg.	Is
R0	the 1861's DMA pointer; only the interrupt sets it
R1	the interrupt routine
R2	the stack; X is 2
R3	the program counter
R4, R5, R7	call, return and jump
R6	the link: the caller's return address
R8, R9	scratch: any routine may change them
RA, RB, RC	kept across calls: yours
RD	RD.0 counts the beep down; RD.1 is the runtime's
RE	counts frames, sixty a second
RF	the hotspot pointer: the macros' only

Starting up

The BIOS runs a cartridge by interpreting its first bytes in its own little language. The runtime's first two, `$04 $02`, tell it to call machine code at `$0402`. From there `s2_entry` turns the interrupt off, sets up the registers above, points R1 at its own display interrupt and R0 at the cartridge's picture, turns the interrupt back on and jumps to `main`.

Calls

The 1802 has no call instruction. The runtime keeps three small routines for it, the *standard call and return technique*, in R4, R5 and R7:

Macro	Is
`FCALL adr`	`SEP R4` and the address: a call. R6, the link, goes on the stack
`FRET`	`SEP R5`: back to the caller
`FJMP adr`	`SEP R7` and the address: a jump to anywhere

D and DF go through a call and a return untouched, so a routine can take its argument and give its result in D.

The display interrupt

The 1861 interrupts two scanlines before it starts fetching the picture. The runtime's routine is shaped like the BIOS's own, for the same reason: the first DMA fetch must find the 1802 between instructions.

CDP1802 ASSEMBLY (AS) · S2CALL.INC

```
isrx:   LDA     R2  ; D back
        RET  ; leaves R1 at isr
isr:    DEC     R2
        SAV  ; T
        DEC     R2
        STR     R2  ; D
        LBR     isr1  ; !3CYCLE: before the first DMA
isr1:   LDI     HI(FN_RASTER)
        PHI     R0
        LDI     0
        PLO     R0
        INC     RE
        GLO     RD
        BZ      isrq
        DEC     RD
        SEQ
        BR      isrw
isrq:   REQ
isrw:   B1      isrw
        BR      isrx
```

It saves T and D, points R0 at the picture, counts the frame in RE and runs the beeper, then waits on EF1 until the 1861 has begun. The one `LBR` in it is marked `!3CYCLE`: it is the only long branch the runtime has, and it is where it is on purpose.

The rules

The cartridge serves the bus in software, and a few things that are ordinary on other machines are wrong on this one. `checkrom.py` refuses each of them in every program it is shown:

- **No instruction from `$C0` to `$CF`.** The long branches and long skips, and `NOP`, take three machine cycles. If the 1861 asks for DMA during one, its line comes up a byte short. Leave a page with `FJMP` (Pages and Far Calls, page 15).
- **No `IDL`** (`$00`). It reads memory through R0 on every cycle while it waits, and R0 points at the picture.
- **RF only with `LDN`.** The macros park RF on page `$0B` between hotspot reads. An `LDA`, `STR` or `SEX` through it would send the cartridge a byte you never meant.
- **X on console RAM.** `OUT` and `IRX` read M(R(X)); pointed at the cartridge, they read it.

The mailbox calls

Routine	Does
`fn_chk`	DF = 1 if a FujiNet cartridge answers
`fn_beg`	R9.1 device, R9.0 command, D parameters: begin
`fn_pb`, `fn_pw`	a one- or two-byte parameter, from D or R8
`fn_txb`	one byte of payload, from D
`fn_path`	a string from R8, padded to 256 bytes
`fn_go`	launch and wait: D = 0 or the error
`fn_ack`	D = 0 for ACK, `FNENAK` for NAK
`fn_frames`, `fn_tick`	a countdown in seconds, polled once a frame
`fn_boot`	arm a staged image and hand over
`fn_config`	back to CONFIG

The text calls

Routine	Does
`d_cls`	clears the screen
`d_at`	the cursor to D: row in the high nibble
`d_putc`, `d_puts`	a character from D; a string from R8
`d_hex`	D as two hexadecimal digits
`d_op`, `d_par`	an operation; a parameter
`d_rpl`, `d_rplpad`	PRINT reply text: offset R8, at most D
`in_scan`, `in_get`	the key held; a key pressed

Building

`listings/asm/build.sh` builds a program in three steps: AS assembles it and writes a listing, `checkrom.py` reads the listing for the rules above, and `mkst2.py` packs the pages into `build/`*name*`.st2`. `./build.sh` alone builds them all.

Pages and Far Calls



fourteen pages · one program

The 1802's short branches are two bytes long. The second is the low byte of the target; the high byte stays what it was. A short branch therefore reaches only the 256-byte page it is on, and the long branches that reach further are three-cycle instructions the cartridge cannot allow. A program bigger than a page is a set of pages joined by the runtime's calls.

Staying on the page

AS knows which page each branch is on and refuses one whose target is elsewhere. The runtime's `FIT` macro keeps a routine whole: `FIT 43` starts the next page unless 43 bytes still fit on this one.

CDP1802 ASSEMBLY (AS) · S2MACRO.INC

```
FIT     MACRO   size
        IF      ((($) & 0FFH) + (size)) > 100H
        ORG     (($) + 0FFH) & 0FF00H
        ENDIF
        ENDM
```

Put a `FIT` in front of every routine that branches, with its size. `tools/fitcheck.py` measures each one in the listing and complains when the size is too small, while the routine still happens to fit.

IMPORTANT:
Never fall through into a `FIT`. When it moves to the next page it leaves a gap of zeros behind, and `$00` is `IDL`. End the code before every `FIT` with a branch, an `FJMP` or an `FRET`.

Leaving the page

`FJMP` goes anywhere, at the cost of a trip through the runtime; `FCALL` calls anywhere and comes back. Neither cares which page it is on, and both keep D and DF. A program's routines can be scattered over all the pages it has.

`RFIT`, CONFIG's version of `FIT` that `osk.inc` brings in, also skips the four pages at the bottom of each 4K block, which a program may not use. A program that runs past `$0FFF` should use it.

The pages

Pages	Count
`$04-$07`	4: the runtime
`$0A`	1
`$0C-$0F`	4
`$x4-$xF`, x = 1 to D	156

One ST2 image holds 64 of them, 16K: CONFIG takes 25, Battleship 42.

Fourteen pages

`pages.asm` runs through fourteen pages. Each is a page of its own, made by a macro, and does two things: calls `mark`, which shows the page it was called from, and jumps to the next.

CDP1802 ASSEMBLY (AS) · PAGES.ASM

```
; PAGE here, next -- a page of its own
PAGE    MACRO   here, next
        ORG     here
        FCALL   mark
        FJMP    next
        ENDM
```

`mark` finds the page in R6, the link the call left. The page number is the high byte of the address the call would return to.

CDP1802 ASSEMBLY (AS) · PAGES.ASM

```
; mark -- the caller's page, in hex, at RC.0;
; five to a row. R6 is the caller's return.
        FIT     43
mark:   GHI     R6
        PLO     RA
        GLO     RC
        FCALL   d_at
        GLO     RA
        FCALL   d_hex
        GLO     RC
        ADI     3
        PLO     RC
        ANI     0FH
        SMI     15
        BNF     mk1             ; still room
        GLO     RC
        ANI     0F0H
        ADI     10H
        PLO     RC
mk1:    FRET
```

Keypads and Sound



2 players · both keypads · the beeper

Each player has a ten-key pad: 1 to 9 in a square and 0 below them. The console reads them through a CD4515 decoder and two of the 1802's flag lines, one key at a time, and the runtime's `input.inc` does the reading.

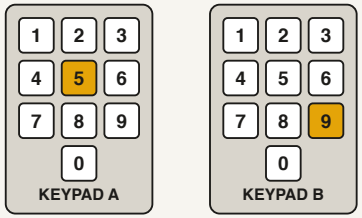


Figure 2. Key 5 on keypad A reads as `$A5`, key 9 on keypad B as `$B9`.

Reading a key

`OUT 2` sends a key number, 0 to 9, to the decoder; EF3 then says whether that key is down on keypad A, and EF4 on keypad B. `in_scan` tries all ten and answers with the first one held:

CDP1802 ASSEMBLY (AS) · INPUT.INC

```
in_scan:
        LDR     R9, V_KTMP
        LDI     0
        PLO     R8
isc1:   GLO     R8
        STR     R9
        SEX     R9
        OUT     2                       ; R9++
        DEC     R9
        SEX     R2
        B3      isc2
        B4      isc3
        INC     R8
        GLO     R8
        XRI     10
        BNZ     isc1
        FRET                            ; D = 0
```

`OUT` takes its byte from memory, M(R(X)), so X points at `V_KTMP` in RAM for the length of it, and back at the stack after. Never point it at the cartridge.

Call	Gives D =
`in_scan`	the key held now: `KEY_A` + k, `KEY_B` + k, or 0
`in_get`	a key just pressed, or held long enough to repeat

`in_get` waits 24 frames before a held key repeats, then repeats it every 6. A menu that must not fire twice, such as CONFIG's, checks for a fresh press itself: its `key_wait` repeats only 2, 4, 6 and 8, the arrows.

Sound

The *Studio II* has one sound: a tone, switched on and off by the 1802's Q line. The runtime's display interrupt runs it for you. While RD.0 is not zero it turns Q on and counts RD.0 down, once a frame; at zero it turns Q off. A beep is one instruction pair:

CDP1802 ASSEMBLY (AS)

```
        LDI     8  ; 8 frames, about 0.13 s
        PLO     RD  ; the interrupt does the rest
```

NOTE:
The interrupt turns Q off on every frame that RD.0 is 0, so a program's own `SEQ` lasts until the next frame at most. Make every tone with RD.0; a longer one is a bigger count.

Both keypads

`keys.asm` draws both keypads, lights the last key pressed on each, shows its code on the bottom row and beeps: briefly for most keys, longer for 0. The main loop is `in_get`, a beep, and a choice of pad:

CDP1802 ASSEMBLY (AS) · KEYS.ASM

```
loop:   FCALL   in_get
        BZ      loop
        PLO     RA
        LDI     96H  ; the code: A5, B9
        FCALL   d_at
        GLO     RA
        FCALL   d_hex
        GLO     RA
        ANI     0FH
        BNZ     lp1
        LDI     40  ; 0: a long beep
        BR      lp2
lp1:    LDI     8
lp2:    PLO     RD  ; the ISR beeps
        GLO     RA
        ANI     0F0H
        XRI     KEY_A
```

```
        BNZ     lp3
        LDI     PADA
        PLO     RB
        LDR     RC, V_LA
        BR      lp4
lp3:    LDI     PADB
        PLO     RB
        LDR     RC, V_LB
lp4:    FCALL   light
        BR      loop
```

Each key's cell is found from a table of ten row-and-column bytes, one for each key, added to the pad's own column:

CDP1802 ASSEMBLY (AS) · KEYS.ASM

```
; cell -- key RA.0 of the pad at column RB.0,
; in inverse when RA.1 is not 0.
        FIT     40
cell:   LDI     LO(t_pos)
        STR     R2
        GLO     RA
        ADD
        PLO     R8
        LDI     HI(t_pos)
        PHI     R8
        LDN     R8  ; row, column
        STR     R2
        GLO     RB
        ADD  ; + the pad's column
        FCALL   d_at
        GHI     RA
        BZ      cel1
        LDI     TOP_INV_ON
        FCALL   d_op
cel1:   GLO     RA
        ADI     '0'
        FCALL   d_putc
        LDI     TOP_INV_OFF
        FCALL   d_op
        FRET
```

First Contact



one transaction · SSID · address · version

The first program to talk to the FujiNet asks it the simplest question it answers: who are you, and where? GET_ADAPTERCONFIG_EXTENDED, command `$C4` of the FUJI device, `$70`, takes no parameters and answers with 240 bytes: the network's name, the adapter's addresses as numbers and as text, and its firmware version.

`fujitest.asm` is the cartridge bring-up's own first contact, unchanged. It is in `listings/runtime/`.

Build it and run it

```
cd listings/asm && ./build.sh fujitest
emu/run.sh listings/asm/build/fujitest.st2
```

On the bench the screen shows the network as `Dummy Cafe`, which is what `fujinet-pc` calls the computer's own network, the address as `127.0.0.1`, and the firmware the adapter was built from.

Is anyone there?

Before anything else the program asks whether a FujiNet cartridge is plugged in at all. `fn_chk` reads two bytes of the status page, which a cartridge paints with `'F'` and `'N'`; with no cartridge, the console reads whatever the bus floats to.

CDP1802 ASSEMBLY (AS) · FUJITEST.ASM

```
main:   FCALL   d_cls
        PUTS    00H, t_title
        FCALL   fn_chk
        BDF     have
        PUTS    20H, t_nocart
        LDI     40H
        FCALL   d_at
        PEEK    FN_MAGIC
        FCALL   d_hex
        PEEK    FN_MAGIC+1
        FCALL   d_hex
        FJMP    idle
```

The transaction

Three registers, no parameters, and the launch:

CDP1802 ASSEMBLY (AS) · FUJITEST.ASM

```
have:   LDI     FNDEVF
        PHI     R9
        LDI     FNCADPX
        PLO     R9
        LDI     0
        FCALL   fn_beg
        FCALL   fn_go
        BNZ     fail
        FCALL   fn_ack
fail:   PLO     RA
        LDR     R8, V_ERR
        GLO     RA
        STR     R8
        BZ      show
        PUTS    20H, t_failed
        LDR     R8, V_ERR
        LDN     R8
        FCALL   d_hex
        FJMP    seq
```

`fn_beg` takes the device in R9.1 and the command in R9.0: the handbook's own programs say the same with `FNBEGIN FNDEVF, FNCADPX, 0`.

The reply

Nothing of it needs to cross into the console. The program prints each field where it lies in the reply window, with PRINT:

CDP1802 ASSEMBLY (AS) · FUJITEST.ASM

```
show1:  PUTS    10H, t_ssid
        LDI     20H
        FCALL   d_at
        LDI     TOP_MARQUEE
        FCALL   d_op
        LDR     R8, AC_SSID
        LDI     32
        FCALL   d_rpl
        PUTS    30H, t_ip
```

The SSID can be 32 characters long and the screen is 16 wide, so its row is armed with MARQUEE first: a PRINT that does not fit scrolls along the row instead of wrapping.

SEQ and ERR

The last row shows the sequence number the cartridge answered and its ERR byte: `SEQ 01  ERR 00`, the first transaction since the console was turned on, without error. Press CLEAR and the



GET_ADAPTERCONFIG_EXTENDED's 240 bytes, at their offsets. Text fields end in a NUL.

program runs again, but the cartridge was not reset: the next transaction is number 2. Turn the console off and on, and it is number 1.

The Network Device



OPEN · STATUS · READ · CLOSE

The FujiNet has eight network channels, devices `$71` to `$78`. Each can hold one connection: an HTTP request, a TCP socket, a file on a TNFS or FTP server, a UDP conversation. The URL says which, and the channel does the rest; a *Studio II* program only opens, asks, reads, writes and closes.

The lifecycle

1. **OPEN** (`$4F`): two one-byte parameters, the mode and the translation, then the devicespec as the payload: `N:` and the URL.
2. **STATUS** (`$53`): two parameters of 0. Four bytes come back: how many bytes are waiting (two), whether the connection is up, and an error code.
3. **READ** (`$52`): one two-byte parameter, the count. The bytes arrive in the reply window.
4. **WRITE** (`$57`): one two-byte parameter, the count, then that many bytes.
5. **CLOSE** (`$43`): no parameters.

Every one of them can take a while: the cartridge gives a network channel 90 seconds, so a program raises its own wait to 95 with `SLOW` after each `FNBEGIN`.

Fetching a file

`netget.asm` fetches a small text file over HTTP and shows it. The URL is the bench's; `build.sh` writes it into `build/gen/bookcfg.inc`, so the program carries no address of its own.

CDP1802 ASSEMBLY (AS) · NETGET.ASM

```
open:   FNBEGIN NETDEV, NCOPEN, 2
        LDI     NMREAD
        FCALL   fn_pb
        LDI     NTRNONE
        FCALL   fn_pb
        LDR     R8, url
        FCALL   fn_str
        SLOW    95  ; past the cart's 90
        GOACK
        FJMP    status
```

Mode 4 is a read: on HTTP, a GET. Translation 0 leaves the bytes alone; 1, 2 and 3 turn the line endings into CR, LF or CR LF. The devicespec goes as the payload at its own length, with `fn_str`.

How much is there?

The adapter answers OPEN as soon as the connection is made, which can be before the whole reply has arrived. STATUS says how many bytes are waiting so far; `netget` asks until two answers agree and are not zero, three frames apart:

CDP1802 ASSEMBLY (AS) · NETGET.ASM

```
stat1:  FNBEGIN NETDEV, NCSTAT, 2
        LDI     0
        FCALL   fn_pb
        LDI     0
        FCALL   fn_pb
        SLOW    95
        GOACK
        LDR     RA, V_AV  ; RA: AV, then PREV
        PEEK    FN_REPLY+NSAVLO
        STR     RA
        INC     RA
```

Then READ takes them, at most 128 here: the reply window holds 1024.

CDP1802 ASSEMBLY (AS) · NETGET.ASM

```
rd2:    GLO     R8              ; fn_beg uses R8
        PLO     RA
        GHI     R8
        PHI     RA
        FNBEGIN NETDEV, NCREAD, 1
        GLO     RA
        PLO     R8
        GHI     RA
        PHI     R8
        FCALL   fn_pw
        SLOW    95
        GOACK
        FJMP    show
```

Showing it

The bytes are in the reply window, RXLEN of them. `netget` walks them with RA and LDA: a CR is skipped, an LF moves to the next row, anything else is drawn.

CDP1802 ASSEMBLY (AS) · NETGET.ASM

```
show:   LDR     RA, FN_REPLY
        PEEK    FN_RXLEN  ; under 256
        PLO     RB
        LDI     ROW0*10H
        PLO     RC  ; RC.0: this row
        FCALL   d_at
shw1:   GLO     RB
        BZ      close
        DEC     RB
        LDA     RA
        XRI     0DH
        BZ      shw1  ; CR: skip it
        XRI     0DH ! 0AH
        BZ      shw2  ; LF: the next row
        XRI     0AH
        FCALL   d_putc
        BR      shw1
shw2:   GLO     RC
        ADI     10H
        PLO     RC
        FCALL   d_at
        BR      shw1
```

NOTE:
`d_putc` wraps at the 16th column by itself, so a long line runs on to the next row; `netget` moves down one more at its LF. The bench's file has short lines.

JSON

The adapter can parse what it fetched and answer one value at a time, so a program never holds a whole JSON document. Set the parser on the open channel, parse, then query by path; the value comes back through STATUS and READ like any other bytes:

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN NETDEV, NCPRSR, 2  ; JSON
        LDI     0
        FCALL   fn_pb
        LDI     NPJSON
        FCALL   fn_pb
        GOACK
        FNBEGIN NETDEV, NCPARSE, 0  ; parse it
        SLOW    95
        GOACK
        FNBEGIN NETDEV, NCQUERY, 0  ; one value
        LDR     R8, query  ; "/0/name"
        FCALL   fn_str
        GOACK  ; then STATUS, READ
```

The parser is the **second** parameter of SET_PARSER; the first is ignored.

Closing

CDP1802 ASSEMBLY (AS) · NETGET.ASM

```
close1: FNBEGIN NETDEV, NCCLOSE, 0
        GOACK
        PUTS    90H, t_done
hang:   BR      hang
```

Close every channel you open, on the way out of a failure too: the adapter keeps a connection open otherwise, even after the console is turned off.

netcat







1 player · Keypad A · any TCP server

netcat is the oldest tool on the network: connect to a server and pass lines back and forth. Every FujiNet book has one, and so does this handbook. It is the largest program in it, 2,281 bytes on 11 pages, and it puts together everything so far: a network channel kept open, a screen that scrolls, the keypads, and an on-screen keyboard.

Playing it

1. The URL comes up in the keyboard, ready to change: `N:TCP://` and a host and port. Press **9** to connect.
2. What the server sends appears in rows 0 to 8, and scrolls up when they are full. Row 9 shows the keys.
3. Press **5** to type a line. Keys 2, 4, 6 and 8 move around the keyboard, **5** types the character under the cursor, **0** deletes the last, **9** sends the line with CR LF, and **1** goes back without sending.
4. Press **0** to close the connection and go back to CONFIG.

NOTE:
Bytes that arrive while you type wait in the FujiNet: netcat reads them when you come back.

The bench's line server, `listings/fixtures/ncserver.py`, greets a connection with `NETCAT SAYS HI` and answers each line with `ECHO` and the line in capitals.

Connecting

The URL opens a TCP connection with mode 12, read and write. Then netcat blanks its copy of the screen and starts the receive view:

CDP1802 ASSEMBLY (AS) · NETCAT.ASM

```
connect:
        FCALL   d_cls
        PUTS    00H, t_conn
        FNBEGIN NETDEV, NCOPEN, 2
        LDI     NMHTTPG  ; 12: read and write
        FCALL   fn_pb
        LDI     NTRNONE
        FCALL   fn_pb
        LDR     R8, V_EDIT
        FCALL   fn_str
        SLOW    95
        GOACK
        SETV    V_CONN, 1
```

Keys every frame, STATUS every eighth

The receive view polls two things: the keypads, every time round, and the connection, every eighth frame. RE, the frame counter, decides which: a press of the keypad takes a few frames, and a loop that slept between polls would miss it.

CDP1802 ASSEMBLY (AS) · NETCAT.ASM

```
poll:   FCALL   in_get
        BZ      pl2
        ANI     0FH
        XRI     5
        BNZ     pl1
        SETV    V_EDWHAT, 1  ; 5: type a line
        SETV    V_EDMAX, 60
        FJMP    st_edit
pl1:    XRI     5  ; 0: close, CONFIG
        BNZ     pl2
        FJMP    quit
pl2:    LDV     V_CONN
        BZ      poll  ; closed: keys only
        LDV     V_TS  ; keys every frame,
        STR     R2  ; STATUS every 8th
        GLO     RE
        SM
        SMI     8
        BNF     poll
        GLO     RE
        STV     V_TS
        FJMP    status
```

STATUS answers how many bytes are waiting, and whether the connection is still up. When nothing is waiting and the server has gone, row 9 says so.

A screen that scrolls

The text engine's SCROLL moves rows 1 to 9 up one, which carries the key row up with them. netcat blanks row 8 again and puts the keys back on row 9:

CDP1802 ASSEMBLY (AS) · NETCAT.ASM

```
nl1:    LDI     TOP_SCROLL      ; rows 1-9 up
        FCALL   d_op
        LDI     80H             ; row 8 blank
        FCALL   d_at
        LDI     TOP_CLREOL
        FCALL   d_op
        FCALL   legend          ; row 9 again
```

A copy of the screen

A program cannot read the text engine's screen back, and the keyboard needs the whole screen. So netcat keeps a copy of rows 0 to 8, 144 bytes at `$0900`, and every character it shows goes there too. After the keyboard, the view is drawn again from the copy: 144 PUTCs, which wrap from row to row by themselves.

CDP1802 ASSEMBLY (AS) · NETCAT.ASM

```
view:   FCALL   d_cls
        LDR     RA, SHADOW
        LDI     NROWS*16
        PLO     RC
vw1:    LDA     RA              ; rows 0-8; PUTC
        FCALL   d_putc          ; wraps the rows
        DEC     RC
        GLO     RC
        BNZ     vw1
```

CONFIG's keyboard

The keyboard is CONFIG's own: rows 3 to 8 hold every character from `$20` to `$7F`, sixteen to a row, so a character's cell is its code less `$20`. `osk.inc` carries CONFIG's routines unchanged, between marker lines that `tools/vendorcheck.py` checks against CONFIG's source. netcat supplies what CONFIG's screens did: the screen around the keyboard, and what 9 and 1 do.

CDP1802 ASSEMBLY (AS) · NETCAT.ASM

```
; ed_ok: 9. ed_esc: 1.
        RFIT    40
ed_ok:  LDV     V_EDWHAT
        BNZ     eok1
        FJMP    connect
eok1:   FJMP    send
ed_esc: LDV     V_EDWHAT
        BNZ     ees1
        FJMP    quit            ; no URL: CONFIG
ees1:   FJMP    view
```

Sending

A WRITE takes one two-byte parameter, the count, then that many bytes: the line, then CR and LF.

CDP1802 ASSEMBLY (AS) · NETCAT.ASM

```
send:   FNBEGIN NETDEV, NCWRITE, 1
        LDV     V_EDLEN
        ADI     2
        PLO     R8
        LDI     0
        PHI     R8
        FCALL   fn_pw           ; the count
        LDR     R8, V_EDIT
        FCALL   fn_str
        LDI     0DH
        FCALL   fn_txb
        LDI     0AH
        FCALL   fn_txb
        SLOW    95
        GOACK
```

Back to CONFIG

The program closes the connection and asks the cartridge for CONFIG. `fn_config` writes `$C5` to the CONFIG register, waits for the cartridge to say it is ready, and hands the console over: CONFIG comes up as if the console had just been turned on.

CDP1802 ASSEMBLY (AS) · NETCAT.ASM

```
quit:   LDV     V_CONN
        BZ      qt1
        FNBEGIN NETDEV, NCCLOSE, 0
        GOACK
qt1:    FCALL   fn_config  ; never returns...
        FJMP    fail  ; ...if not armed
```

Directories, Mount and Boot





hosts · directories · mount · hand over

The FujiNet keeps eight *host slots*, places files come from: its SD card, TNFS servers, web servers. It keeps eight *device slots* too, the drives a computer would boot from. A *Studio II* has no drives, but it boots the same way: a file is named in a device slot, mounted, and the FujiNet sends it to the cartridge, which hands the console over to it.

Listing a directory

`fujidir.asm` lists `/s2book/` on host slot 0, the SD card. Four commands of the FUJI device: MOUNT_HOST connects the host, OPEN_DIRECTORY opens the folder, READ_DIR_ENTRY reads one name a call, CLOSE_DIRECTORY closes it.

CDP1802 ASSEMBLY (AS) · FUJIDIR.ASM

```
mount:  FNBEGIN FNDEVF, FNCMHST, 1
        LDI     HOSTSL
        FCALL   fn_pb
        GOACK
        FNBEGIN FNDEVF, FNCODIR, 1
        LDI     HOSTSL
        FCALL   fn_pb
        LDR     R8, path  ; exactly 256 bytes
        FCALL   fn_path
        GOACK
```

OPEN_DIRECTORY's path goes out as exactly 256 bytes, padded with NULs, and so does every path the FUJI device takes; `fn_path` pads it. After the path's NUL, a filter such as `*.ST2` may follow.

CDP1802 ASSEMBLY (AS) · FUJIDIR.ASM

```
entry:  FNBEGIN FNDEVF, FNCRDIR, 2
        LDI     NAMELN  ; the longest name
        FCALL   fn_pb
        LDI     0  ; flags
        FCALL   fn_pb
        GOACK
        PEEK    FN_REPLY
        XRI     7FH
        BNZ     show
        PEEK    FN_REPLY+1
        XRI     7FH
        BZ      close  ; $7F,$7F: the end
show:   GLO     RC
        FCALL   d_at
        LDR     R8, 0  ; reply offset 0
        LDI     NAMELN
        FCALL   d_rpl
        GLO     RC
        ADI     10H
        PLO     RC
        XRI     0A0H
        BNZ     entry  ; rows 1-9
```

READ_DIR_ENTRY takes the longest name wanted and a flags byte. Each name is printed straight from the reply window. Two bytes of `$7F` say the folder is done.

IMPORTANT:
Never ask for names exactly 31 characters long: at that length the firmware puts icon bytes in front of each name.

Booting

`fujiboot.asm`, the bring-up's, boots `/s2book/hello.st2` from the SD card. The handbook's `build.sh` sets the path; the program is unchanged. Three commands, then the hand-over:

Command	Does
MOUNT_HOST `$F9`	connects host slot 0
SET_DEVICE_FULLPATH `$E2`	device slot, host slot, mode, then the path in 256 bytes
MOUNT_IMAGE `$F8`	device slot, mode: the FujiNet sends the file

MOUNT_IMAGE does two things at once. Before it answers, the FujiNet sends the whole file to the cartridge, over the same USB link, addressed to a device of the cartridge's own. The cartridge stores it beside the program that is running, and paints its progress on the status page: BOOT_STATE goes from 1, coming, to 2, ready, or `$80`, failed, with BOOT_PCT counting to 100 and a bar in the status strip below the text. The cartridge allows a mount 60 seconds; `fujiboot` waits 66.

The hand-over

When BOOT_STATE says ready, `fn_boot` writes `$B5` to BOOTLOCK, which *arms* the new image, and waits for the cartridge to say so on the status page. Then `fn_handover` starts the console over:

1. It waits for the bottom of a frame, when the 1861 is quiet for about 1,850 cycles.
2. It turns the interrupt off and clears `$08A0-$08FF`, the RAM the BIOS does not clear itself.
3. It sets the registers as the console has them after a power-on, and jumps to a stub at `$E9F0` that returns to `$0000`.

CDP1802 ASSEMBLY (AS) · FUJIBOOT.ASM

```
stage:  LDI     4
        PLO     RA
        LDI     10
        FCALL   fn_frames
stg1:   PEEK    FN_BOOTSTATE
        XRI     FN_BOOT_READY
        BZ      stg3
        XRI     FN_BOOT_READY ! FN_BOOT_FAILED  ; AS: ! is XOR, ^ is power
        BZ      stg2
        FCALL   fn_tick
        BDF     stg1
        LDI     FNEWAIT
        PHI     RA
        BR      show
stg2:   PEEK    FN_BOOTERR
        PHI     RA
        LDI     5
        PLO     RA
        BR      show
stg3:   FCALL   fn_boot                 ; returns only if never armed
        PHI     RA
        LDI     6
        PLO     RA
```

4. The BIOS's first fetch from `$0000` is the cartridge's signal: it swaps the new image in, and the BIOS starts it as if it had been in the socket all along.

If the new image claims the mailbox, it finds ACKSEQ at 0 and the screen blank; the status page's MODE reads 3 and SWAPS has gone up by one. If it is an old game, the cartridge serves its pages and nothing else.

NOTE:
CLEAR does not undo a hand-over: it restarts the image that is running. Turning the console off brings CONFIG back.

When it fails

`fujiboot` shows which step failed and with what: `FAILED 03E02`, say, is the third command with ERR 2, a timeout. Step 5 is a failed transfer, with BOOT_ERR: 1 too big, 2 cut short, 3 a page the program may not use. Step 6 is a swap the cartridge never armed.

CDP1802 ASSEMBLY (AS) · FUJIBOOT.ASM

```
        ; ---- MOUNT_IMAGE(dev, mode): the ESP32 streams the file now ----
        LDI     FNDEVF
        PHI     R9
        LDI     FNCMIMG
        PLO     R9
        LDI     2
        FCALL   fn_beg
        LDI     DEVSLOT
        FCALL   fn_pb
        LDI     FMREAD
        FCALL   fn_pb
        LDR     R8, V_TMO
        LDI     66                      ; past the cart's 60 s for MOUNT_IMAGE
        STR     R8
        STEP    3
        FJMP    stage
```

App Keys, Clock and Utilities





app keys · the clock · hashes and ids

A *Studio II* forgets everything when it is turned off. The FujiNet does not: it keeps small *app keys* on its SD card for any program that asks, and it knows the time.

App keys

An app key is up to 64 bytes, named by three numbers: a *creator*, two bytes, an *app* and a *key* number. On the SD card it is the file `/FujiNet/CCCCAAKK.key`. Creator 1, app 1, key 0 is special: every FujiNet game reads the player's name from it, so a name typed once is known everywhere.

`appkey.asm` reads the player's name, then writes a key of the handbook's own, creator `$1802`, app `$FF`, key 0, and reads it back.

CDP1802 ASSEMBLY (AS) · APPKEY.ASM

```
go:     PUTS    20H, t_name  ; the user name
        LDR     RA, k_name
        FCALL   akopen
        FCALL   akread
        LDI     30H
        FCALL   akshow
        PUTS    50H, t_ours  ; ours: write it...
        LDR     RA, k_ourw
        FCALL   akopen
        FCALL   akwrite
        LDR     RA, k_ourr   ; ...read it back
        FCALL   akopen
        FCALL   akread
        LDI     60H
        FCALL   akshow
hang:   BR      hang
```

Opening a key

OPEN_APPKEY chooses the key the next read or write uses. It has no parameters: the key goes as a six-byte payload, the creator low byte first, then the app, the key, the mode (0 read, 1 write) and a reserved 0.

CDP1802 ASSEMBLY (AS) · APPKEY.ASM

```
akopen: FNBEGIN FNDEVF, FNCAKOP, 0
        LDI     6
        PLO     RB
ako1:   LDA     RA
        FCALL   fn_txb
        DEC     RB
        GLO     RB
        BNZ     ako1
        GOACK
        FRET
```

IMPORTANT:
Send the reserved byte. Without it the adapter waits for the sixth byte until the transaction times out.

Reading and writing

READ_APPKEY answers with a two-byte length and then the value; the program prints the value from offset 2 of the reply window.
WRITE_APPKEY takes the length as one parameter and the bytes as the payload. A write ends the selection: open the key again before the next read.

CDP1802 ASSEMBLY (AS) · APPKEY.ASM

```
akwrite:
        FNBEGIN FNDEVF, FNCAKWR, 1
        LDI     VALLEN
        FCALL   fn_pb
        LDR     RA, val
        LDI     VALLEN
        PLO     RB
akw1:   LDA     RA
        FCALL   fn_txb
        DEC     RB
        GLO     RB
        BNZ     akw1
        GOACK
        FRET
```

The clock

The FujiNet's clock is a device of its own, `$45`. GET_ISO_LOCAL (`$49`, an `I`) answers with the local time as text, `2026-10-20T09:57:25` and the zone's offset; GET_ISO_UTC (`$5A`, a `Z`) the same in UTC. `clock.asm` prints the date and the time on two rows, straight out of the reply window:

CDP1802 ASSEMBLY (AS) · CLOCK.ASM

```
ASK     MACRO   cmd, rc
        FNBEGIN CLKDEV, cmd, 0
        GOACK
        LDI     rc
        FCALL   d_at
        LDR     R8, 0           ; YYYY-MM-DD
        LDI     10
        FCALL   d_rpl
        LDI     rc+10H
        FCALL   d_at
        LDR     R8, 11          ; HH:MM:SS
        LDI     8
        FCALL   d_rpl
        ENDM
```

A `fujinet-pc` knows no zone of its own until it is given one, and its local time is UTC. So `clock.asm` first sets one, with command `$74` (a `t`) and a POSIX zone as the payload:

CDP1802 ASSEMBLY (AS) · CLOCK.ASM

```
go:     FNBEGIN CLKDEV, CKSETZA, 0
        LDR     R8, tz          ; the payload
        FCALL   fn_str
        GOACK
```

NOTE:
`$99`, which `fujiCommandID.h` calls SETTZ, sets the *alternate* zone: the one a clock read uses when it is given a parameter of 1. `$74` sets the adapter's own, and the adapter saves it.

GETTIME (`$93`) answers in binary instead: six bytes, day, month, year (two digits), hour, minute, second.

Utilities

The FUJI device has a few services no *Studio II* could afford to do itself:

Command	Gives
GENERATE_GUID `$BB`	a fresh unique id, 36 characters
HASH `$C8`–`$C5`	MD5, SHA-1, SHA-256 or SHA-512 of the bytes you feed it
BASE64 `$D0`–`$C9`	base64, either way

A hash is four transactions: feed the bytes with HASH_INPUT, as often as needed, choose the algorithm with HASH_COMPUTE, and read the result with HASH_OUTPUT, as text or binary.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCHSHI, 1  ; feed it
        LDR     R8, 5
        FCALL   fn_pw
        LDR     R8, data  ; "HELLO"
        FCALL   fn_str
        GOACK
        FNBEGIN FNDEVF, FNCHSHC, 1  ; SHA-256
        LDI     HASHA256
        FCALL   fn_pb
        GOACK
        FNBEGIN FNDEVF, FNCHSHO, 1  ; as hex text
        LDI     1
        FCALL   fn_pb
        GOACK  ; 64 characters
```

The Command Reference has all of them (The Fuji Device, page 30).

Glyphs, Tiles and Mosaics



32 glyphs · 16 tiles · mosaic cells

A program cannot store into the picture, but the text engine draws more than letters. It holds 32 characters of your own, and 16 small tiles that build cells of a finer grain. Battleship draws its four boards with them, and 5 Card Stud its cards. `glyphs.asm` shows each kind.

Glyphs

Codes `$80` to `$9F` are yours to define. A glyph is a whole cell, 4 dots by 12 scanlines, with no blank line or column around it: put two side by side, or one above another, and they join. Push the code, then the twelve rows, top first, each row a byte whose bit 3 is the leftmost dot; then GLYPH (`$40`).

CDP1802 ASSEMBLY (AS) · GLYPHS.ASM

```
        LDR     RA, g_mark
        LDI     4
        PLO     RB
gl1:    LDI     13              ; code + 12 rows
        FCALL   params
        LDI     TOP_GLYPH
        FCALL   d_op
        DEC     RB
        GLO     RB
        BNZ     gl1
```

`params` pushes D bytes from RA, so a glyph is a table:

CDP1802 ASSEMBLY (AS) · GLYPHS.ASM

```
g_mark: DB      80H  ; top left
        DB      1,1,1,1,1,1,0,0,0,0CH,0CH,0CH
        DB      81H  ; top right
        DB      8,8,8,8,8,8,0,0,0,3,3,3
        DB      82H  ; bottom left
        DB      0CH,0CH,0CH,0,0,0,0,0,0,0,0,0
        DB      83H  ; bottom right
        DB      3,3,3,0,0,0,3,3,3,3,3,3
```

Four glyphs make an 8 × 24 dot mark: dots of two by six, which look round on a television. Redefining a glyph redraws every cell already holding it, which makes them good for animation and poor for a picture of many different parts.

Tiles and mosaics

For finer pictures there are 16 tiles, each 2 dots by 4 scanlines, four row bytes with bit 1 the left dot. A *mosaic* cell is two tiles across and three down, six tiles in one character cell, and any cell may hold any six. TILE (`$41`) takes the tile's number and its four rows:

CDP1802 ASSEMBLY (AS) · GLYPHS.ASM

```
t_tiles: DB     0, 0, 0, 0, 0
         DB     1, 3, 3, 3, 3
         DB     2, 2, 1, 2, 1
```

MOSAIC (`$42`) draws cells at the cursor and moves on, like PUTC. Each takes three bytes, its top, middle and bottom thirds, and each byte holds two tiles, the left in the high nibble. Up to five cells go in one operation:

CDP1802 ASSEMBLY (AS) · GLYPHS.ASM

```
m_cells: DB     22H, 22H, 11H
         DB     22H, 11H, 11H
         DB     11H, 11H, 11H
         DB     22H, 11H, 11H
         DB     22H, 22H, 11H
```

Redefining a tile redraws only the mosaic cells that use it. SCROLL carries mosaic cells along, PUTC and CLS replace them, and inverse video inverts the whole cell.

Repeats and inverse video

REPEAT (`$30`) takes a character and a count and draws it that many times, with one read for each parameter and one for the operation:

CDP1802 ASSEMBLY (AS) · GLYPHS.ASM

```
        LDI     SYM_BLOCK  ; REPEAT char, count
        FCALL   d_par
        LDI     6
        FCALL   d_par
        LDI     TOP_REPEAT
```

INV_ON and INV_OFF bracket what is drawn in inverse video; CLS turns it off.

The marquee

A row armed with MARQUEE (`$20`) scrolls a PRINT that is too long for it, a step every quarter of a second, up to 64 characters and three blanks, round and round. `fujitest` uses one for the network's name. One row at a time can have it; MQ_OFF (`$21`) or CLS stops it.

NOTE:
While a transaction is running the cartridge is busy, and the marquee stands still until it is done.

Command Reference: The Fuji Device

Every command device `$70` answers, checked against the firmware the cartridge's ESP32-S3 runs: the RS-232 build's `rs232Fuji.cpp` and the four mixins every FujiNet shares (app keys, base64, hashing, QR codes), at the revision in the Parts List. On the cards, *nparam* is the value for the NPARAM register, and each parameter goes into the TX stream as its size and then its value: a *byte* is 1 and the byte, a *word* 2 and two bytes, low first. The reply is in the reply window at `$E400`.

The examples use the runtime, the handbook's `s2defs.inc` and `booklib.inc`, and labels `failed` and `done` of the program's own. Every one of them is assembled by `tools/snipcheck.py`.

WiFi and the adapter

GET_ADAPTERCONFIG_EXTENDED / GET_ADAPTERCONFIG
$C4 / $E8

DEVICE	$70
NPARAM	0
PARAMS	none
PAYLOAD	none
REPLY	`$C4`: 240 bytes, laid out in First Contact. `$E8`: the first 140, the addresses in binary only

The first-contact command, and the one every status screen wants. It works before the WiFi is up, and its text fields go straight to the screen with PRINT.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCADPX, 0
        FCALL   fn_go
        BNZ     failed
        LDR     R8, 140         ; the IP address, as text
        LDI     15
        FCALL   d_rpl           ; the cart prints it
```

SCAN_NETWORKS
$FD

DEVICE	$70
NPARAM	0
PARAMS	none
PAYLOAD	none
REPLY	1 byte: how many networks were found

Starts a fresh scan and answers when it is done, which takes seconds: well inside the cartridge's five. Follow it with GET_SCAN_RESULT once for each network.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCSCAN, 0
        FCALL   fn_go
        BNZ     failed
        PEEK    FN_REPLY        ; how many
        PLO     RA
```

GET_SCAN_RESULT
$FC

DEVICE	$70
NPARAM	1
PARAMS	byte: which result, from 0
PAYLOAD	none
REPLY	34 bytes: the SSID (33, NUL-terminated), then the signal strength, a signed dBm

The strength is negative: closer to zero is stronger.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCSCNR, 1
        GLO     RA              ; which network
        FCALL   fn_pb
        FCALL   fn_go
        BNZ     failed
        PEEK    FN_REPLY+33     ; strength: negative dBm
```

SET_SSID
$FB

DEVICE	$70
NPARAM	1
PARAMS	byte: any value; **required but ignored**
PAYLOAD	**exactly 97 bytes**: the SSID (33) then the password (64), each NUL-padded
REPLY	ACK

Joins a network and saves it as the default. Two sharp edges: the parameter must be sent although nothing reads it, and the payload must be all 97 bytes; a short one fails the transaction.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCSSSD, 1
        LDI     0
        FCALL   fn_pb           ; must be there
        LDR     RA, ssidpw      ; ssid[33] + password[64]
        LDI     97
        PLO     RB
pw:     LDA     RA
        FCALL   fn_txb
        DEC     RB
        GLO     RB
        BNZ     pw
        FCALL   fn_go
```

GET_SSID
$FE

DEVICE	$70
NPARAM	0
PARAMS	none
PAYLOAD	none
REPLY	97 bytes: the SSID (33) then the password (64)

The network the adapter is set to join, password in clear text.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCGSSD, 0
        FCALL   fn_go
        BNZ     failed          ; SSID at $E400, password +33
```

GET_WIFISTATUS / GET_WIFI_ENABLED
$FA / $EA

DEVICE	$70
NPARAM	0
PARAMS	none
PAYLOAD	none
REPLY	`$FA`: 1 byte, 3 when connected; `$EA`: 1 byte, 1 if the radio is on

What to poll while a join is in progress. CONFIG polls it once a second.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCWIFI, 0
        FCALL   fn_go
        BNZ     failed
        PEEK    FN_REPLY
        XRI     3               ; connected?
        BZ      done
```

RESET
$FF

DEVICE	$70
NPARAM	0
PARAMS	none
PAYLOAD	none
REPLY	ACK, then the adapter restarts

Restarts the ESP32-S3. The cartridge, its mailbox and your program carry on; the link bit of STAT, `$E801`, drops and comes back, and the cartridge waits up to three seconds for it before the next transaction.

STATUS / DEVICE_READY
$53 / $00

DEVICE	$70
NPARAM	1 / 0
PARAMS	STATUS: byte, the kind of status wanted (1: mount times)
PAYLOAD	none
REPLY	STATUS 1: four bytes per device slot, a time, 0 if nothing is mounted; DEVICE_READY: the bus test's answer

STATUS needs its parameter on this build. DEVICE_READY is a round trip with no other effect: it proves the mailbox, the cartridge, the USB link and the adapter at once.

Hosts, slots and mounting

READ_HOST_SLOTS
$F4

DEVICE	$70
NPARAM	0
PARAMS	none
PAYLOAD	none
REPLY	256 bytes: eight host names of 32 bytes

Slot 0 is the SD card on a standard `fujinet-pc`.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCRHST, 0
        FCALL   fn_go
        BNZ     failed
        LDI     20H             ; host 0's name, row 2
        FCALL   d_at
        LDR     R8, 0           ; reply + slot * 32
        LDI     16
        FCALL   d_rpl
```

WRITE_HOST_SLOTS
$F3

DEVICE	$70
NPARAM	0
PARAMS	none
PAYLOAD	256 bytes: all eight slots, 32 bytes each
REPLY	ACK

All or nothing: there is no command to write one slot. Read the table with READ_HOST_SLOTS and stream it back with the one slot changed; the reply window holds still until the next launch, so the stream can be copied straight out of it with LDA. CONFIG renames a host this way.

MOUNT_HOST
$F9

DEVICE	$70
NPARAM	1
PARAMS	byte: the host slot, from 0
PAYLOAD	none
REPLY	ACK; NAK if the host cannot be reached

Connects the host: SD, TNFS, SMB and the rest. Needed before OPEN_DIRECTORY or a mount. UNMOUNT_HOST (`$E6`) is not answered on this build.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCMHST, 1
        LDI     0               ; host slot 0
        FCALL   fn_pb
        FCALL   fn_go
        BNZ     failed
        FCALL   fn_ack
        BNZ     failed
```

READ_DEVICE_SLOTS / WRITE_DEVICE_SLOTS
$F2 / $F1

DEVICE $70
NPARAM 0
PARAMS none
PAYLOAD none
REPLY `$F2`: the device-slot table: host slot, mode and file name for each slot; `$F1` takes the same table as its payload

What is set up in each of the FujiNet's virtual drives. A *Studio II* boots images rather than reading drives, so CONFIG does not show them, but they answer.

SET_DEVICE_FULLPATH
$E2

DEVICE $70
NPARAM 3
PARAMS bytes: device slot, host slot, mode (1 to read)
PAYLOAD **256 bytes**: the path, NUL-padded
REPLY ACK

Names the file for a device slot: the first half of booting.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCSDFP, 3
        LDI     0               ; device slot
        FCALL   fn_pb
        LDI     0               ; host slot
        FCALL   fn_pb
        LDI     FMREAD
        FCALL   fn_pb
        LDR     R8, file
        FCALL   fn_path         ; padded to 256
        GOACK
```

MOUNT_IMAGE / UNMOUNT_IMAGE
$F8 / $E9

DEVICE $70
NPARAM 2 / 1
PARAMS `$F8`: device slot, mode (1 to read); `$E9`: device slot
PAYLOAD none
REPLY ACK when the mount completes

The second half of booting. Before it answers, the FujiNet pushes the file into the cartridge, and BOOT_STATE and BOOT_PCT report the progress; the cartridge allows sixty seconds. Directories, Mount and Boot has the rest.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCMIMG, 2
        LDI     0               ; device slot
        FCALL   fn_pb
        LDI     FMREAD
        FCALL   fn_pb
        SLOW    66              ; past the cart's 60
        GOACK
push:   PEEK    FN_BOOTSTATE    ; the image, staged?
        XRI     FN_BOOT_READY
        BNZ     push
```

MOUNT_ALL / SET_BOOT_MODE / CONFIG_BOOT
$D7 / $D6 / $D9

DEVICE $70
NPARAM 0 / 1 / 1
PARAMS SET_BOOT_MODE: byte, the mode; CONFIG_BOOT: byte, on or off
PAYLOAD none
REPLY ACK

MOUNT_ALL mounts every slot that has a file; the other two choose what the adapter serves at power-on on computers that boot from its drives. On the *Studio II* the cartridge decides what boots, and CONFIG does not send them.

Directories

OPEN_DIRECTORY
$F7

DEVICE $70
NPARAM 1
PARAMS byte: the host slot
PAYLOAD **256 bytes**: the path, a NUL, then an optional filter such as `*.ST2`, the rest NULs
REPLY ACK

Opens a directory on a mounted host. The adapter applies the filter, so a program never sees what it does not want.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCODIR, 1
        LDI     0               ; host slot
        FCALL   fn_pb
        LDR     R8, path
        FCALL   fn_path
        FCALL   fn_go
        BNZ     failed
```

READ_DIR_ENTRY
$F6

DEVICE $70
NPARAM 2
PARAMS bytes: the longest name wanted (**never 31**), flags (0)
PAYLOAD none
REPLY one name, NUL-terminated, a `/` at the end of a directory's; **`$7F`, `$7F` when there are no more**

One name a call; the adapter moves on by itself. At a length of exactly 31 the firmware puts icon bytes in front of the name. Stop at the end marker: an SD host keeps answering `..` past it. CONFIG reads a whole page of names in one call, with flag bits that pack several entries into the reply.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCRDIR, 2
        LDI     15
        FCALL   fn_pb
        LDI     0
        FCALL   fn_pb
        FCALL   fn_go
        BNZ     failed
        PEEK    FN_REPLY
        XRI     7FH             ; the end?
        BZ      done
```

SET_DIRECTORY_POSITION / GET_DIRECTORY_POSITION
$E4 / $E5

DEVICE	$70
NPARAM	1 / 0
PARAMS	$E4: **one word**, the entry number
PAYLOAD	none
REPLY	$E5: a word, the current entry number; $E4: ACK

Seek and tell for the open directory: paging in one command. The position is one two-byte parameter, not two one-byte ones.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCSDPS, 1
        LDR     R8, 8           ; entry 8: page two
        FCALL   fn_pw
        FCALL   fn_go
```

CLOSE_DIRECTORY
$F5

DEVICE	$70
NPARAM	0
PARAMS	none
PAYLOAD	none
REPLY	ACK

Close what you open: the adapter has room for few open directories.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCCDIR, 0
        FCALL   fn_go
```

Files, prefixes and paths

COPY_FILE
$D8

DEVICE	$70
NPARAM	2
PARAMS	bytes: source host, destination host, **counting from 1**
PAYLOAD	source|destination; at its own length, or NUL-padded
REPLY	ACK when the copy is done; the cartridge allows sixty seconds

Host to host, without a byte passing through the console. Its hosts count from 1, unlike every other slot number; 0 is refused. When the destination ends in /, the adapter adds the source's file name. CONFIG's Copy screen sends it at its own length.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCCOPY, 2
        LDI     1               ; host 0, from 1
        FCALL   fn_pb
        LDI     1
        FCALL   fn_pb
        LDR     R8, spec        ; source|destination
        FCALL   fn_str
        SLOW    66
        GOACK
```

SET_HOST_PREFIX / GET_HOST_PREFIX
$E1 / $E0

DEVICE	$70
NPARAM	1
PARAMS	byte: the host slot; SET carries the prefix as its payload
PAYLOAD	none
REPLY	GET: the prefix

A working directory per host, put in front of relative paths.

GET_DEVICE_FULLPATH
$DA

DEVICE	$70
NPARAM	1
PARAMS	byte: the device slot
PAYLOAD	none
REPLY	the path SET_DEVICE_FULLPATH stored

What a device slot holds, as one string.

App keys

OPEN_APPKEY
$DC

DEVICE	$70
NPARAM	0
PARAMS	none
PAYLOAD	6 bytes: creator (word), app, key, mode (0 read, 1 write), and a reserved 0
REPLY	ACK; NAK with no SD card, a creator of 0 or a bad mode

Chooses the key the next read or write uses. The payload is one packed structure; leave off the reserved byte and the adapter waits for it until the transaction times out. On the SD card the key is the file `/FujiNet/CCCCAAKK.key`; it holds 64 bytes at most.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCAKOP, 0
        LDI     02H             ; creator $1802, low first
        FCALL   fn_txb
        LDI     18H
        FCALL   fn_txb
        LDI     0FFH            ; app
        FCALL   fn_txb
        LDI     0               ; key
        FCALL   fn_txb
        LDI     AKMRD           ; mode
        FCALL   fn_txb
        LDI     0               ; reserved: required
        FCALL   fn_txb
        GOACK
```

READ_APPKEY / WRITE_APPKEY / CLOSE_APPKEY
$DD / $DE / $DB

DEVICE	$70
NPARAM	0 / 1 / 0
PARAMS	WRITE: byte, the length
PAYLOAD	WRITE: the value, at most 64 bytes
REPLY	READ: a word, the length, then the value; the others: ACK

A write ends the selection, so each write needs its own OPEN in write mode. App Keys, Clock and Utilities has both directions.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCAKRD, 0
        FCALL   fn_go
        BNZ     failed
        PEEK    FN_REPLY        ; the length
        BZ      done            ; empty
        LDR     R8, 2           ; the value: offset 2
        PEEK    FN_REPLY
        FCALL   d_rpl
```

Utilities

GENERATE_GUID
$BB

DEVICE	$70
NPARAM	0
PARAMS	none
PAYLOAD	none
REPLY	36 characters and a NUL

A fresh, printable, unique id: 36 characters, more than two rows of the screen. RANDOM_NUMBER ($D3) is not answered on this build; the adapter's randomness reaches a program through a GUID or a hash.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCGUID, 0
        FCALL   fn_go
        BNZ     failed
        LDI     40H             ; rows 4 and 5, wrapped
        FCALL   d_at
        LDR     R8, 0
        LDI     32
        FCALL   d_rpl
```

HASH: INPUT, COMPUTE, LENGTH, OUTPUT
$C8 / $C7 / $C6 / $C5

DEVICE	$70
NPARAM	1
PARAMS	INPUT: word, the byte count; COMPUTE: byte, the algorithm (0 MD5, 1 SHA-1, 3 SHA-256, 4 SHA-512); LENGTH and OUTPUT: byte, 1 for hex text, 0 for binary
PAYLOAD	INPUT: the bytes
REPLY	LENGTH: the size; OUTPUT: the hash

Four steps on the adapter: feed bytes with INPUT, as often as needed; COMPUTE; ask the LENGTH; read the OUTPUT. App Keys, Clock and Utilities has the whole sequence. HASH_COMPUTE_NO_CLEAR ($C3) keeps the input for another algorithm, and HASH_CLEAR ($C2) empties it.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCHSHL, 1
        LDI     1               ; as text
        FCALL   fn_pb
        FCALL   fn_go
        BNZ     failed
        PEEK    FN_REPLY        ; 64 for SHA-256
```

BASE64: ENCODE and DECODE
$D0–$CD / $CC–$C9

DEVICE	$70
NPARAM	varies
PARAMS	INPUT: word, the byte count; OUTPUT: word, how many bytes to read
PAYLOAD	INPUT: the bytes
REPLY	LENGTH: 4 bytes, the size; OUTPUT: the result

The same four steps as the hash. Encoding is INPUT $D0, COMPUTE $CF, LENGTH $CE, OUTPUT $CD; decoding is $CC, $CB, $CA, $C9.

QR CODE: INPUT, ENCODE, LENGTH, OUTPUT

$BC / $BD / $BE / $BF

DEVICE	$70
NPARAM	varies
PARAMS	INPUT: word, the count; ENCODE: version (0 to choose), error correction (0–3), shorten; LENGTH: the output mode (0 bytes, 2 bitmap); OUTPUT: word, the count
PAYLOAD	INPUT: the text
REPLY	LENGTH: 4 bytes, the size; OUTPUT: the modules

The adapter draws a QR code's matrix; a program only has to plot it. A version 1 code is 21 × 21 modules: with a mosaic cell of 2 × 3 squares, 11 cells by 7.

Answered with a NAK on this build

These are in `fujiCommandID.h` but fall through every table the RS-232 build dispatches, so they earn a NAK: `ENABLE_UDPSTREAM` (`$F0`), `SET_BAUDRATE` (`$EB`), `NEW_DISK` (`$E7`), `UNMOUNT_HOST` (`$E6`), `SET_HSIO_INDEX` (`$E3`), `SET_SIO_EXTERNAL_CLOCK` (`$DF`), `ENABLE_DEVICE` and `DISABLE_DEVICE` (`$D5`, `$D4`), `RANDOM_NUMBER` (`$D3`), `GET_TIME` (`$D2`; use the clock device), `DEVICE_ENABLE_STATUS` (`$D1`), `GET_HEAP` (`$C1`), `QRCODE_CLEAR` (`$BA`), `GET_DEVICE1..10_FULLPATH` (`$A0`–`$A9`), `UPDATE_FIRMWARE` (`$90`) and `HSIO_INDEX` (`$3F`). Most tune other computers' buses.

Command Reference: The Network Device

Every command devices `$71`–`$78` answer, checked against the network device every FujiNet shares, `NDevice.cpp`. Most of the codes are the ASCII letter of their name, `'O'`pen, `'R'`ead, `'W'`rite, a habit from the Atari 8-bits. What a devicespec can say, and how each protocol behaves, belongs to *The FujiNet Network Protocol Handbook*; these are the cards. A READ's answer must fit the 1K reply window, and a WRITE's bytes the 320-byte TX stream with the parameter in front of them.

Each network command may take the cartridge 90 seconds, so the examples raise the wait with `SLOW 95`. They use channel 1, `NETDEV` (`$71`).

The lifecycle

NET_OPEN
$4F 'O'

DEVICE	$71
NPARAM	2
PARAMS	bytes: mode (4 read, 8 write, 12 read and write, 13 POST), translation (0 none, 1 CR, 2 LF, 3 CR LF)
PAYLOAD	the devicespec, `N:` and the URL
REPLY	ACK once the connection is set up

On HTTP, modes 4 and 12 are both GET; 12 also lets a program set headers (NET_SET_HTTP_MODE). The request itself waits for the first STATUS or READ. Other HTTP modes: 14 PUT with headers, 5 and 9 DELETE. On TCP, mode 12 is a socket both ways, as the netcat uses it. Opening a channel that is already open closes the old connection first.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN NETDEV, NCOPEN, 2
        LDI     NMHTTPG         ; 12
        FCALL   fn_pb
        LDI     NTRNONE
        FCALL   fn_pb
        LDR     R8, url         ; as long as it is
        FCALL   fn_str
        SLOW    95
        GOACK
```

NET_STATUS
$53 'S'

DEVICE	$71
NPARAM	2
PARAMS	two bytes of 0
PAYLOAD	none
REPLY	4 bytes: bytes waiting (word), connected, status (1 good, 136 end of file)

The heartbeat. A terminal watches *connected*; a program that fetches and shows watches the status for end of file; every program reads the count, until two readings agree (The Network Device).

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN NETDEV, NCSTAT, 2
        LDI     0
        FCALL   fn_pb
        LDI     0
        FCALL   fn_pb
        SLOW    95
        GOACK
        PEEK    FN_REPLY+NSAVLO ; bytes waiting
        STR     R2
        PEEK    FN_REPLY+NSAVHI
        OR
        BZ      done            ; nothing yet
```

NET_READ
$52 'R'

DEVICE	$71
NPARAM	1
PARAMS	**one word**: how many bytes, no more than are waiting and no more than 1024
PAYLOAD	none
REPLY	that many bytes

A short or failed read is a NAK. Use the bytes, or copy them, before the next transaction paints over the window.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN NETDEV, NCREAD, 1
        LDR     R8, 128         ; 128 bytes
        FCALL   fn_pw
        SLOW    95
        GOACK                   ; RXLEN bytes at $E400
```

NET_WRITE
$57 'W'

DEVICE	$71
NPARAM	1
PARAMS	word: how many bytes
PAYLOAD	that many bytes
REPLY	ACK

The count goes twice: as the parameter, and as the length of the payload. The two parameter bytes and the count leave 317 bytes for the data in one transaction.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN NETDEV, NCWRITE, 1
        LDR     R8, 5
        FCALL   fn_pw           ; the count...
        LDR     R8, data        ; ...and the bytes
        FCALL   fn_str
        SLOW    95
        GOACK
```

NET_CLOSE
$43 'C'

DEVICE	$71
NPARAM	0
PARAMS	none
PAYLOAD	none
REPLY	ACK

Close every connection you open, including on the way out of a failure; the adapter keeps it open otherwise, even after the console is turned off.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN NETDEV, NCCLOSE, 0
        SLOW    95
        GOACK
```

Parsers, parameters and line endings

NET_SET_PARSER / NET_PARSE / NET_QUERY

$FC / $50 'P' / $51 'Q'

DEVICE	$71
NPARAM	2 / 0 / 0
PARAMS	SET_PARSER: byte (any), then the parser: 0 none, 1 JSON, 2 HTML, 3 XML
PAYLOAD	QUERY: the path, such as `/0/name`
REPLY	QUERY: ACK; then STATUS and READ the value as text

Switch an open connection to one of the adapter's parsers (the parser is the **second** parameter), parse the body, then ask for one value by its path.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN NETDEV, NCPRSR, 2
        LDI     0
        FCALL   fn_pb
        LDI     NPJSON          ; the second one
        FCALL   fn_pb
        GOACK
        FNBEGIN NETDEV, NCPARSE, 0
        SLOW    95
        GOACK
        FNBEGIN NETDEV, NCQUERY, 0
        LDR     R8, query
        FCALL   fn_str
        GOACK                   ; then STATUS, READ
```

NET_SET_PARAMETER

$FB

DEVICE	$71
NPARAM	2
PARAMS	bytes: which (0 the query flags, 1 the line ending), the value
PAYLOAD	none
REPLY	ACK

How the parser answers a query: flags remap characters and choose the output form; the line ending is the byte that ends each value.

NET_SET_HTTP_MODE

$4D 'M'

DEVICE	$71
NPARAM	2
PARAMS	byte (any), then the mode: 0 body, 1 collect headers, 2 get headers, 3 set headers, 4 POST data
PAYLOAD	none
REPLY	ACK; NAK on a connection that is not HTTP

HTTP only. In mode 3 each WRITE sets a request header; in mode 4 a WRITE supplies the body of a POST; in mode 2 a READ returns the headers collected in mode 1. Mode 0 is the page itself. Open with mode 12 to use it.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN NETDEV, NCHTTPM, 2
        LDI     0
        FCALL   fn_pb
        LDI     3               ; WRITE sets headers
        FCALL   fn_pb
        GOACK
```

NET_SEEK / NET_TELL
$25 '%' / $26 '&'

DEVICE $71
NPARAM 1 / 0
PARAMS SEEK: **one four-byte parameter**, the offset
PAYLOAD none
REPLY TELL: 4 bytes, the position

Random access where the protocol has it: TNFS and other files, HTTP ranges. The runtime has no four-byte helper: stream a 4 with `fn_txb`, then the four bytes, low first.

NET_TRANSLATION / NET_SET_EOL / NET_SET_INT_RATE
$54 'T' / $4C 'L' / $5A 'Z'

DEVICE $71
NPARAM 2 / 0 / 1
PARAMS TRANSLATION: byte (any), then the code; SET_INT_RATE: the rate
PAYLOAD SET_EOL: the line ending, as a string; empty restores the default
REPLY ACK

Line-ending translation and terminators after OPEN. SET_INT_RATE paces an interrupt line on computers that have one; the cartridge has no such line, so the command is accepted and does nothing. Polling ACKSEQ is the only way a *Studio II* program learns a transaction is done.

Files, sockets and credentials

RENAME / DELETE / LOCK / UNLOCK / MKDIR / RMDIR
$20 / $21 / $23 / $24 / $2A / $2B

DEVICE $71
NPARAM 2
PARAMS bytes: mode (4), and 0
PAYLOAD the devicespec; RENAME: `N:…/old,new`
REPLY ACK

File operations on the protocols that have files: TNFS, FTP, SMB and the SD card. Each one opens a connection just for itself and closes it again.

NET_CHDIR / NET_GETCWD
$2C / $30

DEVICE $71
NPARAM 0
PARAMS none
PAYLOAD CHDIR: the path: absolute, relative, .., or a whole new `SCHEME://host/`
REPLY GETCWD: the prefix

A working directory for the channel, put in front of every devicespec that does not name its own host.

NET_ACCEPT / NET_CLOSE_CLIENT

$41 'A' / $63 'c'

DEVICE	$71
NPARAM	0
PARAMS	none
PAYLOAD	none
REPLY	ACK

For a TCP server: a channel opened on `N:TCP://:port/` listens; ACCEPT takes the waiting client, and CLOSE_CLIENT hangs up on that client while the channel keeps listening. A *Studio II* can be the bulletin board.

NET_GET_REMOTE / NET_SET_DESTINATION / NET_USERNAME / NET_PASSWORD

$72 'r' / $44 'D' / $FD / $FE

DEVICE	$71
NPARAM	0
PARAMS	none
PAYLOAD	SET_DESTINATION: `host:port`; USERNAME and PASSWORD: the credential
REPLY	GET_REMOTE: who sent the last UDP datagram

UDP's "who said that" and "talk to them instead", and credentials for protocols that log in (FTP, SMB and others). Send the credentials before the OPEN.

Answered with a NAK on this build

In `fujiCommandID.h` but not in the network device's table: `NET_GET_DSTATS_VALUE` (`$FF`), `NET_SET_UNIT` (`$FA`), `NET_SET_HSIO_INDEX` (`$E3`), `NET_QUERY_ALT` and `NET_PARSE_ALT` (`$81`, `$80`), `NET_GET_ERROR` (`$45`; STATUS's fourth byte says what went wrong), and `NET_HSIO_INDEX` (`$3F`).

Command Reference: Disk, Printer, Clock and Modem

The mailbox is a transport, not a translator: whatever byte goes into the DEVICE register goes onto the wire. Beyond the FUJI and network devices, the FujiNet answers four more for a *Studio II*: disk drives, a printer, a clock and a Hayes-style modem. The clock is exercised by a program in this handbook, and its replies below were measured on the bench. The others are reachable, transcribed from the firmware's handlers, and offered as such.

Disk: devices $31–$38

Eight virtual drives, $31 plus the drive number, each serving the image mounted in its device slot. The sector size is the image's. On the *Studio II* a mount also pushes the image into the cartridge; reading it as a disk as well is possible, and a way for a program to read a file a sector at a time.

DISK_READ / DISK_WRITE / DISK_PUT
$52 / $57 / $50

DEVICE	$31
NPARAM	1
PARAMS	the sector number, as one parameter (two or four bytes)
PAYLOAD	WRITE and PUT: one sector
REPLY	READ: one sector; WRITE and PUT: ACK

With nothing mounted, READ answers NAK. WRITE verifies what it wrote; PUT does not.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN DSKDEV, 52H, 1  ; DISK_READ, drive 1
        LDR     R8, 1           ; sector 1
        FCALL   fn_pw
        GOACK                   ; the sector at $E400
```

DISK_STATUS
$53

DEVICE	$31
NPARAM	0 or 1
PARAMS	optional byte: the kind of status
PAYLOAD	none
REPLY	4 bytes: drive status, controller status, format timeout, 0

The Atari-style status block, from whatever image is mounted.

DISK_FORMAT / FORMAT_MEDIUM / PERCOM_READ / PERCOM_WRITE
$21 / $22 / $4E / $4F

DEVICE	$31
NPARAM	0
PARAMS	none
PAYLOAD	none
REPLY	FORMAT: a map of bad sectors; PERCOM_READ: a 12-byte geometry block

Atari-drive commands the RS-232 build keeps; on an image they do little.

Printer: device $40

PRINTER_WRITE / PRINTER_PUT / PRINTER_STATUS
$57 / $50 / $53

DEVICE	$40
NPARAM	0 / 0 / 1
PARAMS	STATUS: byte, required but ignored
PAYLOAD	WRITE and PUT: the bytes to print, fewer than 255
REPLY	WRITE and PUT: ACK; STATUS: 4 bytes

The page goes to a file on the adapter, in the printer it is set to imitate, and comes out through its web interface. A printer switched off in the adapter's settings does not answer at all, and the cartridge reports a timeout.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN PRNDEV, 57H, 0  ; PRINTER_WRITE
        LDR     R8, val
        FCALL   fn_str
        LDI     0DH             ; a carriage return
        FCALL   fn_txb
        GOACK
```

Clock: device $45

GETTIME / GET_ISO_LOCAL / GET_ISO_UTC
$93 / $49 'I' / $5A 'Z'

DEVICE	$45
NPARAM	0 or 1
PARAMS	optional byte: 1 for the alternate zone
PAYLOAD	none
REPLY	GETTIME: 6 bytes: day, month, year (two digits), hour, minute, second. The ISO forms: `2026-10-20T09:57:25-0400` and a NUL, 25 bytes

On by default. A read with a parameter of 1 uses the alternate time zone instead of the adapter's own. The lengths below are what the bench's `fujinet-pc` answered.

App Keys, Clock and Utilities prints the ISO forms. GET_ISO_UTC ignores the alternate zone.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN CLKDEV, CKGTIME, 0
        FCALL   fn_go
        BNZ     failed
        PEEK    FN_REPLY+3      ; the hour
        PLO     RA
```

The other formats
$54 $50 $53 $4D $47 $4C $9A

DEVICE $45
NPARAM 0 or 1
PARAMS none
PAYLOAD none
REPLY $54 'T': 7 bytes, century, year, month, day, hour, minute, second; $50 'P': 4 bytes, ProDOS; $53 'S': 19 characters, Apple III SOS; $4D 'M': the 7 bytes and hundredths; $47 'G': the zone's name, NUL-terminated; $4C 'L': one byte, its length; $9A: GETTIME in the alternate zone

Every format the FujiNet's clock knows, and the zone. Each also has a lower-case twin, $69 'i', $7A 'z', $70 'p', $73 's' and $61 'a' (the Atari's own six bytes, also $41), that reads in the alternate zone. $54 doubles as a write command on buses that have one; on this one it only reads.

SETTZ_ALT / SETTZ
$74 't' / $99

DEVICE $45
NPARAM 0
PARAMS none
PAYLOAD a POSIX time-zone string, such as EST5EDT
REPLY ACK

$74 sets the zone the adapter uses for local time, and saves it; $99 sets the **alternate** zone that a read with a parameter of 1 uses. A string that is not a valid zone is dropped without a word.

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN CLKDEV, 74H, 0  ; the adapter's zone
        LDR     R8, tz
        FCALL   fn_str
        GOACK
```

Modem: device $50

A Hayes-style modem that dials TCP instead of telephone lines: ATDT host:port opens a connection, and ATNET1 runs it through telnet negotiation. Ten packet commands set it up and poll it; the characters themselves flow through WRITE and STATUS.

MODEM_STATUS / MODEM_WRITE / MODEM_STREAM
$53 / $57 / $58

DEVICE $50
NPARAM 1 / 0 / 0
PARAMS STATUS: byte, required but ignored
PAYLOAD WRITE: an AT command line, or data once connected
REPLY STATUS: 2 bytes, the second the handshake lines ($CD connected with data waiting, $CC connected, $00 idle); WRITE: ACK; STREAM: a 9-byte baud table

The other seven set the modem up: CONTROL $41, CONFIGURE $42, SET_DUMP $44, LISTEN $4C, UNLISTEN $4D, BAUDRATELOCK $4E, AUTOANSWER $4F; and $3F, $40, $21 and $26 answer the Atari's handler loader and polls. Switched off in the adapter's settings, the modem does not answer.

Command Reference: The Cartridge Itself

Everything the cartridge answers by itself, with the names its firmware gives them: `pico/studio2/firmware/include/fuji_mailbox.h`, `s2_text.h` and the shared `pico/common/include/fuji_mailbox_proto.h`, at the revision in the Parts List. `tools/cartcheck.py` checks that every one of those names is on this page. The runtime's own names, from `fujinet.inc`, are given beside them where they differ.

The arena

Pages `$E4`–`$EF`, served whenever the running image claims the mailbox.

Name	Value	Is
`FN_R_DATA`	`$E400`	the reply window: the reply, `FN_R_SLICE_LEN` (`$400`) bytes; runtime `FN_REPLY`
`FN_R_NSLICES`	1	slices of the reply: one, so the whole reply is always in the window
`FN_R_BASE`	`$E800`	the status page, painted from here to `FN_R_PAINT_END` (`$E8FC`); runtime `FN_STATUS`
REGSEL	`$E9`*nn*	select register nn (`$00`–`$7F`); runtime `FN_REGSEL`
REGDATA	`$EA`*vv*	the selected register = vv; runtime `FN_REGDATA`
TX	`$EB`*bb*	append bb to the TX stream, at most `FN_TX_MAX`, 320; runtime `FN_TXDATA`
stub	`$E9F0`	`$EC $70 $23`: SEX RC; RET; and RET's byte. The hand-over's last step; runtime `FN_STUB`
text	`$EC`–`$EF`	the text engine's four pages, below

The registers

Set by a read of REGSEL, then one of REGDATA. A register is good for one write; a REGDATA read with no REGSEL before it does nothing.

Name	Value	Is
`FN_REG_DEVICE`	`$00`	the FujiBus device
`FN_REG_CMD`	`$01`	the command
`FN_REG_NPARAM`	`$02`	how many parameters lead the TX stream; the cartridge takes 8 at most
`FN_REG_DATA_RST`	`$05`	any value: the TX stream starts over
`FN_REG_RXSLICE`	`$06`	which slice of the reply the window shows: always 0 here
`FN_REG_SEQ`	`$10`	not 0 and not ACKSEQ: launch. ACKSEQ again: nothing
`FN_REG_BOOTLOCK`	`$11`	`FN_BOOTLOCK_MAGIC` (`$B5`): arm the staged image, if BOOT_STATE is READY
`FN_REG_BOOTSEL_1`	`$12`	`FN_BOOTSEL_MAGIC1` (`$B5`), then…
`FN_REG_BOOTSEL_2`	`$13`	…`FN_BOOTSEL_MAGIC2` (`$4A`) at once: the RP2354B restarts into its USB loader, for new firmware
`FN_REG_CONFIG`	`$14`	`FN_CONFIG_MAGIC` (`$C5`): stage the built-in CONFIG and arm it. The *Studio II*'s own

The status page

Painted with every reply; ACKSEQ last. Offsets in the firmware count from the start of the arena, so `FN_R_ACKSEQ` is `$400` there and `$E800` to the console.

Name	Value	Is
`FN_R_ACKSEQ`	`$E800`	the sequence number answered; runtime `FN_ACKSEQ`
`FN_R_STATUS`	`$E801`	bit 0, `FN_R_STATUS_LINK`: the ESP32-S3 link is up. Bit 1, `FN_R_STATUS_BUSY`, is never set on this cartridge; runtime `FN_STAT`
`FN_R_ERR`	`$E802`	the transaction's error, below; runtime `FN_ERR`
`FN_R_REPLY_CMD`	`$E803`	`$06` ACK or `$15` NAK; runtime `FN_REPLYCMD`
`FN_R_RXLEN_LO`, `FN_R_RXLEN_HI`	`$E804`	the reply's length, at most 1024; runtime `FN_RXLEN`
`FN_R_BOOT_STATE`	`$E806`	a staged image: below
`FN_R_BOOT_PCT`	`$E807`	how much of it has come: 0–100
`FN_R_BOOT_ERR`	`$E808`	why it failed: below
`FN_R_MAGIC0`, `FN_R_MAGIC1`	`$E809`	`'F'`, `'N'`: a FujiNet cartridge
`FN_R_PROTO_VER`	`$E80B`	the mailbox's version: 1
`FN_R_SLICE_ECHO`	`$E80C`	the slice the window shows
`FN_R_MODE`	`$E812`	what is being served: 0 the image from power-on (CONFIG), 2 a game, 3 a FujiNet program
`FN_R_LINK`	`$E815`	1 while the ESP32-S3's link is up. Not latched, whatever its comment says: it can fall back to 0
`FN_R_BOOT_GOT0`	`$E816`	image bytes received, three bytes, low first
`FN_R_BOOT_TOT0`	`$E819`	the image's size as the FujiNet declared it, three bytes
`FN_R_ARMED`	`$E81C`	1 once the swap is armed
`FN_R_STAGED`	`$E81D`	the staged image: bit 0 ready, bit 1 it claims the mailbox, bit 2 it is CONFIG
`FN_R_STAGED_CRC`	`$E81F`	its CRC-32, four bytes, low first
`FN_R_SWAPS`	`$E823`	hand-overs since power-on, modulo 256
`FN_R_FRAMES`	`$E824`	picture frames served, modulo 256
`FN_R_PAINT_END`	`$E8FC`	where the painted page ends

Codes

ERR, BOOT_STATE and BOOT_ERR. Error and Status Codes, in Appendix B, has the runtime's codes beside them.

Name	Value	Is
`FN_ERR_OK`	0	the transaction went through
`FN_ERR_NOLINK`	1	the ESP32-S3 is not there
`FN_ERR_TIMEOUT`	2	no reply in time: 5 s, 60 s for a mount or copy, 90 s on a network channel
`FN_ERR_BADFRAME`	3	a parameter size other than 1, 2 or 4, or a reply frame that fails its checks
`FN_ERR_TOOBIG`	4	the request or its reply would not fit the cartridge's buffers
`FN_BOOT_IDLE`	0	nothing staged
`FN_BOOT_XFER`	1	an image is coming
`FN_BOOT_READY`	2	staged, ready to arm

Name	Value	Is
FN_BOOT_FAILED	$80	it failed: BOOT_ERR says why
FN_BOOT_ERR_TOOBIG	1	larger than an ST2 can be, 16,640 bytes
FN_BOOT_ERR_TRUNCATED	2	shorter than the FujiNet said it would be
FN_BOOT_ERR_NOMAP	3	it names a page it may not use
FN_BOOT_ERR_STOREBUSY	4	never on this cartridge

The claim

Name	Value	Is
FN_R_CLAIM_SIG	"FUJI"	at $07FC, FN_R_CLAIM_LEN (4) bytes: the image claims the mailbox; runtime FN_CLAIM, the S2CLAIM macro

The text engine

Pages $EC (cursor), $ED (PUTC), $EE (operation) and $EF (parameter): S2T_H_CURSOR to S2T_H_PARAM, the runtime's TX_CURSOR to TX_PARAM. 16 columns, 10 rows, 16 parameters at most.

Name	Value	Is
S2T_OP_CLS	$00	clear rows 0–9, cursor home, inverse off, marquee off
S2T_OP_INV_ON	$01	inverse video on
S2T_OP_INV_OFF	$02	inverse video off
S2T_OP_CLREOL	$03	blank from the cursor to the end of the row
S2T_OP_SCROLL	$04	rows 1–9 up one, row 9 blank; the cursor stays
S2T_OP_PRINT	$10	offset low, offset high, the most (0: 256): reply text to a NUL
S2T_OP_PRINT_PAD	$11	the same, then spaces to the most
S2T_OP_MARQUEE	$20	the cursor's row scrolls a PRINT too long for it: 250 ms a step, 64 characters
S2T_OP_MQ_OFF	$21	the marquee off
S2T_OP_REPEAT	$30	a character, a count
S2T_OP_GLYPH	$40	a code $80–$9F, then 12 rows, bit 3 the left dot
S2T_OP_TILE	$41	a tile 0–15, then 4 rows, bit 1 the left dot
S2T_OP_MOSAIC	$42	three bytes a cell, up to five cells, at the cursor; the cell reads $A0
S2T_SYM_SPADE	$01	♠
S2T_SYM_HEART	$02	♥
S2T_SYM_DIAMOND	$03	♦
S2T_SYM_CLUB	$04	♣
S2T_SYM_BLOCK	$05	▉
S2T_SYM_RIGHT	$06	▶
S2T_SYM_LEFT	$07	◀
S2T_SYM_UP	$08	▲

Name	Value	Is
S2T_SYM_DOWN	$09	▼
S2T_SYM_TEN	$0A	10, “10” in one cell
S2T_SYM_BACK	$0B	, a card’s back

The image push

MOUNT_IMAGE makes the FujiNet send the file to the cartridge on the same USB link, as three kinds of frame to a device of the cartridge’s own, $FF: an OPEN with the file’s size, WRITEs of the bytes, and a CLOSE. The cartridge answers each, refuses the OPEN of an image too large, and checks the pages when the CLOSE arrives. BOOT_STATE and the progress bar follow it. A program never sends these frames; it only watches the status page.

Reset and power

CLEAR resets the 1802 and the 1861, not the cartridge: the image and ACKSEQ survive, and a swap that is armed is taken on the BIOS’s first fetch. A read of $0000 after 50 µs with no reads at all counts as a restart too. Power off, for longer than 10 ms, resets the cartridge’s bus engine: it lets go of the bus and serves CONFIG at the next power-on.

Battleship



2 to 4 players · either keypad

Fuji Battleship is played across the FujiNet by up to four players at once, people or the server's bots, each with a fleet of five ships on a ten-by-ten sea. One shot lands on every opponent's board at the same square. The *Studio II* port draws all four boards on one screen, every hit, miss and hull of them, with the cartridge's mosaic cells. It is in `fujinet-battleship/studio2/`, and the handbook carries it unchanged in `listings/battleship/`.

Playing it

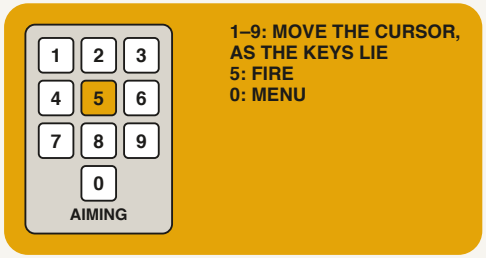


Figure 3. Aiming: the keys move the cursor the way they lie on the pad, diagonals too.

Screen	Keys
Name	2 4 6 8 move on the grid, 5 types, 1 deletes, 3 done, 0 keeps the old name
Tables	1–8 joins that table, 0 the menu
Waiting room	5 ready or not, 0 the menu
Placement	2 4 6 8 move the blinking ship, 1 turns it, 5 keeps it, 3 rolls the rest again, 0 the menu
Your turn	1–9 move the cursor, 5 fires, 0 the menu
Menu	1 or 0 back, 2 leaves the table, 3 changes the name, 4 quits to CONFIG

The fleet starts as a random legal roll, so five presses of 5 is a fleet. A beep marks your turn, and a shorter one every hit and sink.

The screen



Figure 4. The tables, and the waiting room.

Your board is at the bottom left and the others go clockwise from the top left, as in every other port. Each board has its name above it, the player to move in inverse, and the panel at the right shows the clock, whose turn it is, the last shot and the keys.

Four boards on 160 cells

Four boards of a hundred squares will not go in 32 glyphs: two squares by three to a cell, with five states each, is thousands of combinations. The text engine's mosaic cells were added for this game. A square is one tile; `tiles.inc` defines sixteen, and they are chosen so one OR makes a square's tile: the game field's byte (0 unknown, 1 hit, 2 miss) ORed with a mark (hull, cursor, the ship being placed).

`board.inc` composes a board row, fifteen tile pairs, into one MOSAIC operation: four to a board, sixteen to redraw all four, about 7,500 instructions, a tenth of a second. The cursor blinks by redefining three tiles: the cartridge then redraws only the cells that use them.

NOTE:
The boards are lit fields with dark marks. With a dark sea, the boards vanished into the black around them.

The network

`net.inc` is 5 Card Stud's, as is, and carries the rules every FujiNet game client on a small console has learned:

- The CLOSE goes at the **start** of the next request, never after the READ: every transaction repaints the reply window, and the screens draw straight out of it between polls.

CDP1802 ASSEMBLY (AS) · NET.INC

```
; nbeg -- begin an N1: transaction: R9.0 = command, D = parameter count.
; The timeout outlasts the cart's own 90 s, so the cart always answers.
nbeg:   PHI     R8
        LDI     NDEV
        PHI     R9
        GHI     R8
        FCALL   fn_beg
        SETV    V_TMO, NETTMO
        FRET
```

- Take the reply's length right after the READ.
- STATUS reports as soon as *some* of the response is in: poll until two readings agree **and** a known minimum has arrived.
- STATUS's fourth byte must be 1: an HTTP error page has a readable body too.
- Check the length against what the player count implies before believing a byte.

The requests are short paths on the server, `state`, `ready/1`, `place/a,b,c,d,e`, `attack/p`, each with `?table=&player=&bin=1&v=2`. `url.inc` streams the devicespec into the TX stream a byte at a time, percent-encoding the name and the table, and never builds it in RAM: a placement is about 110 of the 320 bytes a transaction carries.

NOTE:
The settle minimum must be the shortest state that can follow the last one. Using the last state's exact length wedged every move from the lobby to placement, because placement's reply is ten bytes shorter.

The server, at home

The server is `fujinet-game-system/battleship` in the FujiNet servers repository, a Go program:

```
cd servers/fujinet-game-system/battleship
PORT=8771 go run .
```

Build the client against it with `ENDPOINT=http://127.0.0.1:8771/ make`. It does not tell the public lobby about itself unless `GO_PROD=1` is set. Test against your own; the public server is where people play.

Played through, in MAME

The game's own `emu/bsdrive.lua` plays a whole game against the local server, with the handbook's bench: it types a name when the app key is empty, joins `AI - 3 BOTS`, readies, moves and turns the carrier, keeps the fleet, fires on every turn, visits the menu, and leaves the table at the end. `emu/games.sh battleship` runs it; the screens in this chapter are its snapshots. It runs MAME at real speed: the server's clocks are real time.

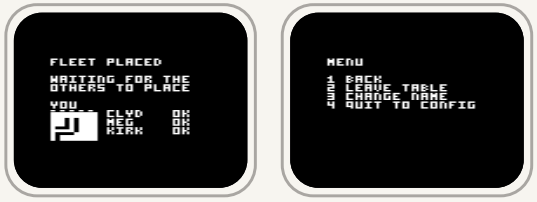


Figure 5. The fleet placed, and the menu.

CDP1802 ASSEMBLY (AS) · TILES.INC

```
tiles:  DB      3, 3, 3, 3              ; 0  T_SEA    ##  ##  ##  ##
        DB      0, 0, 0, 0              ; 1  T_HIT    ..  ..  ..  ..
        DB      3, 1, 1, 3              ; 2  T_MISS   ##  .#  .#  ##
        DB      0, 3, 3, 0              ; 3  T_PIP    ..  ##  ##  ..
        DB      0, 0, 0, 0              ; 4  T_HULL   ..  ..  ..  ..
        DB      2, 1, 2, 1              ; 5  T_HHIT   #.  .#  #.  .#
        DB      0, 0, 0, 0              ; 6  (a miss on a hull cannot be)
        DB      0, 0, 2, 0              ; 7  T_PIPX   ..  ..  #.  ..
        DB      3, 3, 3, 3              ; 8-10, 11: tblink's
        DB      0, 0, 0, 0
        DB      3, 1, 1, 3
        DB      0, 0, 0, 0
        DB      0, 0, 0, 0              ; 12 T_BLANK
        DB      0, 0, 0, 0              ; 13, 14 unused
        DB      0, 0, 0, 0
        DB      2, 1, 2, 1              ; 15: tblink's
```

5 Card Stud



up to 8 players · either keypad

5 Card Stud is the FujiNet's poker table: one card down, four up, a round of betting after each, against people on other computers or the server's bots. It was the first game on the *Studio II* port, and Battleship took its network, name and lobby code. It is in `fujinet-5cardstud/studio2/`; the handbook carries it unchanged in `listings/5cardstud/`.

Playing it

Screen	Keys
Name	2 4 6 8 move on the grid, 5 types, 1 deletes, 3 done, 0 keeps the old name
Lobby	1–8 joins that table, 0 the menu
Table	1–5 sends that move, 9 shows cards or chips, 0 the menu
Menu	1 or 0 back, 2 leaves the table, 3 changes the name, 4 quits to CONFIG

On your turn the bottom two rows become the moves, two to a row: an inverse key, then the move, `1FOLD 2CALL` and `3RAISE5`. A beep says it is your turn.

The screen



Figure 6. The showdown, and the same table with 9 pressed: each seat's chips and last move.

Each seat has a row: its name to the first space, then its five cards, or `FOLDED`, `LEFT` or `WAITING`. Row 8 is the pot, the street and the clock, and row 9 the result while it is news, scrolling under a marquee.

A card is two cells, its rank and its suit from the cartridge's font, ♠ ♥ ♦ ♣, white on black. The ten is the font's own `10` in one cell, **H**. A face-down card is a pair of glyphs the program defines, a hatched back:

Inverse video would be the obvious card face, and it is wrong: on a 3 × 5 letter the outer strokes merge into the gap beside them, and a K loses its stem.

The name

Every FujiNet game reads the player's name from the shared app key, creator 1, app 1, key 0. The editor comes up only when the key is empty or cannot be read, and writes the name back when it is done; a name typed in 5 Card Stud is the name in Battleship. The name and the table go into the URL percent-encoded.

The transport

`net.inc` is the family's, described in Battleship. 5 Card Stud's own numbers: STATUS settles on 154 bytes at least for `/state` and 1 for `/tables`, and the reply must be `154 + 33 × players` or `1 + 36 × tables` long. It waits 100 seconds on a network transaction, past the cartridge's 90. The keypads are read while a transaction runs, so a press during a poll is kept. Leaving a table is `/leave?bin=1`, which has no body: an OPEN and one STATUS.

Size

5,188 bytes on 25 pages: the runtime, then blocks at `$0C00`, `$1400`, `$2400` and `$3400`. The client's variables are all on page `$08`, so RA.1 stays fixed for the whole run, and the stack uses under 40 bytes.

Played through, in MAME

The firmware's `emu/5carddrive.lua` plays it against the bench's server: it types a name when the app key is empty, joins `AI ROOM - 2`, plays legal moves through two hands, then 0 and 4 back to CONFIG, and waits for the cartridge's hand-over. `emu/games.sh 5card` runs it. The server is `fujinet-game-system/5cardstud/server`; `PORT=8772 go run .` starts it.

CDP1802 ASSEMBLY (AS) · CARDS.INC

```
; glyphs -- teach the cart the app glyphs.
        FIT     32
glyphs: LDR     RB, gtab
gly1:   LDA     RB
        BZ      gly3
        FCALL   d_par                   ; the code
        LDI     12
        PLO     RC
gly2:   LDA     RB
        FCALL   d_par                   ; 12 scanlines, bit 3 the leftmost
        DEC     RC
        GLO     RC
        BNZ     gly2
        LDI     TOP_GLYPH
        FCALL   d_op
        BR      gly1
gly3:   FRET

gtab:   DB      G_BACKL, 0, 0AH, 05H, 0AH, 05H, 0AH, 05H, 0AH, 05H, 0AH, 05H, 0
        DB      G_BACKR, 0, 0AH, 04H, 0AH, 04H, 0AH, 04H, 0AH, 04H, 0AH, 04H, 0
        DB      0
```

CONFIG



hosts · networks · files · boot

CONFIG is the program a FujiNet computer meets first: the WiFi, the host slots, the files on them, and the one to boot. On the *Studio II* it lives in the cartridge itself. The firmware carries it, built, as `baked/fujiconfigrom.h`, and the cartridge serves it from the moment the console is turned on. Its source is `fujinet-config/studio2/`, carried by the handbook in `listings/config/`.

It does not share the other consoles' CONFIG code: that code assumes a keyboard, four device slots and kilobytes of buffers, and the *Studio II* has twenty keys and 512 bytes. It does what the Channel F's does, in 4,152 bytes of 1802 on top of the runtime: 25 pages.

The screens



Figure 7. Adapter info, and the networks the adapter can see.

Screen	Shows
Status	the network and the address; with a network already joined, it moves on to the host slots after three seconds
Host slots	the eight hosts; 5 opens one, 7 renames it, 1 the networks, 3 adapter info
Files	a folder, eight names to a page; 5 opens a folder or boots a file, 7 copies it to another host
Networks	what the adapter found; 5 picks one, then the keyboard for its password
Info	the adapter's addresses, version and MAC

Both keypads work alike: 2 and 8 move, 4 and 6 page, 5 selects, 0 goes back.

A page in one round trip

The file browser shows a page of eight names with one READ_DIR_ENTRY. In *block mode*, flag bits `$C0` plus the group's size, the FujiNet answers with a whole group of entries, full-length names and a folder flag for each, in one reply. CONFIG notes

CDP1802 ASSEMBLY (AS) · FUJICMD.INC

```
; c_rdblk -- READ_DIR_ENTRY in block mode: one group of NROWS entries (and
; whatever follows that fits) from the position the directory is at.
        RFIT    18
c_rdblk:
        FCALL   fc_beg
        DB      FNCRDIR, 2
        LDI     DIRPAGES
        FCALL   fn_pb
        LDI     DIRMODE
        FCALL   fn_pb
        FJMP    go_ack
```

CDP1802 ASSEMBLY (AS) · BOOT.INC

```
; boot.inc -- st_boot: mount the file V_PATH names into device slot 0 and
; hand the console to it.
;
; MOUNT_IMAGE makes the ESP32 push the file to the cart before it answers;
; the cart stages it and draws the progress bar in the status strip itself.
; Once BOOTSTATE is READY, fn_boot arms the swap and restarts the BIOS, which
; the cart answers with the new image. It returns only if the swap never arms.

        RFIT    182
st_boot:
        STATE
        FCALL   pr_cls
        DB      "BOOTING", 0
        LDI     20H                     ; the name, while the reply still has it
        FCALL   d_at
        LDI     TOP_MARQUEE
        FCALL   d_op
        LDV     V_CUR
        FCALL   ent_off
        LDI     0
        FCALL   d_rpl
        FCALL   pr_at
```

where each name starts, prints the rows from the reply window, and moves the bar without asking again:

The folder stays open until the next OPEN_DIRECTORY, or a boot, closes it.

The keyboard

Names and passwords are typed on a keyboard of six rows, all of `$20` to `$7F`, sixteen to a row. A character's cell is its code less `$20`, so there is no table and no shift key: the font's lower case is a letter of its own. The netcat uses the same keyboard, carried over unchanged (netcat, page 22).

Booting

`boot.inc` boots a file as `fujiboot` does: MOUNT_HOST, SET_DEVICE_FULLPATH, MOUNT_IMAGE, then `fn_boot` once BOOT_STATE says ready. The file's name goes on the screen under a marquee, straight from the reply window, while the cartridge draws the progress bar in its status strip.

Building it into the cartridge

1. `make` in `fujinet-config/studio2/` builds `build/config.st2`; `make install` copies it into the firmware's `pico/studio2/build/`.
2. `build-cart.sh --bake` in the firmware turns it into `baked/fujiconfigrom.h`, which the cartridge's firmware and the MAME model both compile in.
3. In MAME, `FUJINET=1` serves the baked CONFIG from power-on, as the cartridge will.

NOTE:
CONFIG is the one image the cartridge lets claim the mailbox without the `"FUJI"` check: it is staged with the claim forced on, so the console always has a way back to the network.

Things That Cost Real Time

Every fact in this handbook was bought with time, and some of them were expensive. Here are the expensive ones, so you do not have to pay again.

The milestones

The cartridge was brought up in the FujiNet MAME model, one milestone at a time, each with a test that had to pass before the next began.

In the cartridge

- **There is no write line on the edge.** Every byte a program sends travels in the address of a read, which is why the mailbox and the text engine are made of hotspot pages, and why a program cannot store a dot.
- **A three-cycle instruction shortens a DMA line.** In MAME, an instruction from `$C0` to `$CF` in flight when the 1861 asks for DMA costs the line a byte. The runtime bans them, `checkrom.py` refuses them, and the display interrupt keeps one on purpose, where it lines the first fetch up. Whether a real 1861 does the same is one of the questions a real cartridge will answer.
- **The naive hand-over failed three times in four.** Jumping to `$0000` from anywhere in the frame let the 1861's DMA read `$0000` first, or an interrupt arrive between the jump and the fetch. The hand-over now waits for the bottom of a frame.
- **A plain nine-instruction interrupt routine re-entered itself at line 79.** The one shaped like the BIOS's, with its long branch, does not.
- **MAME's 1802 never runs its reset at power-on:** the interrupt enable stays off until CLEAR. Tests that need a true power-on pulse CLEAR first.
- **A Lua read of the address space is a bus read.** It fires the cartridge's hotspots. The test scripts read the device's save items instead.
- **In AS, `^` is power and `!` is exclusive or.** A test written with `^` compiled, and never failed when it should have.

In this handbook

- **AS takes one include list.** A second `-i` replaces the first, and a long list of absolute paths is cut short without a word: AS then says only that it cannot open a file. The build scripts give one list of relative paths.
- **A program that sleeps misses keys.** The first netcat paused eight frames between polls, and a press of six frames fell in the gap. It now reads the keypads every frame and the network every eighth.
- **A game server counts real seconds.** Run unthrottled, MAME used up twenty seconds of its own time in a fraction of one of the server's, and the netcat never saw its greeting. Tests that talk to a server run at real speed.
- **A greeting one character too long** wrapped, and the test looking for it on one row never found it. The line server now says `NETCAT SAYS HI`.
- **`FIT` sizes that happen to work** are not the right sizes. Half of the handbook's first programs had sizes too small that AS let pass because the routines still fit. `tools/fitcheck.py` measures them all.
- **A FIT counts a routine to the next FIT.** The runtime puts one only before a routine that branches, so its routines look short when they are not: the check skips the runtime's own.

Milestone	What it proved	Exit test
M0	The BIOS starts a cartridge program; the cartridge serves the picture and the text engine draws.	`hello`: text on screen, the frame counter moving, a key read, no DMA line short.
M1	One real transaction, and the cartridge outliving CLEAR and a power cycle.	`fujitest` shows the live SSID; after CLEAR the next sequence number is 2, after power 1.
M2	Booting over the network.	`fujiboot` mounts an image and the cartridge swaps it in.
Tier A	The cartridge serves each of thirteen known cartridges exactly as MAME's own does.	Frame for frame: 13 of 13, and the BIOS refuses the same images both ways.
Tier B	A swap leaves the console as a power-on would.	Registers and RAM at each game's first instruction: 13 of 13.
Tier C	Each pushed over the network, and played.	13 of 13.

- **Helvetica has no suits and no arrows.** The book draws them from the cartridge's own font instead.
- **Typst's bold and italic are not settings but adjustments.** Inside them the Bodoni wordmark came out bold, or upright, until the book replaced both outright.

Troubleshooting Checklist

If a program does not do what you expect, find the symptom below before you look anywhere else. Most of them were met while this handbook was written.

Building

SYMPTOM	PROBABLE CAUSE AND REMEDY
AS says "error in opening file" for an include that is there.	• AS takes one include list: a second `-i` replaces the first. Give one `-i` with the folders separated by colons. • A long list of absolute paths is cut short without a word. Make the paths relative.
AS says a branch target is out of range, or off its page.	• A short branch reaches only its own page. Put a `FIT` with the routine's size before it, or leave the page with `FJMP`.
`checkrom.py` refuses a 3-cycle opcode.	• `LBR`, the long branches and skips, and `NOP` (`$C0`–`$CF`) cost the 1861 a byte of its line. Use a short branch on the page, or `FJMP` off it.
`checkrom.py` refuses RF.	• RF is the hotspot pointer, read only with `LDN`. Use another register for a pointer of your own.
`mkst2.py` refuses a page.	• Code landed on `$x000`–`$x3FF`, `$0B`, `$08`–`$09` or `$E4` and up. `RFIT` instead of `FIT` skips the low pages; an `ORG` puts code where it may go.
`fitcheck.py` says a FIT is short.	• The routine grew. Make the size what fitcheck measured: today it fits by luck.
AS says `fn_go` (or `d_at`) is undefined.	• The runtime's includes go at `ORG 0400H`: `s2call.inc`, `fujilib.inc`, `fujidisp.inc`, and `input.inc` for the keypads.

The mailbox

SYMPTOM	PROBABLE CAUSE AND REMEDY
The program says no FujiNet cartridge answers.	• The image lacks `"FUJI"` at `$07FC`: end it with `S2CLAIM`. Without it the cartridge serves it as a game, with no mailbox. • In MAME, `FUJINET=1` or `FUJINET_BOOT=` was not set, and the device is asleep.

`fn_go` returns `$FF`, FNEWAIT.	• The transaction outlived the program's wait. `SLOW` above the cartridge's own: 66 for a mount or copy, 95 on a network channel. • MAME is talking to no `fujinet-pc`: set `FUJINET_TCP`. Unset, the model tries port 9995, which `run.sh`'s comment says it never uses.
ERR is 1, NOLINK.	• The cartridge cannot see the ESP32-S3. On the bench, `fujinet-pc` is not running, or another MAME holds its one connection.
ERR is 2, a timeout.	• The FujiNet did not answer in 5 seconds (60 for a mount, 90 for a network channel). A command that waits for a missing byte of payload times out: OPEN_APPKEY needs all six bytes, SET_SSID all 97.
ERR is 3, BADFRAME.	• A parameter's size was not 1, 2 or 4. Stream parameters with `fn_pb` and `fn_pw`, and set NPARAM to their number.
`fn_ack` says NAK.	• The FujiNet refused the command: check its card in the Command Reference. Some are not answered on this build at all; their lists end each chapter.
A reply's bytes are wrong, or zero.	• A later transaction painted over the window. Use the reply before the next launch, or copy what must last.
The second transaction after a CLEAR is number 3.	• Nothing is wrong: the cartridge kept ACKSEQ across CLEAR. Never count sequence numbers yourself.
STAT's link bit, or LINK at `$E815`, reads 0 after it was 1.	• `FN_R_LINK` is not latched, whatever its comment says: it follows the link. A RESET of the FujiNet drops it for a moment.

The screen

SYMPTOM	PROBABLE CAUSE AND REMEDY
The picture rolls, slips or shows stripes.	• The program read `$F400`–`$FFFF`: the cartridge counted the reads as the 1861's. • The display interrupt was changed. R0 must be set in it, with its one long branch where it is. • An `IDL` ran: it reads through R0 on every cycle. Look for a fall into a FIT's gap of zeros.
Text runs off the bottom and is lost.	• PUTC never scrolls. Use SCROLL, as the netcat does.
A PRINT shows nothing.	• The offset or the most was wrong, or the reply has a NUL first. PRINT stops at a NUL.
The marquee stops.	• A transaction is running: the cartridge does one thing at a time. It goes on when the reply is in.

The progress bar is still there after the image is ready.	• The strip stays through READY, and goes with the swap or a failure, whatever `s2_text.h`'s comment says.

The keypads

SYMPTOM	PROBABLE CAUSE AND REMEDY
Presses go missing.	• The program waits between reads. Read the keypads every frame and do the slow work every few.
Every key reads as pressed, or none.	• `OUT 2` read the cartridge: X must point at console RAM, `V_KTMP`, while it runs.
A key repeats when it should not.	• `in_get` repeats any held key. Check for a fresh press, as CONFIG's `key_wait` does for every key but the arrows.

Booting

SYMPTOM	PROBABLE CAUSE AND REMEDY
`fujiboot` **shows** `FAILED 03E02`**.**	• MOUNT_IMAGE timed out. The program's wait must pass the cartridge's 60 seconds.
`FAILED 05E03`**.**	• BOOT_ERR 3: the image names a page a FujiNet program may not use.
`FAILED 05E01`**.**	• BOOT_ERR 1: the image is larger than 16,640 bytes.
`FAILED 06E`**, anything.**	• The swap was never armed: BOOTLOCK was written before BOOT_STATE said READY.
A game boots, and CONFIG cannot be reached again.	• A game does not claim the mailbox. Turn the console off: power is the only way back.

The bench

SYMPTOM	PROBABLE CAUSE AND REMEDY
MAME starts but the cartridge cannot connect.	• `fujinet-pc` is down (`bench/up.sh`), on another port, or already talking to another MAME.
A drive script against a game server times out.	• MAME ran unthrottled and the server's seconds are real. `THROTTLE=1`.
A Lua test changes what it measures.	• It read the address space and fired a hotspot. Read the device's save items, as `s2screen.lua` does.
The firmware README's list of a program's pages leaves out `$0A`**.**	• The planner allows it, and this handbook follows the planner, `st2.py`.

Parts List

Every address, register, command and listing in this handbook was transcribed from these, at these revisions, and every program was built and run against them. When ordering repair parts, always include the part number.

The cartridge

Window server, mailbox, text engine, loader	`fujinet-firmware pico/studio2 @ 07e9bbaa9`
Shared mailbox protocol and fujimail	`fujinet-firmware pico/common @ 07e9bbaa9`
Runtime and the bring-up programs	`pico/studio2/testrom @ 07e9bbaa9`
Checkers: checkrom.py, mkst2.py, st2.py	`pico/studio2/tools @ 07e9bbaa9`
MAME cartridge model and its Lua	`pico/studio2/emu @ 07e9bbaa9`
ESP32-S3 board: fujiversal-studio2	`build-platforms, include/pinmap @ 07e9bbaa9`
Board, Rev0: RP2354B, ESP32-S3, ‘541, CP2102N	`fujinet-hardware RCA-Studio-II/FujiNet-StudioII-Rev0 @ 0e539b8`

The adapter

Command dispatch: FUJI device and mixins	`lib/device/rs232, fujiDevice @ 07e9bbaa9`
Network device	`lib/device/NDevice @ 07e9bbaa9`
Clock	`lib/device/fujiClock @ 07e9bbaa9`
fujinet-pc on the bench	`v1.6-07e9bbaa9, RS232 build`
Bus-over-IP	`127.0.0.1:9973`

The programs

CONFIG	`fujinet-config studio2 @ 86c763d`
5 Card Stud	`fujinet-5cardstud studio2 @ 1008f9a`
Battleship	`fujinet-battleship studio2 @ 87b8cc7`
Game servers	`servers fujinet-game-system @ 19770c3`
This handbook’s programs	`listings/asm`

The tools

Macroassembler AS	`1.42 Beta, build 306`
MAME, studio2 driver	`533800328a2 + the cartridge graft`
Python	`3.14`
Typst	`0.14.2`
Fonts	`Helvetica; Adobe Bodoni Std Book Italic; Source Code Pro`

Further reading

The 1802 itself	RCA MPM-201A, CDP1802 User Manual, 1976
The console	RCA Studio II Owner's Manual, 1977
Every protocol a devicespec can name	The FujiNet Network Protocol Handbook
The other consoles	the FujiNet programmer's handbooks for the 2600, 7800, Astrocade, ColecoVision, Intellivision and NES

FN-S2-PH. First edition, 2026.

APPENDIX A

Mailbox Quick Reference

The arena

Address	Is
`$E400-$E7FF`	the reply window, RXLEN bytes
`$E800`	the status page
`$E9`*nn* then `$EA`*vv*	register nn = vv
`$EB`*bb*	one byte of the TX stream
`$EC`*rc*	the cursor to row r, column c
`$ED`*cc*	draw character cc
`$EE`*op*	run text operation op
`$EF`*vv*	push parameter vv
`$F400-$FFFF`	the picture: never read it

The registers

Name	No.	Value
DEVICE	`$00`	`$70` FUJI, `$71` network, `$45` clock …
CMD	`$01`	the command
NPARAM	`$02`	parameters: 0 to 8
DATA_RST	`$05`	any: the TX stream starts over
SEQ	`$10`	ACKSEQ + 1, skipping 0: launch
BOOTLOCK	`$11`	`$B5`: arm the staged image
CONFIG	`$14`	`$C5`: back to CONFIG

The status page

Address	Runtime name	Is
`$E800`	`FN_ACKSEQ`	the answer's sequence number; last painted
`$E801`	`FN_STAT`	bit 0: the link
`$E802`	`FN_ERR`	0, or the cartridge's error
`$E803`	`FN_REPLYCMD`	`$06` ACK, `$15` NAK
`$E804`	`FN_RXLEN`	the reply's length, low first
`$E806`	`FN_BOOTSTATE`	0 idle, 1 coming, 2 ready, `$80` failed
`$E807`	`FN_BOOTPCT`	0–100
`$E808`	`FN_BOOTERR`	1 too big, 2 short, 3 bad page
`$E809`	`FN_MAGIC`	`'F'`, `'N'`
`$E812`	`FN_MODE`	0 CONFIG, 2 a game, 3 a FujiNet program

Address	Runtime name	Is
`$E81C`	`FN_ARMED`	1: the swap is armed
`$E823`	`FN_SWAPS`	hand-overs since power-on

A transaction

CDP1802 ASSEMBLY (AS)

```
        FNBEGIN FNDEVF, FNCADPX, 0  ; device, command, n
        SLOW    95                  ; if it can take long
        FCALL   fn_go               ; launch and wait
        BNZ     failed              ; the cart's error
        FCALL   fn_ack
        BNZ     failed              ; the FujiNet's NAK
        PEEK    FN_RXLEN            ; and the reply
```

The RAM

Address	Is
`$0800`–`$080F`	the runtime's `V_*`
`$0810` up	the program's, from `V_APP`
down from `$09FF`	the stack, R2

APPENDIX B

Error and Status Codes

What `fn_go`, `fn_ack` and the runtime return in D, and what the cartridge paints.

Name	Value	Means
FNEOK	$00	the transaction went through (FN_ERR_OK)
FN_ERR_NOLINK	$01	no link to the ESP32-S3
FN_ERR_TIMEOUT	$02	the FujiNet did not answer in time
FN_ERR_BADFRAME	$03	a parameter size other than 1, 2 or 4; a reply that failed its checks
FN_ERR_TOOBIG	$04	the request or the reply would not fit
FNENAK	$EE	`fn_ack`: the FujiNet said NAK
FNENARM	$FC	`fn_boot` / `fn_config`: the swap was never armed
FNENOC	$FD	no FujiNet cartridge: the magic is missing
FNEWAIT	$FF	`fn_go` gave up before the cartridge answered
ACK	$06	REPLYCMD: the FujiNet agreed
NAK	$15	REPLYCMD: the FujiNet refused
BOOT_STATE 0	$00	nothing staged
BOOT_STATE 1	$01	an image is coming
BOOT_STATE 2	$02	staged and ready to arm
BOOT_STATE $80	$80	failed: see BOOT_ERR
BOOT_ERR 1	$01	larger than an ST2 can be
BOOT_ERR 2	$02	shorter than declared
BOOT_ERR 3	$03	it names a page a program may not use
MODE 0	$00	serving the image from power-on: CONFIG
MODE 2	$02	serving a game, the mailbox off
MODE 3	$03	serving a FujiNet program
NET STATUS 1	1	network channel status byte: all is well
NET STATUS 136	$88	the end of the file

APPENDIX C

Program Listings

Every program in this handbook, as built and run. The runtime, `listings/runtime/`, is the cartridge bring-up's, unchanged.

The runtime

fujinet.inc `runtime/fujinet.inc`

```
; fujinet.inc -- the 1802 side of the FujiNet Studio II cartridge: names.
;
; Mirrors firmware/include/fuji_mailbox.h and s2_text.h by hand;
; tools/checkdefs.py compares them. Equates only: include before any ORG.

; ---- registers, and the conventions every client keeps ----
R0      EQU     0       ; DMA pointer: only the ISR sets it
R1      EQU     1       ; interrupt PC (isr)
R2      EQU     2       ; stack, top at STACK, X = 2 outside s2call.inc
R3      EQU     3       ; PC
R4      EQU     4       ; SCRT call  (FCALL)
R5      EQU     5       ; SCRT return (FRET)
R6      EQU     6       ; SCRT link
R7      EQU     7       ; SCRT jump  (FJMP)
R8      EQU     8       ; scratch: any routine may clobber R8, R9
R9      EQU     9
RA      EQU     10      ; RA-RC: kept across calls
RB      EQU     11
RC      EQU     12
RD      EQU     13      ; .0 beep frames (ISR), .1 SCRT's D while it runs
RE      EQU     14      ; frames since power-on (ISR)
RF      EQU     15      ; hotspot pointer: macros only, parked on FN_PARK

HI      FUNCTION x,((x)>>8)&0FFH
LO      FUNCTION x,(x)&0FFH

STACK   EQU     09FFH   ; console RAM is $0800-$09FF, all the app's
RAMVARS EQU     0800H   ; s2call.inc's and fujilib.inc's variables

; ---- the arena (fuji_mailbox.h) ----
FN_REPLY        EQU     0E400H          ; 1K reply window
FN_STATUS       EQU     0E800H          ; status page
FN_REGSEL       EQU     0E900H          ; read +n: select register n
FN_REGDATA      EQU     0EA00H          ; read +v: the selected register = v
FN_TXDATA       EQU     0EB00H          ; read +b: append one TX byte
FN_STUB         EQU     0E9F0H          ; SEX RC; RET; $23 -- run with P = C
FN_PARK         EQU     0BH             ; RF.1 between hotspot reads: inert
FN_RASTER       EQU     0F400H          ; 64x128, 8 bytes a line

FN_ACKSEQ       EQU     FN_STATUS+00H
FN_STAT         EQU     FN_STATUS+01H
FN_ERR          EQU     FN_STATUS+02H
FN_REPLYCMD     EQU     FN_STATUS+03H
FN_RXLEN        EQU     FN_STATUS+04H   ; LE
FN_BOOTSTATE    EQU     FN_STATUS+06H
FN_BOOTPCT      EQU     FN_STATUS+07H
FN_BOOTERR      EQU     FN_STATUS+08H
FN_MAGIC        EQU     FN_STATUS+09H   ; 'F','N'
FN_PROTOVER     EQU     FN_STATUS+0BH
FN_MODE         EQU     FN_STATUS+12H
FN_LINK         EQU     FN_STATUS+15H
FN_ARMED        EQU     FN_STATUS+1CH
FN_STAGED       EQU     FN_STATUS+1DH
FN_SWAPS        EQU     FN_STATUS+23H
FN_FRAMES       EQU     FN_STATUS+24H

FN_REG_DEVICE   EQU     00H
FN_REG_CMD      EQU     01H
FN_REG_NPARAM   EQU     02H
FN_REG_DATA_RST EQU     05H
FN_REG_RXSLICE  EQU     06H
FN_REG_SEQ      EQU     10H
FN_REG_BOOTLOCK EQU     11H
FN_REG_BOOTSEL_1 EQU    12H
FN_REG_BOOTSEL_2 EQU    13H
FN_REG_CONFIG   EQU     14H
FN_BOOTLOCK_MAGIC EQU   0B5H
FN_CONFIG_MAGIC EQU     0C5H

FN_BOOT_IDLE    EQU     0
FN_BOOT_XFER    EQU     1
FN_BOOT_READY   EQU     2
FN_BOOT_FAILED  EQU     80H
FN_MODE_BOOT    EQU     0
FN_MODE_GAME    EQU     2
FN_MODE_APP     EQU     3
FN_CLAIM        EQU     07FCH           ; "FUJI"

; ---- the text engine (s2_text.h) ----
TX_CURSOR       EQU     0ECH            ; page: read +rc
TX_PUTC         EQU     0EDH            ; page: read +char
TX_OP           EQU     0EEH            ; page: read +op
TX_PARAM        EQU     0EFH            ; page: read +byte
TOP_CLS         EQU     00H
TOP_INV_ON      EQU     01H
TOP_INV_OFF     EQU     02H
TOP_CLREOL      EQU     03H
TOP_SCROLL      EQU     04H
TOP_PRINT       EQU     10H
TOP_PRINT_PAD   EQU     11H
TOP_MARQUEE     EQU     20H
TOP_MQ_OFF      EQU     21H
TOP_REPEAT      EQU     30H
TOP_GLYPH       EQU     40H
TOP_TILE        EQU     41H
TOP_MOSAIC      EQU     42H
SYM_SPADE       EQU     01H
SYM_HEART       EQU     02H
SYM_DIAMOND     EQU     03H
SYM_CLUB        EQU     04H
SYM_BLOCK       EQU     05H
SYM_RIGHT       EQU     06H
SYM_LEFT        EQU     07H
SYM_UP          EQU     08H
SYM_DOWN        EQU     09H
SYM_TEN         EQU     0AH
SYM_BACK        EQU     0BH

; ---- FujiBus (lib/device/fujiDevice) ----
FNDEVF          EQU     70H             ; the FUJI device
FNCADPX         EQU     0C4H            ; GET_ADAPTERCONFIG_EXTENDED
FNCMHST         EQU     0F9H            ; MOUNT_HOST
FNCMIMG         EQU     0F8H            ; MOUNT_IMAGE
FNCSDFP         EQU     0E2H            ; SET_DEVICE_FULLPATH
FMREAD          EQU     1               ; disk access mode: read

; ---- what fujilib.inc returns in D ----
FNEOK           EQU     0               ; FN_ERR_OK
FNEWAIT         EQU     0FFH            ; the cart never answered at all
FNENAK          EQU     0EEH            ; the server said NAK
FNENOC          EQU     0FDH            ; no cartridge answered ('F','N' missing)
FNENARM         EQU     0FCH            ; the swap was never armed

; ---- the keypads (input.inc) ----
KEY_A           EQU     0A0H            ; pad A key k: KEY_A + k
KEY_B           EQU     0B0H            ; pad B key k: KEY_B + k
```

s2macro.inc `runtime/s2macro.inc`

```
; s2macro.inc -- macros and RAM variables for Studio II FujiNet clients.
; Include after fujinet.inc and before any ORG.

; ---- SCRT: calls, returns and far jumps (s2call.inc) ----
; No LBR anywhere in a client: a 3-cycle instruction while the 1861 is
; fetching costs a byte of the line (tools/checkrom.py). Leave a page with
; FJMP, or call into it.
FCALL   MACRO   adr
        SEP     R4
        DW      adr
        ENDM

FRET    MACRO
        SEP     R5
        ENDM

FJMP    MACRO   adr
        SEP     R7
        DW      adr
        ENDM

; Start the next page unless `size` bytes still fit in this one. Every
; routine with a short branch is preceded by one: a short branch reaches
; only its own page (AS checks; the size is measured, not derived).
FIT     MACRO   size
        IF      ((($) & 0FFH) + (size)) > 100H
        ORG     (($) + 0FFH) & 0FF00H
        ENDIF
        ENDM

; R = 16-bit constant.
LDR     MACRO   r, val
        LDI     HI(val)
        PHI     r
        LDI     LO(val)
        PLO     r
        ENDM

; One hotspot read at page:D, RF parked again after. Clobbers D.
HOT     MACRO   page
        PLO     RF
        LDI     page
        PHI     RF
        LDN     RF
        LDI     FN_PARK
        PHI     RF
        ENDM

; D = the cart byte at adr (status page, reply window). RF parked after.
PEEK    MACRO   adr
        LDI     LO(adr)
        PLO     RF
        LDI     HI(adr)
        PHI     RF
        LDN     RF
        PHI     RD
        LDI     FN_PARK
        PHI     RF
        GHI     RD
        ENDM

; The claim that keeps the mailbox live: last, at $07FC.
S2CLAIM MACRO
        ORG     FN_CLAIM
        DB      "FUJI"
        ENDM

; ---- RAM variables of s2call.inc / fujilib.inc / input.inc ----
V_TMO   EQU     RAMVARS+00H     ; fn_go's timeout, seconds
V_SEQ   EQU     RAMVARS+01H     ; the sequence number fn_go launched
V_CNT   EQU     RAMVARS+02H     ; fn_frames countdown, 16-bit
V_LAST  EQU     RAMVARS+04H     ; RE.0 when fn_tick last looked
V_KTMP  EQU     RAMVARS+05H     ; the key OUT 2 latches
V_KLAST EQU     RAMVARS+06H     ; input.inc: the key held at the last scan
V_KTIME EQU     RAMVARS+07H     ; input.inc: RE.0 when it auto-repeats next
V_T     EQU     RAMVARS+08H     ; scratch for one routine at a time
V_APP   EQU     RAMVARS+10H     ; the client's own variables start here
```

s2call.inc `runtime/s2call.inc`

```
; s2call.inc -- the runtime every Studio II FujiNet client starts with: the
; entry the BIOS calls, the display interrupt, SCRT, and the hand-over.
```

```
; Include at ORG $0400; it is under a page, so its short branches resolve.

; ---- entry ----
; The BIOS byte-code jumps to $0400; "04 02" there calls native $0402 with
; P = 3, IE on and the BIOS ISR live. Swap the vector with interrupts off.
        DB      04H, 02H
s2_entry:
        SEX     R3
        DIS                             ; X = 3, P = 3, IE off
        DB      33H
        LDR     R1, isr
        LDR     R2, STACK
        LDR     R4, scrt_call
        LDR     R5, scrt_ret
        LDR     R7, scrt_jump
        LDI     0
        PLO     RD
        PHI     RD
        PLO     RE
        PHI     RE
        LDI     FN_PARK
        PHI     RF
        LDR     R0, FN_RASTER
        SEX     R3
        RET                             ; X = 2, P = 3, IE on
        DB      23H
        FJMP    main

; ---- the display interrupt ----
; The 1861 raises INT two lines before its first DMA and holds it 28 cycles.
; Like the BIOS's: one LBR so the first DMA lands on an instruction boundary,
; then wait out EF1 (it drops at the first DMA) so INT is gone before RET.
; R0 points the 1024 DMA reads at the cart's raster, which serves them by
; position. RE counts frames; RD.0 is the beep, Q on while it counts down.
        FIT     32
isrx:   LDA     R2                      ; D back
        RET                             ; leaves R1 at isr
isr:    DEC     R2
        SAV                             ; T
        DEC     R2
        STR     R2                      ; D
        LBR     isr1                    ; !3CYCLE: before the first DMA
isr1:   LDI     HI(FN_RASTER)
        PHI     R0
        LDI     0
        PLO     R0
        INC     RE
        GLO     RD
        BZ      isrq
        DEC     RD
        SEQ
        BR      isrw
isrq:   REQ
isrw:   B1      isrw
        BR      isrx

; ---- SCRT ----
; FCALL: SEP R4, DW target. R6, the link, goes on the stack; D and DF survive.
        FIT     20
        SEP     R3
scrt_call:
        PHI     RD
        SEX     R2
        GHI     R6
        STXD
        GLO     R6
        STXD
        GHI     R3
        PHI     R6
        GLO     R3
        PLO     R6
        LDA     R6
        PHI     R3
        LDA     R6
        PLO     R3
        GHI     RD
        BR      scrt_call-1

; FRET: SEP R5.
        FIT     18
        SEP     R3
scrt_ret:
        PHI     RD
        GHI     R6
        PHI     R3
        GLO     R6
        PLO     R3
        SEX     R2
        IRX
        LDXA
        PLO     R6
        LDX
        PHI     R6
        GHI     RD
        BR      scrt_ret-1

; FJMP: SEP R7, DW target.
        FIT     15
        SEP     R3
scrt_jump:
        PHI     RD
        SEX     R2
        LDA     R3
        STXD
        LDN     R3
        PLO     R3
        IRX
        LDX
        PHI     R3
        GHI     RD
        BR      scrt_jump-1

; ---- the hand-over ----
; Start the BIOS over, exactly as a power-on would reach it, with the cart
; swapping the staged image in on the BIOS's first fetch. Call it once
; FN_ARMED is set. Runs in the ~1,850 cycles between the bottom of one frame
; and the next frame's INT: no DMA, no interrupt. Never returns.
        FIT     70
fn_handover:
        GLO     RE                      ; the next frame begins...
        STR     R2
hov1:   GLO     RE
        XOR
        BZ      hov1
hov2:   BN1     hov2                    ; ...and runs to EF1 at its bottom
hov3:   B1      hov3                    ; line 208: quiet until line 78
        SEX     R3
        DIS                             ; X = 3, P = 3, IE off
        DB      33H
        LDR     R8, 08FFH               ; RAM the BIOS does not clear
        SEX     R8
hov4:   LDI     0
        STXD
        GLO     R8
        XRI     9FH
        BNZ     hov4
        LDR     RB, hov5
        SEP     RB                      ; R3 is free from here
hov5:   LDR     R0, 0001H               ; R0.1 = 0 for the BIOS's GHI R0
        LDR     R1, 001CH
        LDR     R2, 08BFH
        LDI     08H
        PHI     R6
        PHI     R8
        LDI     0
        PLO     R3
        PHI     R3
        REQ
        LDR     RC, FN_STUB
        SEP     RC                      ; SEX RC; RET; $23: X=2, P=3 at $0000
```

fujilib.inc `runtime/fujilib.inc`

```
; fujilib.inc -- the 1802 side of the FujiNet cartridge mailbox: the code.
;
; The Studio II edge has no write strobe, so every byte reaches the cart in
; the address of a read: a register write is a read of FN_REGSEL+n, then of
; FN_REGDATA+v. Every routine here is FCALLed, may clobber R8 and R9, and
; returns its result in D.

; fn_rw -- register R8.0 = D. Leaves R9 alone (fn_beg needs it).
fn_rw:  PHI     RD                      ; RD.1 is free between calls
        LDI     HI(FN_REGSEL)
        PHI     RF
        GLO     R8
        PLO     RF
        LDN     RF
        LDI     HI(FN_REGDATA)
        PHI     RF
        GHI     RD
        PLO     RF
        LDN     RF
        LDI     FN_PARK
        PHI     RF
        FRET

; fn_txb -- append D to the TX stream.
fn_txb: HOT     HI(FN_TXDATA)
        FRET

; fn_pb -- append a one-byte parameter with value D.
fn_pb:  PLO     R9
        LDI     1
        HOT     HI(FN_TXDATA)
        GLO     R9
        HOT     HI(FN_TXDATA)
        FRET

; fn_pw -- append a two-byte little-endian parameter, R8.
fn_pw:  LDI     2
        HOT     HI(FN_TXDATA)
        GLO     R8
        HOT     HI(FN_TXDATA)
        GHI     R8
        HOT     HI(FN_TXDATA)
        FRET

; fn_path -- append the NUL-terminated string at R8 to the TX stream, padded
; with NULs to EXACTLY 256 bytes: OPEN_DIRECTORY and SET_DEVICE_FULLPATH
; both read exactly 256. R9.0 counts.
        FIT     32
fn_path:
        LDI     0
        PLO     R9
        LDI     HI(FN_TXDATA)
        PHI     RF
fpth1:  LDA     R8
        BZ      fpth2
        PLO     RF
        LDN     RF
        DEC     R9
        GLO     R9
        BNZ     fpth1
        BR      fpth4
fpth2:  DEC     R8                      ; hold on the NUL
fpth3:  LDI     0
        PLO     RF
        LDN     RF
        DEC     R9
        GLO     R9
        BNZ     fpth3
fpth4:  LDI     FN_PARK
        PHI     RF
        FRET

; fn_chk -- DF = 1 if a FujiNet cartridge is answering ('F','N').
        FIT     40
fn_chk: PEEK    FN_MAGIC
        XRI     'F'
        BNZ     fchk0
        PEEK    FN_MAGIC+1
        XRI     'N'
        BNZ     fchk0
        SMI     0                       ; D = 0: DF = 1
        FRET
fchk0:  ADI     0                       ; D != 0, no carry: DF = 0
        FRET

; fn_beg -- begin a transaction: R9.1 = device, R9.0 = command, D = number
; of parameters. Rewinds the TX stream last, so parameters may be appended
; at once. Sets the timeout to 12 s; a slow command raises V_TMO after this.
fn_beg: STXD
        LDI     FN_REG_DEVICE
        PLO     R8
        GHI     R9
        FCALL   fn_rw
        LDI     FN_REG_CMD
        PLO     R8
        GLO     R9
        FCALL   fn_rw
        IRX
        LDX
        PHI     R8
        LDI     FN_REG_NPARAM
```

```
        PLO     R8
        GHI     R8
        FCALL   fn_rw
        LDI     FN_REG_DATA_RST
        PLO     R8
        LDI     0
        FCALL   fn_rw
        LDR     R8, V_TMO
        LDI     12
        STR     R8
        FRET

; fn_frames -- start a countdown of D seconds (x 64 frames, near enough).
fn_frames:
        PLO     R9
        LDR     R8, V_CNT
        GLO     R9
        SHR
        SHR
        STR     R8                      ; hi = secs / 4
        INC     R8
        GLO     R9
        ANI     3
        SHL
        SHL
        SHL
        SHL
        SHL
        SHL
        STR     R8                      ; lo = (secs % 4) * 64
        INC     R8
        GLO     RE
        STR     R8                      ; V_LAST
        FRET

; fn_tick -- count the countdown down by one if a frame has passed since the
; last call: poll it at least once a frame. DF = 1 while time remains, 0 once
; it has run out.
        FIT     50
fn_tick:
        LDR     R9, V_CNT
        LDA     R9
        BNZ     ftk1
        LDN     R9
        BNZ     ftk1
        ADI     0                       ; D = 0: DF = 0, expired
        FRET
ftk1:   LDR     R8, V_LAST
        GLO     RE
        SEX     R8
        XOR
        SEX     R2
        BZ      ftk2                    ; the same frame
        GLO     RE
        STR     R8
        LDR     R9, V_CNT               ; hi, lo
        LDA     R9
        PHI     R8
        LDN     R9
        PLO     R8
        DEC     R8
        GLO     R8
        STR     R9
        DEC     R9
        GHI     R8
        STR     R9
ftk2:   SMI     0                       ; DF = 1
        FRET

; fn_go -- commit and wait. D = FNE* (0: the transaction completed). The
; next sequence number comes from the CART's ACKSEQ + 1, never a counter in
; RAM: a power cycle restarts the client, not the cart.
        FIT     88
fn_go:  PEEK    FN_ACKSEQ
        ADI     1
        BNZ     fgo1
        LDI     1                       ; 0 is reserved as "never used"
fgo1:   PHI     R9
        LDR     R8, V_SEQ
        GHI     R9
        STR     R8
        LDI     FN_REG_SEQ
        PLO     R8
        GHI     R9
        FCALL   fn_rw                   ; launching it is this one pair
        LDR     R8, V_TMO
        LDN     R8
        FCALL   fn_frames
fgo2:   PEEK    FN_ACKSEQ               ; published LAST, after the reply
        STR     R2
        LDR     R8, V_SEQ
        LDN     R8
        XOR
        BZ      fgo3
        FCALL   fn_tick
        BDF     fgo2
        LDI     FNEWAIT
        FRET
fgo3:   PEEK    FN_ERR
        FRET

; fn_ack -- 0 on success, FNENAK if the server said no. After fn_go gave 0.
        FIT     20
fn_ack: PEEK    FN_REPLYCMD
        XRI     06H
        BZ      fack1
        LDI     FNENAK
fack1:  FRET

; fn_boot -- arm the staged image and hand the console to it. Returns only if
; the cart never armed the swap, with D = FNENARM.
fn_boot:
        LDI     FN_REG_BOOTLOCK
        PLO     R8
        LDI     FN_BOOTLOCK_MAGIC
        FCALL   fn_rw
        FJMP    fn_arm

; fn_config -- back to CONFIG. Returns only if the cart never armed it.
fn_config:
        LDI     FN_REG_CONFIG
        PLO     R8
        LDI     FN_CONFIG_MAGIC
        FCALL   fn_rw
        FJMP    fn_arm                  ; never fall into a FIT's gap

; fn_arm -- wait for FN_ARMED, then hand over; D = FNENARM if it never is.
        FIT     32
fn_arm: LDI     2
        FCALL   fn_frames
farm1:  PEEK    FN_ARMED
        BNZ     farm2
        FCALL   fn_tick
        BDF     farm1
        LDI     FNENARM
        FRET
farm2:  FJMP    fn_handover
```

fujidisp.inc `runtime/fujidisp.inc`

```
; fujidisp.inc -- drawing through the cart's text engine (s2_text.h): every
; call is a few hotspot reads; the cart renders into the raster it serves.
; FCALLed; may clobber R8, R9.

; d_at -- cursor to D = row * 16 + column.
d_at:   HOT     TX_CURSOR
        FRET

; d_putc -- draw D at the cursor and advance.
d_putc: HOT     TX_PUTC
        FRET

; d_op -- operation D (TOP_*), taking the parameters pushed so far.
d_op:   HOT     TX_OP
        FRET

; d_par -- push parameter D.
d_par:  HOT     TX_PARAM
        FRET

; d_cls -- clear the screen, cursor home.
d_cls:  LDI     TOP_CLS
        HOT     TX_OP
        FRET

; d_puts -- draw the NUL-terminated string at R8. Leaves R8 past the NUL.
        FIT     16
d_puts: LDI     TX_PUTC
        PHI     RF
dps1:   LDA     R8
        BZ      dps2
        PLO     RF
        LDN     RF
        BR      dps1
dps2:   LDI     FN_PARK
        PHI     RF
        FRET

; d_hex -- draw D as two hex digits.
        FIT     32
d_hex:  PLO     R9
        SHR
        SHR
        SHR
        SHR
        FCALL   dhx1
        GLO     R9
        ANI     0FH
dhx1:   SMI     10
        BDF     dhx2
        ADI     '0'+10
        BR      dhx3
dhx2:   ADI     'A'
dhx3:   HOT     TX_PUTC
        FRET

; d_rpl -- print reply text from offset R8, at most D characters (0: up to
; the NUL), without crossing the CPU: the cart reads its own reply window.
        FIT     48
d_rpl:  PLO     R9
        LDI     TOP_PRINT
        BR      drp1
; d_rplpad -- the same, then blanks out to D characters.
d_rplpad:
        PLO     R9
        LDI     TOP_PRINT_PAD
drp1:   PHI     R9
        GLO     R8
        HOT     TX_PARAM
        GHI     R8
        HOT     TX_PARAM
        GLO     R9
        HOT     TX_PARAM
        GHI     R9
        HOT     TX_OP
        FRET
```

input.inc `runtime/input.inc`

```
; input.inc -- the two Studio II keypads.
;
; OUT 2 latches a key number (0-9, the low nibble of M(R(X))) into the
; console's CD4515; EF3 then reads that key on pad A, EF4 on pad B. OUT is a
; memory READ of M(R(X)), so R(X) points at console RAM (V_KTMP), never at
; the cart. FCALLed; may clobber R8, R9.

KEY_DELAY       EQU     24      ; frames before a held key repeats
KEY_RATE        EQU     6       ; frames between repeats

; in_scan -- D = the key held now: KEY_A + k, KEY_B + k, or 0.
        FIT     34
in_scan:
        LDR     R9, V_KTMP
        LDI     0
        PLO     R8
isc1:   GLO     R8
        STR     R9
        SEX     R9
        OUT     2                       ; R9++
        DEC     R9
        SEX     R2
        B3      isc2
        B4      isc3
        INC     R8
        GLO     R8
        XRI     10
        BNZ     isc1
        FRET                            ; D = 0
isc2:   GLO     R8
        ORI     KEY_A
        FRET
isc3:   GLO     R8
        ORI     KEY_B
        FRET

; in_get -- D = a key just pressed, or held long enough to repeat; else 0.
        FIT     48
in_get: FCALL   in_scan
```

```
        PHI     R8
        LDR     R9, V_KLAST
        GHI     R8
        SEX     R9
        XOR
        SEX     R2
        BZ      ign2                    ; the same as last scan
        GHI     R8
        STR     R9                      ; a change: a press (or
a release)
        INC     R9
        GLO     RE
        ADI     KEY_DELAY
        STR     R9                      ; V_KTIME
        GHI     R8
        FRET
ign2:   GHI     R8
        BZ      ign4                    ; nothing held
        INC     R9
        LDN     R9                      ; repeat when RE.0
reaches V_KTIME
        STR     R2
        GLO     RE
        SM                              ; RE.0 - V_KTIME,
signed
        ANI     80H
        BNZ     ign4                    ; not yet
        GLO     RE
        ADI     KEY_RATE
        STR     R9
        GHI     R8
        FRET
ign4:   LDI     0
        FRET
```

The handbook's additions

s2defs.inc asm/s2defs.inc

```
; s2defs.inc -- the devices and commands the
; bring-up's fujinet.inc does not name, for this
; handbook's programs. Values from the firmware's
; include/fujiDeviceID.h and fujiCommandID.h;
; tools/defcheck.py compares them. Kept apart so
; fujinet.inc stays verbatim. Include after
; fujinet.inc and s2macro.inc, before any ORG.

; ---- devices ----
DSKDEV  EQU     31H     ; D1: (D2:-D8: $32-$38)
PRNDEV  EQU     40H     ; P1:
CLKDEV  EQU     45H     ; the clock (APETIME)
MDMDEV  EQU     50H     ; R: the modem
NETDEV  EQU     71H     ; N1: (N2:-N8: $72-$78)

; ---- FUJI ($70) commands ----
FNCRST  EQU     0FFH    ; RESET
FNCGSSD EQU     0FEH    ; GET_SSID
FNCSCAN EQU     0FDH    ; SCAN_NETWORKS
FNCSCNR EQU     0FCH    ; GET_SCAN_RESULT
FNCSSSD EQU     0FBH    ; SET_SSID
FNCWIFI EQU     0FAH    ; GET_WIFISTATUS
FNCODIR EQU     0F7H    ; OPEN_DIRECTORY
FNCRDIR EQU     0F6H    ; READ_DIR_ENTRY
FNCCDIR EQU     0F5H    ; CLOSE_DIRECTORY
FNCRHST EQU     0F4H    ; READ_HOST_SLOTS
FNCWHST EQU     0F3H    ; WRITE_HOST_SLOTS
FNCRDEV EQU     0F2H    ; READ_DEVICE_SLOTS
FNCWDEV EQU     0F1H    ; WRITE_DEVICE_SLOTS
FNCWIEN EQU     0EAH    ; GET_WIFI_ENABLED
FNCUIMG EQU     0E9H    ; UNMOUNT_IMAGE
FNCADPC EQU     0E8H    ; GET_ADAPTERCONFIG
FNCNEWD EQU     0E7H    ; NEW_DISK
FNCUHST EQU     0E6H    ; UNMOUNT_HOST
FNCGDPS EQU     0E5H    ; GET_DIRECTORY_POSITION
FNCSDPS EQU     0E4H    ; SET_DIRECTORY_POSITION
FNCSHPX EQU     0E1H    ; SET_HOST_PREFIX
FNCGHPX EQU     0E0H    ; GET_HOST_PREFIX
FNCAKWR EQU     0DEH    ; WRITE_APPKEY
FNCAKRD EQU     0DDH    ; READ_APPKEY
FNCAKOP EQU     0DCH    ; OPEN_APPKEY
FNCAKCL EQU     0DBH    ; CLOSE_APPKEY
FNCGDFP EQU     0DAH    ; GET_DEVICE_FULLPATH
FNCCBOT EQU     0D9H    ; CONFIG_BOOT
FNCCOPY EQU     0D8H    ; COPY_FILE
FNCMALL EQU     0D7H    ; MOUNT_ALL
FNCSBM  EQU     0D6H    ; SET_BOOT_MODE
FNCRAND EQU     0D3H    ; RANDOM_NUMBER
FNCB64EI EQU    0D0H    ; BASE64_ENCODE_INPUT
FNCB64EC EQU    0CFH    ; BASE64_ENCODE_COMPUTE
FNCB64EL EQU    0CEH    ; BASE64_ENCODE_LENGTH
FNCB64EO EQU    0CDH    ; BASE64_ENCODE_OUTPUT
FNCB64DI EQU    0CCH    ; BASE64_DECODE_INPUT
FNCB64DC EQU    0CBH    ; BASE64_DECODE_COMPUTE
FNCB64DL EQU    0CAH    ; BASE64_DECODE_LENGTH
FNCB64DO EQU    0C9H    ; BASE64_DECODE_OUTPUT
FNCHSHI EQU     0C8H    ; HASH_INPUT
FNCHSHC EQU     0C7H    ; HASH_COMPUTE
FNCHSHL EQU     0C6H    ; HASH_LENGTH
FNCHSHO EQU     0C5H    ; HASH_OUTPUT
FNCHSHN EQU     0C3H    ; HASH_COMPUTE_NO_CLEAR
FNCHSHX EQU     0C2H    ; HASH_CLEAR
FNCQRO  EQU     0BFH    ; QRCODE_OUTPUT
FNCQRL  EQU     0BEH    ; QRCODE_LENGTH
FNCQRE  EQU     0BDH    ; QRCODE_ENCODE
FNCQRI  EQU     0BCH    ; QRCODE_INPUT
FNCGUID EQU     0BBH    ; GENERATE_GUID
FNCSTAT EQU     53H     ; STATUS
FNCRDY  EQU     00H     ; DEVICE_READY

; ---- app keys (AppKeyMixin.h) ----
; OPEN_APPKEY's payload is one packed struct:
; creator (2 bytes, low first), app, key, mode,
; and a reserved byte that is not optional.
AKMRD   EQU     0       ; mode: read
AKMWR   EQU     1       ; mode: write
AKCREAT EQU     1       ; the shared user-name key:
AKAPP   EQU     1       ;   creator 1, app 1, key 0,
AKKEY   EQU     0       ;   which every game reads

; ---- the clock ($45) ----
CKGTIME EQU     93H     ; GETTIME: 6 bytes, binary
CKISOL  EQU     49H     ; 'I' GET_ISO_LOCAL
CKISOU  EQU     5AH     ; 'Z' GET_ISO_UTC
CKGEN   EQU     47H     ; 'G' GET_GENERAL
CKTZLEN EQU     4CH     ; 'L'  GETTZ_LEN
CKSETTZ EQU     99H     ; SETTZ: payload the TZ
CKTZTIM EQU     9AH     ; GETTZTIME

; ---- hash algorithms (HASH_COMPUTE) ----
HAMD5   EQU     0
HASHA1  EQU     1
HASHA256 EQU    3
HASHA512 EQU    4

; ---- the N: device ($71-$78) ----
NCOPEN  EQU     4FH     ; 'O' mode, trans; spec
NCCLOSE EQU     43H     ; 'C'
NCREAD  EQU     52H     ; 'R' one 2-byte count
NCWRITE EQU     57H     ; 'W' 2-byte count; data
NCSTAT  EQU     53H     ; 'S' two 1-byte zeros
NCPARSE EQU     50H     ; 'P' parse as JSON
NCQUERY EQU     51H     ; 'Q' payload the path
NCPRSR  EQU     0FCH    ; SET_PARSER
NCSETP  EQU     0FBH    ; SET_PARAMETER
NCTRANS EQU     54H     ; 'T' translation
NCEOL   EQU     4CH     ; 'L' set the EOL
NCRATE  EQU     5AH     ; 'Z' interrupt rate
NCSEEK  EQU     25H     ; '%' one 4-byte offset
NCTELL  EQU     26H     ; '&' reply 4 bytes
NCREN   EQU     20H     ; rename "old,new"
NCDEL   EQU     21H     ; '!' delete
NCLOCK  EQU     23H     ; '#' lock
NCUNLK  EQU     24H     ; '$' unlock
NCMKDIR EQU     2AH     ; '*' mkdir
NCRMDIR EQU     2BH     ; '+' rmdir
NCCHDIR EQU     2CH     ; ',' chdir
NCGETCW EQU     30H     ; '0' getcwd
NCACCPT EQU     41H     ; 'A' accept a client
NCCCLI  EQU     63H     ; 'c' close the client
NCRMT   EQU     72H     ; 'r' UDP: who sent it
NCDEST  EQU     44H     ; 'D' UDP: host:port
NCHTTPM EQU     4DH     ; 'M' HTTP channel mode
NCUSER  EQU     0FDH    ; payload the user name
NCPASS  EQU     0FEH    ; payload the password
NPNONE  EQU     0       ; parsers
NPJSON  EQU     1
NPHTML  EQU     2
NPXML   EQU     3
NMREAD  EQU     4       ; open: read (HTTP GET)
NMWRITE EQU     8       ; write (HTTP PUT)
NMHTTPG EQU     12      ; read/write (GET+headers)
NMPOST  EQU     13      ; HTTP POST
NTRNONE EQU     0       ; translation: none
NTRCR   EQU     1       ; CR
NTRLF   EQU     2       ; LF
NTRCRLF EQU     3       ; CR LF
NSAVLO  EQU     0       ; STATUS: bytes waiting, lo
NSAVHI  EQU     1       ;   hi
NSCONN  EQU     2       ;   connected
NSDEVST EQU     3       ;   error (1 = fine)

; ---- macros ----
; FNBEGIN dev, cmd, nparam -- begin a transaction.
FNBEGIN MACRO   dev, cmd, np
        LDI     dev
        PHI     R9
        LDI     cmd
        PLO     R9
        LDI     np
        FCALL   fn_beg
        ENDM

; PUTS rc, str -- the cursor to rc, then a string.
PUTS    MACRO   rc, str
        LDI     rc
        FCALL   d_at
        LDR     R8, str
        FCALL   d_puts
        ENDM

; SLOW secs -- after fn_beg: wait up to secs.
SLOW    MACRO   secs
        LDR     R8, V_TMO
        LDI     secs
        STR     R8
        ENDM

; GOACK -- go_ack, and to `fail` unless it was 0.
GOACK   MACRO
        FCALL   go_ack
        BZ      $+5             ; over the FJMP
        FJMP    fail
        ENDM
```

booklib.inc asm/booklib.inc

```
; booklib.inc -- what this handbook's programs
; share: a transaction that must succeed, the
; failure screen, strings into the TX stream and
; short waits. Include after ORG 0C00H. FCALLed;
; may clobber R8 and R9, like the runtime.

; go_ack -- fn_go, then fn_ack: D = 0, or the
; error (FNE* or the cart's FN_ERR).
        FIT     12
go_ack: FCALL   fn_go
        BNZ     gak1
        FCALL   fn_ack
gak1:   FRET

; fn_str -- append the NUL-terminated string at
; R8 to the TX stream: a payload, not padded.
        FIT     16
fn_str: LDA     R8
        BZ      fst1
        HOT     HI(FN_TXDATA)
        BR      fn_str
fst1:   FRET

; pause -- wait D frames; RE counts them.
        FIT     16
pause:  PLO     R9
pse1:   GLO     RE
        STR     R2
pse2:   GLO     RE
        XOR
        BZ      pse2
        DEC     R9
        GLO     R9
        BNZ     pse1
        FRET

; fail -- FJMP here with D = the error: FAILED
; and the code on the last row, and stop.
        FIT     40
fail:   PLO     RA
        PUTS    90H, t_fail
        GLO     RA
        FCALL   d_hex
fl1:    BR      fl1
t_fail: DB      "FAILED E", 0
```

build.sh asm/build.sh

```
#!/usr/bin/env bash
# build.sh -- this handbook's 1802 programs, with
Macroassembler AS.
#
#   ./build.sh                    every program
#   ./build.sh netget clock       just those
#
# Each program is build/<name>.st2, what the cartridge serves.
AS runs
# case-sensitive (-U). The runtime (../runtime) is the
firmware's testrom,
# verbatim; build/gen/bookcfg.inc carries the bench's URLs and
paths.
```

```
set -euo pipefail
cd "$(dirname "$0")"
BOOK=$(cd ../.. && pwd)
. "$BOOK/bench/ports.env"

ASL=${ASL:-$HOME/asl/asl}
[ -x "$ASL" ] || ASL=$(command -v asl) || { echo "build.sh: no asl" >&2; exit 1; }
TOOLS=../runtime/tools
mkdir -p build/gen build/obj

HOST=${HOST:-127.0.0.1}
cat > build/gen/bookcfg.inc <<EOC
; GENERATED by build.sh -- do not edit.
HTTP_URL MACRO
        DB      "N:http://$HOST:$HTTP_PORT/hello.txt", 0
        ENDM
NC_URL  MACRO
        DB      "N:TCP://$HOST:$NC_PORT/", 0
        ENDM
EOC
# fujiboot's own config, as the firmware's build.sh writes it
cat > build/gen/bootcfg.inc <<EOC
; GENERATED by build.sh -- do not edit.
BOOTHST EQU     ${BOOT_HOST:-0}
BOOTPATH MACRO
        DB      "${BOOT_PATH:-/s2book/hello.st2}", 0
        ENDM
EOC

build_one() {
    local name=$1 src
    if [ -f "$name.asm" ]; then src=.; else src=../runtime; fi
    [ -f "$src/$name.asm" ] || { echo "build.sh: no $name.asm" >&2; exit 1; }
    # relative paths: AS truncates a long include list
    local r
    r=$(realpath --relative-to="$src" .)
    (cd "$src" && "$ASL" -q -U -L -i "$r:$r/../runtime:$r/build/gen" \
        -o "$r/build/obj/$name.p" -olist "$r/build/obj/$name.lst" "$name.asm")
    python3 "$TOOLS/checkrom.py" "build/obj/$name.lst"
    python3 "$TOOLS/mkst2.py" "build/obj/$name.p" "build/$name.st2"
}

targets=("$@")
[ ${#targets[@]} -gt 0 ] || targets=(hello fujitest fujiboot netget fujidir appkey clock keys glyphs pages netcat)
for t in "${targets[@]}"; do
    build_one "$t"
done
```

The programs

hello.asm runtime/hello.asm

```
; hello.asm -- M0: the display path. The cart's text engine draws into the
; raster it serves to the 1861's DMA; this client's own ISR points R0 there.
; Shows the frame counter (the ISR runs) and the last key pressed (input.inc).

        CPU     1802
        INCLUDE "fujinet.inc"
        INCLUDE "s2macro.inc"

        ORG     0400H
        INCLUDE "s2call.inc"
        INCLUDE "fujidisp.inc"
        INCLUDE "input.inc"

        FIT     96
main:   FCALL   d_cls
        LDI     01H
        FCALL   d_at
        LDR     R8, t_hello
        FCALL   d_puts
        LDI     21H
        FCALL   d_at
        LDR     R8, t_lower
        FCALL   d_puts
        LDI     41H
        FCALL   d_at
        LDR     R8, t_syms
        FCALL   d_puts
        LDI     61H
        FCALL   d_at
        LDR     R8, t_frame
        FCALL   d_puts
        LDI     81H
        FCALL   d_at
        LDR     R8, t_key
        FCALL   d_puts
loop:   LDI     68H
        FCALL   d_at
        GHI     RE
        FCALL   d_hex
        GLO     RE
        FCALL   d_hex
        FCALL   in_get
        BZ      loop
        PLO     RA
        LDI     88H
        FCALL   d_at
        GLO     RA
        FCALL   d_hex
        LDI     40                      ; a beep: 40 frames of Q
        PLO     RD
        BR      loop

t_hello: DB     "HELLO STUDIO II", 0
t_lower: DB     "fujinet 0-9", 0
t_syms:  DB     SYM_SPADE, SYM_HEART, SYM_DIAMOND, SYM_CLUB, " ", SYM_UP, SYM_DOWN, SYM_LEFT, SYM_RIGHT, " ", SYM_TEN, SYM_BACK, SYM_BLOCK, 0
t_frame: DB     "FRAME", 0
t_key:   DB     "KEY", 0

        S2CLAIM
```

fujitest.asm runtime/fujitest.asm

```
; fujitest.asm -- M1: one real transaction, and proof the cart's interlock
; survives the console's CLEAR.
;
; Asks the FujiNet for GET_ADAPTERCONFIG_EXTENDED and puts the live SSID, IP
; address and firmware version on screen, straight from the reply window (the
; cart's text engine prints it; the bytes never cross the CPU). Then SEQ and
; ERR: after CLEAR the client runs again and its sequence number must carry
; on from the cart's ACKSEQ (emu/resettest.lua).
;
; The last row is hardware check #1: for each 4K block x = 1..F, whether
; $x200-$x20F reads the BIOS's $0200-$020F ('M', a mirror: the BIOS ROMs
; decode only A10-A11) or not ('.'). The cart never drives those pages.

        CPU     1802
        INCLUDE "fujinet.inc"
        INCLUDE "s2macro.inc"

; Field offsets in GET_ADAPTERCONFIG_EXTENDED (lib/device/fujiDevice/
; fujiDevice.h, packed).
AC_SSID EQU     0                       ; char[33]
AC_VER  EQU     125                     ; char[15]
AC_SIP  EQU     140                     ; char[16], the IP as text

V_ERR   EQU     V_APP                   ; the transaction's verdict
V_BLK   EQU     V_APP+1                 ; the mirror probe's 4K block

        ORG     0400H
        INCLUDE "s2call.inc"
        INCLUDE "fujilib.inc"
        INCLUDE "fujidisp.inc"

; The runtime fills $0400-$07FB; the client's own code starts at $0C00.
        ORG     0C00H

PUTS    MACRO   rc, str
        LDI     rc
        FCALL   d_at
        LDR     R8, str
        FCALL   d_puts
        ENDM

        FIT     120
main:   FCALL   d_cls
        PUTS    00H, t_title
        FCALL   fn_chk
        BDF     have
        PUTS    20H, t_nocart
        LDI     40H
        FCALL   d_at
        PEEK    FN_MAGIC
        FCALL   d_hex
        PEEK    FN_MAGIC+1
        FCALL   d_hex
        FJMP    idle

have:   LDI     FNDEVF
        PHI     R9
        LDI     FNCADPX
        PLO     R9
        LDI     0
        FCALL   fn_beg
        FCALL   fn_go
        BNZ     fail
        FCALL   fn_ack
fail:   PLO     RA
        LDR     R8, V_ERR
        GLO     RA
        STR     R8
        BZ      show
        PUTS    20H, t_failed
        LDR     R8, V_ERR
        LDN     R8
        FCALL   d_hex
        FJMP    seq
show:   FJMP    show1

        FIT     120
show1:  PUTS    10H, t_ssid
        LDI     20H
        FCALL   d_at
        LDI     TOP_MARQUEE
        FCALL   d_op
        LDR     R8, AC_SSID
        LDI     32
        FCALL   d_rpl
        PUTS    30H, t_ip
        LDI     40H
        FCALL   d_at
        LDR     R8, AC_SIP
        LDI     15
        FCALL   d_rpl
        PUTS    50H, t_fw
        LDI     60H
        FCALL   d_at
        LDR     R8, AC_VER
        LDI     15
        FCALL   d_rpl
        FJMP    seq

        FIT     100
seq:    PUTS    70H, t_seq
        PEEK    FN_ACKSEQ
        FCALL   d_hex
        PUTS    78H, t_err
        PEEK    FN_ERR
        FCALL   d_hex
        FCALL   probe
idle:   LDI     0F0H                    ; the frame count, bottom right
        BR      idle1
idle1:  LDI     9CH
        FCALL   d_at
        GHI     RE
        FCALL   d_hex
        GLO     RE
        FCALL   d_hex
        BR      idle1

; probe -- row 8: 'M' where $x200-$x20F equals the BIOS's $0200-$020F.
        FIT     80
probe:  LDI     80H
        FCALL   d_at
        LDR     R8, V_BLK
        LDI     10H
        STR     R8
prb1:   LDR     R8, V_BLK
        LDN     R8
        ADI     02H                     ; R8 = $x200
        PHI     R8
        LDI     0
        PLO     R8
        LDR     R9, 0200H
prb2:   LDN     R9
        STR     R2
        LDA     R8
        XOR
        BNZ     prb3                    ; differs: not a mirror
        INC     R9
        GLO     R9
        XRI     10H
        BNZ     prb2
```

```
        LDI     'M'
        BR      prb4
prb3:   LDI     '.'
prb4:   FCALL   d_putc
        LDR     R8, V_BLK
        LDN     R8
        ADI     10H
        STR     R8
        BNZ     prb1
        FRET

t_title:  DB    "FUJINET STUDIO2", 0
t_nocart: DB    "NO FUJINET CART", 0
t_failed: DB    "FAILED: E", 0
t_ssid:   DB    "SSID", 0
t_ip:     DB    "IP", 0
t_fw:     DB    "FW", 0
t_seq:    DB    "SEQ ", 0
t_err:    DB    "ERR ", 0

        S2CLAIM
```

netget.asm asm/netget.asm

```
; netget.asm -- fetch a file over HTTP through
; the N: device, and show it.
;
; OPEN the URL as an HTTP GET; STATUS until the
; byte count settles; READ it into the reply
; window; show it a line at a time (LF ends a
; row, CR is skipped); CLOSE. The URL is the
; bench's, from build.sh.

        CPU     1802
        INCLUDE "fujinet.inc"
        INCLUDE "s2macro.inc"
        INCLUDE "s2defs.inc"
        INCLUDE "bookcfg.inc"

NMAX    EQU     128             ; the most we read
ROW0    EQU     2               ; the first row
NTRIES  EQU     30              ; STATUS readings
V_AV    EQU     V_APP           ; waiting: lo, hi
V_PREV  EQU     V_APP+2         ; the reading before
V_TRY   EQU     V_APP+4         ; readings so far

        ORG     0400H
        INCLUDE "s2call.inc"
        INCLUDE "fujilib.inc"
        INCLUDE "fujidisp.inc"

        ORG     0C00H
        INCLUDE "booklib.inc"

; ---- OPEN: mode and translation, then the URL --
        FIT     77
main:   FCALL   d_cls
        PUTS    00H, t_title
        FCALL   fn_chk
        BDF     open
        LDI     FNENOC
        FJMP    fail
open:   FNBEGIN NETDEV, NCOPEN, 2
        LDI     NMREAD
        FCALL   fn_pb
        LDI     NTRNONE
        FCALL   fn_pb
        LDR     R8, url
        FCALL   fn_str
        SLOW    95              ; past the cart's 90
        GOACK
        FJMP    status

; ---- STATUS until two readings agree, not 0 ----
        FIT     154
status: LDR     R8, V_PREV
        LDI     0FFH
        STR     R8
        INC     R8
        STR     R8
        INC     R8
        LDI     0
        STR     R8              ; V_TRY
stat1:  FNBEGIN NETDEV, NCSTAT, 2
        LDI     0
        FCALL   fn_pb
        LDI     0
        FCALL   fn_pb
        SLOW    95
        GOACK
        LDR     RA, V_AV        ; RA: AV, then PREV
        PEEK    FN_REPLY+NSAVLO
        STR     RA
        INC     RA
        PEEK    FN_REPLY+NSAVHI
        STR     RA
        DEC     RA
        LDR     RB, V_PREV
        SEX     RB
        LDA     RA              ; AV lo = PREV lo?
        XOR
        PHI     RC
        INC     RB
        LDN     RA              ; AV hi = PREV hi?
        XOR
        SEX     R2
        STR     R2
        GHI     RC
        OR
        BNZ     again           ; changed
        DEC     RA              ; RA = V_AV
        LDA     RA
        STR     R2
        LDN     RA
        OR                      ; lo | hi: 0 or not
        BNZ     read
again:  LDR     RA, V_AV        ; PREV = AV
        LDR     RB, V_PREV
        LDA     RA
        STR     RB
        INC     RB
        LDA     RA
        STR     RB
        INC     RB
        LDN     RB              ; V_TRY + 1
        ADI     1
        STR     RB
        XRI     NTRIES
        BNZ     wait
        LDI     FNEWAIT
        FJMP    fail
wait:   LDI     3               ; three frames
        FCALL   pause
        BR      stat1
read:   FJMP    read1

; ---- READ: one two-byte count, at most NMAX ----
        FIT     65
read1:  LDR     RA, V_AV
        LDA     RA
        PLO     R8              ; R8 = the count
        LDN     RA
        PHI     R8
        BNZ     rd1             ; 256 or more
        GLO     R8
        SMI     NMAX+1
        BNF     rd2             ; under NMAX + 1
rd1:    LDR     R8, NMAX
rd2:    GLO     R8              ; fn_beg uses R8
        PLO     RA
        GHI     R8
        PHI     RA
        FNBEGIN NETDEV, NCREAD, 1
        GLO     RA
        PLO     R8
        GHI     RA
        PHI     R8
        FCALL   fn_pw
        SLOW    95
        GOACK
        FJMP    show

; ---- show it: RA walks the reply, RB.0 counts --
        FIT     64
show:   LDR     RA, FN_REPLY
        PEEK    FN_RXLEN        ; under 256
        PLO     RB
        LDI     ROW0*10H
        PLO     RC              ; RC.0: this row
        FCALL   d_at
shw1:   GLO     RB
        BZ      close
        DEC     RB
        LDA     RA
        XRI     0DH
        BZ      shw1            ; CR: skip it
        XRI     0DH ! 0AH
        BZ      shw2            ; LF: the next row
        XRI     0AH
        FCALL   d_putc
        BR      shw1
shw2:   GLO     RC
        ADI     10H
        PLO     RC
        FCALL   d_at
        BR      shw1
close:  FJMP    close1

; ---- CLOSE, and say so ----
        FIT     43
close1: FNBEGIN NETDEV, NCCLOSE, 0
        GOACK
        PUTS    90H, t_done
hang:   BR      hang

url:     HTTP_URL
t_title: DB     "NETGET", 0
t_done:  DB     "CLOSED", 0

        S2CLAIM
```

fujidir.asm asm/fujidir.asm

```
; fujidir.asm -- list a directory on a FujiNet
; host.
;
; MOUNT_HOST, then OPEN_DIRECTORY with the path,
; then READ_DIR_ENTRY until the $7F,$7F marker
; (or the screen is full), each name printed
; straight from the reply window, then
; CLOSE_DIRECTORY.

        CPU     1802
        INCLUDE "fujinet.inc"
        INCLUDE "s2macro.inc"
        INCLUDE "s2defs.inc"

HOSTSL  EQU     0               ; slot 0: the SD
NAMELN  EQU     15              ; never 31: icons

        ORG     0400H
        INCLUDE "s2call.inc"
        INCLUDE "fujilib.inc"
        INCLUDE "fujidisp.inc"

        ORG     0C00H
        INCLUDE "booklib.inc"

; ---- mount the host, open the directory ----
        FIT     88
main:   FCALL   d_cls
        PUTS    00H, t_title
        FCALL   fn_chk
        BDF     mount
        LDI     FNENOC
        FJMP    fail
mount:  FNBEGIN FNDEVF, FNCMHST, 1
        LDI     HOSTSL
        FCALL   fn_pb
        GOACK
        FNBEGIN FNDEVF, FNCODIR, 1
        LDI     HOSTSL
        FCALL   fn_pb
        LDR     R8, path        ; exactly 256 bytes
        FCALL   fn_path
        GOACK
        FJMP    list

; ---- one name a call, a row each ----
        FIT     90
list:   LDI     10H
        PLO     RC              ; RC.0: row, column
entry:  FNBEGIN FNDEVF, FNCRDIR, 2
        LDI     NAMELN          ; the longest name
        FCALL   fn_pb
        LDI     0               ; flags
        FCALL   fn_pb
        GOACK
        PEEK    FN_REPLY
        XRI     7FH
        BNZ     show
        PEEK    FN_REPLY+1
        XRI     7FH
        BZ      close           ; $7F,$7F: the end
show:   GLO     RC
        FCALL   d_at
        LDR     R8, 0           ; reply offset 0
        LDI     NAMELN
        FCALL   d_rpl
        GLO     RC
        ADI     10H
        PLO     RC
        XRI     0A0H
        BNZ     entry           ; rows 1-9
close:  FJMP    close1

        FIT     32
close1: FNBEGIN FNDEVF, FNCCDIR, 0
        GOACK
hang:   BR      hang

path:    DB     "/s2book/", 0
t_title: DB     "SD:/S2BOOK/", 0

        S2CLAIM
```

fujiboot.asm runtime/fujiboot.asm

```
; fujiboot.asm -- M2: mount a cartridge image over the network and boot it.
;
; MOUNT_HOST -> SET_DEVICE_FULLPATH -> MOUNT_IMAGE. The last one makes the
; ESP32 side stream the file back to the cartridge on the same link,
; addressed to the DBC device, while MOUNT_IMAGE's own reply is outstanding;
; the cart stages it and reports BOOT_STATE (and draws the progress strip).
; Then BOOTLOCK, and fn_handover starts the BIOS over: the cart swaps the
; image in on the BIOS's first fetch.
;
; Target host slot and path come from bootcfg.inc, written by build.sh:
;   BOOT_HOST=0 BOOT_PATH=/game.st2 ./build.sh fujiboot

        CPU     1802
        INCLUDE "fujinet.inc"
        INCLUDE "s2macro.inc"
        INCLUDE "bootcfg.inc"

DEVSLOT EQU     0

        ORG     0400H
        INCLUDE "s2call.inc"
        INCLUDE "fujilib.inc"
        INCLUDE "fujidisp.inc"

        ORG     0C00H

PUTS    MACRO   rc, str
        LDI     rc
        FCALL   d_at
        LDR     R8, str
        FCALL   d_puts
        ENDM

; STEP n: RA.0 = n, then fn_go and fn_ack; on failure to `show` with RA.1.
STEP    MACRO   n
        LDI     n
        PLO     RA
        FCALL   go_ack
        BZ      $+5                     ; over the FJMP
        FJMP    show
        ENDM

        FIT     250
main:   FCALL   d_cls
        PUTS    00H, t_title
        PUTS    20H, t_path
        LDI     30H
        FCALL   d_at
        LDR     R8, bootpath
        FCALL   d_puts
        PUTS    50H, t_mount
        LDI     0
        PLO     RA
        LDI     FNENOC
        PHI     RA
        FCALL   fn_chk
        BDF     $+5
        FJMP    show

        ; ---- MOUNT_HOST(host) ----
        LDI     FNDEVF
        PHI     R9
        LDI     FNCMHST
        PLO     R9
        LDI     1
        FCALL   fn_beg
        LDI     BOOTHST
        FCALL   fn_pb
        STEP    1

        ; ---- SET_DEVICE_FULLPATH(dev, host, mode, path) ----
        ; three one-byte parameters, then a FULL 256-byte NUL-padded path
        LDI     FNDEVF
        PHI     R9
        LDI     FNCSDFP
        PLO     R9
        LDI     3
        FCALL   fn_beg
        LDI     DEVSLOT
        FCALL   fn_pb
        LDI     BOOTHST
        FCALL   fn_pb
        LDI     FMREAD
        FCALL   fn_pb
        LDR     R8, bootpath
        FCALL   fn_path
        STEP    2

        ; ---- MOUNT_IMAGE(dev, mode): the ESP32 streams the file now ----
        LDI     FNDEVF
        PHI     R9
        LDI     FNCMIMG
        PLO     R9
        LDI     2
        FCALL   fn_beg
        LDI     DEVSLOT
        FCALL   fn_pb
        LDI     FMREAD
        FCALL   fn_pb
        LDR     R8, V_TMO
        LDI     66                      ; past the cart's 60 s for MOUNT_IMAGE
        STR     R8
        STEP    3
        FJMP    stage

; go_ack -- fn_go, then fn_ack: D = 0, or RA.1 = D = the error.
        FIT     24
go_ack: FCALL   fn_go
        BNZ     gak1
        FCALL   fn_ack
gak1:   PHI     RA
        FRET

; ---- wait for the staged image, then hand over ----
        FIT     80
stage:  LDI     4
        PLO     RA
        LDI     10
        FCALL   fn_frames
stg1:   PEEK    FN_BOOTSTATE
        XRI     FN_BOOT_READY
        BZ      stg3
        XRI     FN_BOOT_READY ! FN_BOOT_FAILED  ; AS: ! is XOR, ^ is power
        BZ      stg2
        FCALL   fn_tick
        BDF     stg1
        LDI     FNEWAIT
        PHI     RA
        BR      show
stg2:   PEEK    FN_BOOTERR
        PHI     RA
        LDI     5
        PLO     RA
        BR      show
stg3:   FCALL   fn_boot                 ; returns only if never armed
        PHI     RA
        LDI     6
        PLO     RA

; ---- the failure screen: WHICH step, with WHAT ----
show:   PUTS    70H, t_fail
        GLO     RA
        FCALL   d_hex
        LDI     'E'
        FCALL   d_putc
        GHI     RA
        FCALL   d_hex
hang:   BR      hang

t_title: DB     "FUJINET BOOT", 0
t_path:  DB     "IMAGE:", 0
t_mount: DB     "MOUNTING...", 0
t_fail:  DB     "FAILED ", 0
bootpath:
        BOOTPATH

        S2CLAIM

```

appkey.asm asm/appkey.asm

```
; appkey.asm -- read the shared FujiNet user
; name, then write a key of our own and read it
; back.
;
; An app key is a small file on the adapter's
; SD card, named by a creator id, an app id and
; a key number. Creator 1, app 1, key 0 is the
; one every FujiNet game reads the player's
; name from. Ours is creator $1802, app $FF (this
; handbook's scratch key), key 0.
;
; OPEN_APPKEY's payload is one six-byte struct:
; creator (low first), app, key, mode and a
; reserved 0. READ_APPKEY's reply is a two-byte
; length, then the value. WRITE_APPKEY takes one
; parameter, the length, then the bytes.

        CPU     1802
        INCLUDE "fujinet.inc"
        INCLUDE "s2macro.inc"
        INCLUDE "s2defs.inc"

OURCRE  EQU     1802H
OURAPP  EQU     0FFH
OURKEY  EQU     0

        ORG     0400H
        INCLUDE "s2call.inc"
        INCLUDE "fujilib.inc"
        INCLUDE "fujidisp.inc"

        ORG     0C00H
        INCLUDE "booklib.inc"

        FIT     104
main:   FCALL   d_cls
        PUTS    00H, t_title
        FCALL   fn_chk
        BDF     go
        LDI     FNENOC
        FJMP    fail
go:     PUTS    20H, t_name     ; the user name
        LDR     RA, k_name
        FCALL   akopen
        FCALL   akread
        LDI     30H
        FCALL   akshow
        PUTS    50H, t_ours     ; ours: write it...
        LDR     RA, k_ourw
        FCALL   akopen
        FCALL   akwrite
        LDR     RA, k_ourr      ; ...read it back
        FCALL   akopen
        FCALL   akread
        LDI     60H
        FCALL   akshow
hang:   BR      hang

; akopen -- OPEN_APPKEY: no parameters, the
; six-byte struct at RA as the payload.
        FIT     40
akopen: FNBEGIN FNDEVF, FNCAKOP, 0
        LDI     6
        PLO     RB
ako1:   LDA     RA
        FCALL   fn_txb
        DEC     RB
        GLO     RB
        BNZ     ako1
        GOACK
        FRET

; akread -- READ_APPKEY: the value is left in
; the reply window after its two-byte length.
        FIT     20
akread: FNBEGIN FNDEVF, FNCAKRD, 0
        GOACK
        FRET

; akwrite -- WRITE_APPKEY: the length, then it.
        FIT     42
akwrite:
        FNBEGIN FNDEVF, FNCAKWR, 1
        LDI     VALLEN
        FCALL   fn_pb
        LDR     RA, val
        LDI     VALLEN
        PLO     RB
akw1:   LDA     RA
        FCALL   fn_txb
        DEC     RB
        GLO     RB
        BNZ     akw1
        GOACK
        FRET

; akshow -- the value just read, at D = row and
; column; "(EMPTY)" when there is none.
        FIT     49
akshow: FCALL   d_at
        PEEK    FN_REPLY        ; the length, < 64
        BZ      aks1
        LDR     R8, 2           ; after the length
        PEEK    FN_REPLY
        FCALL   d_rpl
        FRET
aks1:   LDR     R8, t_empty
        FCALL   d_puts
        FRET

```

```
k_name: DB      LO(AKCREAT), HI(AKCREAT), AKAPP
        DB      AKKEY, AKMRD, 0
k_ourw: DB      LO(OURCRE), HI(OURCRE), OURAPP
        DB      OURKEY, AKMWR, 0
k_ourr: DB      LO(OURCRE), HI(OURCRE), OURAPP
        DB      OURKEY, AKMRD, 0
val:    DB      "HELLO STUDIO II"
VALLEN  EQU     $-val
t_title: DB     "APP KEYS", 0
t_name:  DB     "PLAYER NAME", 0
t_ours:  DB     "OURS, READ BACK", 0
t_empty: DB     "(EMPTY)", 0

        S2CLAIM
```

clock.asm asm/clock.asm

```
; clock.asm -- the time of day, from the
; adapter's clock.
;
; The clock is a device of its own ($45).
; GET_ISO_LOCAL answers with the local time as
; text -- "2026-10-20T10:45:31" and more -- in
; the adapter's time zone, GET_ISO_UTC the same
; in UTC. The date and the time go on the screen
; straight out of the reply window. First the
; adapter's own zone is set with $74 (SETTZ's
; twin: $99 sets the alternate zone instead).

        CPU     1802
        INCLUDE "fujinet.inc"
        INCLUDE "s2macro.inc"
        INCLUDE "s2defs.inc"

CKSETZA EQU     74H             ; the adapter's zone

        ORG     0400H
        INCLUDE "s2call.inc"
        INCLUDE "fujilib.inc"
        INCLUDE "fujidisp.inc"

        ORG     0C00H
        INCLUDE "booklib.inc"

; ASK cmd, rc -- a clock command with no
; parameters; its date and time at rc.
ASK     MACRO   cmd, rc
        FNBEGIN CLKDEV, cmd, 0
        GOACK
        LDI     rc
        FCALL   d_at
        LDR     R8, 0           ; YYYY-MM-DD
        LDI     10
        FCALL   d_rpl
        LDI     rc+10H
        FCALL   d_at
        LDR     R8, 11          ; HH:MM:SS
        LDI     8
        FCALL   d_rpl
        ENDM

        FIT     187
main:   FCALL   d_cls
        PUTS    00H, t_title
        FCALL   fn_chk
        BDF     go
        LDI     FNENOC
        FJMP    fail
go:     FNBEGIN CLKDEV, CKSETZA, 0
        LDR     R8, tz          ; the payload
        FCALL   fn_str
        GOACK
        PUTS    20H, t_local
        ASK     CKISOL, 30H
        PUTS    60H, t_utc
        ASK     CKISOU, 70H
hang:   BR      hang

tz:      DB     "EST5EDT", 0
t_title: DB     "CLOCK", 0
t_local: DB     "LOCAL", 0
t_utc:   DB     "UTC", 0

        S2CLAIM
```

keys.asm asm/keys.asm

```
; keys.asm -- both keypads, drawn on the screen;
; the last key pressed on each is lit, the key
; code is shown, and every press beeps.
;
; in_get (input.inc) returns a key just pressed
; or repeating: KEY_A + k for pad A, KEY_B + k
; for pad B. The ISR sounds the beeper while
; RD.0 counts down.

        CPU     1802
        INCLUDE "fujinet.inc"
        INCLUDE "s2macro.inc"
        INCLUDE "s2defs.inc"

PADA    EQU     01H             ; pad A: column 1
PADB    EQU     0AH             ; pad B: column 10
V_LA    EQU     V_APP           ; pad A's lit key
V_LB    EQU     V_APP+1         ; pad B's, or $FF

        ORG     0400H
        INCLUDE "s2call.inc"
        INCLUDE "fujilib.inc"
        INCLUDE "fujidisp.inc"
        INCLUDE "input.inc"

        ORG     0C00H
        INCLUDE "booklib.inc"

        FIT     141
main:   FCALL   d_cls
        PUTS    00H, t_title
        PUTS    71H, t_pada
        PUTS    7AH, t_padb
        PUTS    90H, t_last
        LDI     PADA
        PLO     RB
        FCALL   pad
        LDI     PADB
        PLO     RB
        FCALL   pad
        LDR     R8, V_LA        ; nothing lit yet
        LDI     0FFH
        STR     R8
        INC     R8
        STR     R8
loop:   FCALL   in_get
        BZ      loop
        PLO     RA
        LDI     96H             ; the code: A5, B9
        FCALL   d_at
        GLO     RA
        FCALL   d_hex
        GLO     RA
        ANI     0FH
        BNZ     lp1
        LDI     40              ; 0: a long beep
        BR      lp2
lp1:    LDI     8
lp2:    PLO     RD              ; the ISR beeps
        GLO     RA
        ANI     0F0H
        XRI     KEY_A
        BNZ     lp3
        LDI     PADA
        PLO     RB
        LDR     RC, V_LA
        BR      lp4
lp3:    LDI     PADB
        PLO     RB
        LDR     RC, V_LB
lp4:    FCALL   light
        BR      loop

; light -- unlight the key at M(RC), light key
; RA.0 & $0F of the pad at column RB.0.
        FIT     40
light:  LDN     RC
        XRI     0FFH
        BZ      lgt1            ; none lit
        LDN     RC
        PLO     RA              ; RA.1 = 0: plain
        LDI     0
        PHI     RA
        FCALL   cell
lgt1:   GLO     RA
        ANI     0FH
        PLO     RA
        STR     RC
        LDI     1
        PHI     RA              ; RA.1 = 1: lit
        FCALL   cell
        FRET

; pad -- the ten keys of the pad at RB.0.
        FIT     24
pad:    LDI     0
        PHI     RA
        PLO     RA
pad1:   FCALL   cell
        INC     RA
        GLO     RA
        XRI     10
        BNZ     pad1
        FRET

; cell -- key RA.0 of the pad at column RB.0,
; in inverse when RA.1 is not 0.
        FIT     40
cell:   LDI     LO(t_pos)
        STR     R2
        GLO     RA
        ADD
        PLO     R8
        LDI     HI(t_pos)
        PHI     R8
        LDN     R8              ; row, column
        STR     R2
        GLO     RB
        ADD                     ; + the pad's column
        FCALL   d_at
        GHI     RA
        BZ      cel1
        LDI     TOP_INV_ON
        FCALL   d_op
cel1:   GLO     RA
        ADI     '0'
        FCALL   d_putc
        LDI     TOP_INV_OFF
        FCALL   d_op
        FRET

; where each key sits: 1-9 in a square, 0 below
        FIT     10
t_pos:  DB      52H, 20H, 22H, 24H, 30H
        DB      32H, 34H, 40H, 42H, 44H
t_title: DB     "KEYPADS", 0
t_pada: DB      "PAD A", 0
t_padb: DB      "PAD B", 0
t_last: DB      "LAST", 0

        S2CLAIM
```

glyphs.asm asm/glyphs.asm

```
; glyphs.asm -- the text engine draws more than
; text: 32 glyphs of your own, 16 tiles and the
; mosaic cells built from them, repeats and
; inverse video.
;
; A GLYPH is 12 row bytes, bit 3 the leftmost of
; its four dots. Four of them make an 8 x 24 dot
; FujiNet mark. A TILE is 2 dots by 4 lines, four
; row bytes, bit 1 the left dot; a MOSAIC cell is
; 2 x 3 tiles, a byte for each third (left tile
; in the high nibble).

        CPU     1802
        INCLUDE "fujinet.inc"
        INCLUDE "s2macro.inc"
        INCLUDE "s2defs.inc"

        ORG     0400H
        INCLUDE "s2call.inc"
        INCLUDE "fujilib.inc"
        INCLUDE "fujidisp.inc"

        ORG     0C00H
        INCLUDE "booklib.inc"

        FIT     120
main:   FCALL   d_cls
        PUTS    00H, t_title
        ; four glyphs, then the mark from them
        LDR     RA, g_mark
        LDI     4
        PLO     RB
gl1:    LDI     13              ; code + 12 rows
        FCALL   params
        LDI     TOP_GLYPH
        FCALL   d_op
        DEC     RB
        GLO     RB
        BNZ     gl1
        PUTS    20H, t_glyph
        LDI     28H
        FCALL   d_at
        LDI     80H
        FCALL   d_putc
        LDI     81H
        FCALL   d_putc
        LDI     38H
        FCALL   d_at
        LDI     82H
        FCALL   d_putc
```

```
        LDI     83H
        FCALL   d_putc
        FJMP    tiles

; params -- push D bytes from RA as parameters.
        FIT     16
params: PLO     R9
prm1:   LDA     RA
        FCALL   d_par
        DEC     R9
        GLO     R9
        BNZ     prm1
        FRET

        FIT     120
tiles:  LDR     RA, t_tiles     ; tiles 0, 1, 2
        LDI     3
        PLO     RB
tl1:    LDI     5               ; tile + 4 rows
        FCALL   params
        LDI     TOP_TILE
        FCALL   d_op
        DEC     RB
        GLO     RB
        BNZ     tl1
        PUTS    50H, t_mosaic
        LDI     56H
        FCALL   d_at
        LDR     RA, m_cells     ; five cells
        LDI     15
        FCALL   params
        LDI     TOP_MOSAIC
        FCALL   d_op
        PUTS    70H, t_rpt
        LDI     77H
        FCALL   d_at
        LDI     SYM_BLOCK       ; REPEAT char, count
        FCALL   d_par
        LDI     6
        FCALL   d_par
        LDI     TOP_REPEAT
        FCALL   d_op
        LDI     TOP_INV_ON
        FCALL   d_op
        PUTS    80H, t_inv
        LDI     TOP_INV_OFF
        FCALL   d_op
hang:   BR      hang

; the mark: dots of 2 x 6, four glyphs of 4 x 12
g_mark: DB      80H                     ; top left
        DB      1,1,1,1,1,1,0,0,0,0CH,0CH,0CH
        DB      81H                     ; top right
        DB      8,8,8,8,8,8,0,0,0,3,3,3
        DB      82H                     ; bottom left
        DB      0CH,0CH,0CH,0,0,0,0,0,0,0,0,0
        DB      83H                     ; bottom right
        DB      3,3,3,0,0,0,3,3,3,3,3,3
; tiles: blank, solid, a checker
t_tiles: DB     0, 0, 0, 0, 0
        DB      1, 3, 3, 3, 3
        DB      2, 2, 1, 2, 1
; a pyramid on a checkered sky, five cells
m_cells: DB     22H, 22H, 11H
        DB      22H, 11H, 11H
        DB      11H, 11H, 11H
        DB      22H, 11H, 11H
        DB      22H, 22H, 11H
t_title: DB     "GLYPHS, TILES", 0
t_glyph: DB     "GLYPH", 0
t_mosaic: DB    "MOSAIC", 0
t_rpt:   DB     "REPEAT", 0
t_inv:   DB     "INVERSE", 0

        S2CLAIM
```

pages.asm asm/pages.asm

```
; pages.asm -- one program across fourteen
; pages of the cartridge.
;
; A short branch reaches only its own page, so a
; program that outgrows a page leaves it with
; FJMP and reaches other pages with FCALL. Here
; each page calls `mark`, which shows the page
; it was called from, and jumps to the next.
; The cartridge's app pages are $0C-$0F and
; $x4-$xF for x = 1 to D.

        CPU     1802
        INCLUDE "fujinet.inc"
        INCLUDE "s2macro.inc"
        INCLUDE "s2defs.inc"

        ORG     0400H
        INCLUDE "s2call.inc"
        INCLUDE "fujilib.inc"
        INCLUDE "fujidisp.inc"

        ORG     0C00H
        INCLUDE "booklib.inc"

        FIT     40
main:   FCALL   d_cls
        PUTS    00H, t_title
        LDI     20H
        PLO     RC              ; where mark writes
        FCALL   mark            ; page $0C
        FJMP    1400H

; mark -- the caller's page, in hex, at RC.0;
; five to a row. R6 is the caller's return.
        FIT     43
mark:   GHI     R6
        PLO     RA
        GLO     RC
        FCALL   d_at
        GLO     RA
        FCALL   d_hex
        GLO     RC
        ADI     3
        PLO     RC
        ANI     0FH
        SMI     15
        BNF     mk1             ; still room
        GLO     RC
        ANI     0F0H
        ADI     10H
        PLO     RC
mk1:    FRET

done:   PUTS    90H, t_done
hang:   BR      hang

t_title: DB     "PAGES", 0
t_done:  DB     "ALL OK", 0

; PAGE here, next -- a page of its own
PAGE    MACRO   here, next
        ORG     here
        FCALL   mark
        FJMP    next
        ENDM

        PAGE    1400H, 2400H
        PAGE    2400H, 3400H
        PAGE    3400H, 4400H
        PAGE    4400H, 5400H
        PAGE    5400H, 6400H
        PAGE    6400H, 7400H
        PAGE    7400H, 8400H
        PAGE    8400H, 9400H
        PAGE    9400H, 0A400H
        PAGE    0A400H, 0B400H
        PAGE    0B400H, 0C400H
        PAGE    0C400H, 0D400H
        PAGE    0D400H, done

        S2CLAIM
```

netcat.asm asm/netcat.asm

```
; netcat.asm -- talk to a TCP server, a line at
; a time.
;
; The URL comes up in CONFIG's on-screen
; keyboard, ready to change; 9 connects. Then
; rows 0-8 show what the server sends, scrolling
; up as it comes, and row 9 the keys: 5 types a
; line (9 sends it with CR LF), 0 closes the
; connection and goes back to CONFIG. What
; arrives while you type waits in the FujiNet.
;
; The text engine has no memory of its own a
; program can read back, so a shadow of rows 0-8
; lives in console RAM to repaint the view after
; the keyboard has used the screen.

        CPU     1802
        INCLUDE "fujinet.inc"
        INCLUDE "s2macro.inc"
        INCLUDE "s2defs.inc"
        INCLUDE "bookcfg.inc"

; ---- RAM ledger: $0800-$09FF ----
; $0800-$080F  the runtime's V_*
; $0810-$081A  the state below
; $0870-$08B0  V_EDIT, the line being typed
; $0900-$098F  SHADOW, rows 0-8 as shown
; $0990-$09FF  the stack (112 bytes)
V_ROW   EQU     V_APP+00H       ; receive row, 0-8
V_COL   EQU     V_APP+01H       ; its column, 0-16
V_EDLEN EQU     V_APP+02H       ; keyboard: length
V_EDMAX EQU     V_APP+03H       ;   the most
V_OSK   EQU     V_APP+04H       ;   its cell, 0-95
V_KOLD  EQU     V_APP+05H       ; key_wait's
V_KTO   EQU     V_APP+06H       ;   own two
V_EDWHAT EQU    V_APP+07H       ; 0: URL, 1: line
V_CONN  EQU     V_APP+08H       ; 1: connected
V_N     EQU     V_APP+09H       ; reply bytes left
V_TS    EQU     V_APP+0AH       ; RE.0 at STATUS
V_EDIT  EQU     0870H           ; 65
SHADOW  EQU     0900H           ; 9 rows of 16
NROWS   EQU     9
NMAX    EQU     128             ; the most a READ

        ORG     0400H
        INCLUDE "s2call.inc"
        INCLUDE "fujilib.inc"
        INCLUDE "fujidisp.inc"
        INCLUDE "input.inc"

        ORG     0C00H
        INCLUDE "booklib.inc"
        INCLUDE "osk.inc"

; ---- start: the URL into the keyboard ----
        RFIT    60
main:   LDI     HI(V_APP)
        PHI     RB              ; for LDV and STV
        FCALL   fn_chk
        BDF     mn1
        LDI     FNENOC
        FJMP    fail
mn1:    LDR     R8, url
        LDR     R9, V_EDIT
mn2:    LDA     R8              ; copy it, NUL too
        STR     R9
        INC     R9
        BNZ     mn2
        SETV    V_EDWHAT, 0
        SETV    V_EDMAX, 63
        SETV    V_CONN, 0
        FJMP    st_edit

; ---- the keyboard's screen ----
        RFIT    94
st_edit:
        FCALL   d_cls
        PUTS    00H, t_okeys
        LDV     V_EDWHAT
        BNZ     ste1
        PUTS    20H, t_url
        BR      ste2
ste1:   PUTS    20H, t_line
ste2:   LDR     R8, V_EDIT      ; its length
ste3:   LDA     R8
        BNZ     ste3
        DEC     R8
        GLO     R8
        SMI     LO(V_EDIT)
        STV     V_EDLEN
        SETV    V_OSK, 'a'-20H
        FCALL   ed_grid
        LDV     V_OSK
        FCALL   ed_sel
        FCALL   ed_text
        FJMP    ed_in

; ed_ok: 9. ed_esc: 1.
        RFIT    40
ed_ok:  LDV     V_EDWHAT
        BNZ     eok1
        FJMP    connect
eok1:   FJMP    send
ed_esc: LDV     V_EDWHAT
        BNZ     ees1
        FJMP    quit            ; no URL: CONFIG
ees1:   FJMP    view

; ---- OPEN the URL, read and write ----
        RFIT    111
connect:
        FCALL   d_cls
        PUTS    00H, t_conn
        FNBEGIN NETDEV, NCOPEN, 2
        LDI     NMHTTPG         ; 12: read and write
        FCALL   fn_pb
        LDI     NTRNONE
        FCALL   fn_pb
        LDR     R8, V_EDIT
        FCALL   fn_str
        SLOW    95
        GOACK
        SETV    V_CONN, 1
```

```
        LDR     R8, SHADOW      ; a blank shadow
        LDI     NROWS*16
        PLO     R9
cn1:    LDI     ' '
        STR     R8
        INC     R8
        DEC     R9
        GLO     R9
        BNZ     cn1
        SETV    V_ROW, 0
        SETV    V_COL, 0
        LDR     R8, V_EDIT      ; an empty line
        LDI     0
        STR     R8
        FJMP    view

; ---- the receive view: repaint, then poll ----
        RFIT    60
view:   FCALL   d_cls
        LDR     RA, SHADOW
        LDI     NROWS*16
        PLO     RC
vw1:    LDA     RA              ; rows 0-8; PUTC
        FCALL   d_putc          ; wraps the rows
        DEC     RC
        GLO     RC
        BNZ     vw1
        FCALL   legend
        FJMP    poll

        RFIT    60
poll:   FCALL   in_get
        BZ      pl2
        ANI     0FH
        XRI     5
        BNZ     pl1
        SETV    V_EDWHAT, 1     ; 5: type a line
        SETV    V_EDMAX, 60
        FJMP    st_edit
pl1:    XRI     5               ; 0: close, CONFIG
        BNZ     pl2
        FJMP    quit
pl2:    LDV     V_CONN
        BZ      poll            ; closed: keys only
        LDV     V_TS            ; keys every frame,
        STR     R2              ; STATUS every 8th
        GLO     RE
        SM
        SMI     8
        BNF     poll
        GLO     RE
        STV     V_TS
        FJMP    status

; ---- STATUS: anything waiting? ----
        RFIT    106
status: FNBEGIN NETDEV, NCSTAT, 2
        LDI     0
        FCALL   fn_pb
        LDI     0
        FCALL   fn_pb
        SLOW    95
        GOACK
        PEEK    FN_REPLY+NSAVHI
        BNZ     st2             ; 256 or more
        PEEK    FN_REPLY+NSAVLO
        BNZ     st1
        PEEK    FN_REPLY+NSCONN
        BNZ     st3             ; nothing yet
        SETV    V_CONN, 0       ; the server closed
        FCALL   legend
st3:    FJMP    poll
st1:    SMI     NMAX+1
        BNF     st4             ; under NMAX + 1
st2:    LDI     NMAX
        BR      st5
st4:    ADI     NMAX+1          ; the count again
st5:    PLO     RA
        FJMP    read

; ---- READ RA.0 bytes, and show them ----
        RFIT    78
read:   FNBEGIN NETDEV, NCREAD, 1
        GLO     RA
        PLO     R8
        LDI     0
        PHI     R8
        FCALL   fn_pw
        SLOW    95
        GOACK
        PEEK    FN_RXLEN        ; what came
        STV     V_N
        LDR     RA, FN_REPLY    ; RA walks it
rd1:    LDV     V_N
        BZ      rd2
        SMI     1
        STR     RB
        LDA     RA
        FCALL   rx
        BR      rd1
rd2:    FJMP    poll

; rx -- one byte from the server: CR is
; dropped, LF ends the row, the rest is shown.
; The byte waits in RC.1: newline counts in RC.0.
        RFIT    48
rx:     PHI     RC
        XRI     0DH
        BZ      rx2             ; CR
        XRI     0DH ! 0AH
        BNZ     rx1
        FJMP    newline         ; LF
rx1:    LDV     V_COL
        XRI     16
        BNZ     rx3
        FCALL   newline         ; a full row
rx3:    FCALL   putch
rx2:    FRET

; putch -- RC.1 at V_ROW, V_COL, on the screen
; and in the shadow.
        RFIT    48
putch:  LDV     V_ROW
        SHL
        SHL
        SHL
        SHL
        STR     R2
        LDV     V_COL
        OR                      ; row * 16 + col
        PLO     RC
        FCALL   d_at
        GHI     RC
        FCALL   d_putc
        GLO     RC              ; the shadow index
        PLO     R9
        LDI     HI(SHADOW)
        PHI     R9
        GHI     RC
        STR     R9
        LDV     V_COL
        ADI     1
        STR     RB
        FRET

; newline -- the next row; at the bottom, the
; screen and the shadow scroll up a row.
        RFIT    80
newline:
        SETV    V_COL, 0
        LDV     V_ROW
        XRI     NROWS-1
        BZ      nl1
        LDV     V_ROW
        ADI     1
        STR     RB
        FRET
nl1:    LDI     TOP_SCROLL      ; rows 1-9 up
        FCALL   d_op
        LDI     80H             ; row 8 blank
        FCALL   d_at
        LDI     TOP_CLREOL
        FCALL   d_op
        FCALL   legend          ; row 9 again
        LDR     R8, SHADOW+16   ; the shadow, too
        LDR     R9, SHADOW
        LDI     (NROWS-1)*16
        PLO     RC
nl2:    LDA     R8
        STR     R9
        INC     R9
        DEC     RC
        GLO     RC
        BNZ     nl2
        LDI     16              ; R9: its last row
        PLO     RC
nl3:    LDI     ' '
        STR     R9
        INC     R9
        DEC     RC
        GLO     RC
        BNZ     nl3
        FRET

; legend -- row 9 in inverse: the keys, or
; that the server has gone.
        RFIT    48
legend: LDI     TOP_INV_ON
        FCALL   d_op
        LDV     V_CONN
        BZ      lg1
        PUTS    90H, t_keys
        BR      lg2
lg1:    PUTS    90H, t_gone
lg2:    LDI     TOP_INV_OFF
        FCALL   d_op
        FRET

; ---- WRITE the line, CR LF after it ----
        RFIT    80
send:   FNBEGIN NETDEV, NCWRITE, 1
        LDV     V_EDLEN
        ADI     2
        PLO     R8
        LDI     0
        PHI     R8
        FCALL   fn_pw           ; the count
        LDR     R8, V_EDIT
        FCALL   fn_str
        LDI     0DH
        FCALL   fn_txb
        LDI     0AH
        FCALL   fn_txb
        SLOW    95
        GOACK
        LDR     R8, V_EDIT      ; empty again
        LDI     0
        STR     R8
        FJMP    view

; ---- CLOSE, and back to CONFIG ----
        RFIT    40
quit:   LDV     V_CONN
        BZ      qt1
        FNBEGIN NETDEV, NCCLOSE, 0
        GOACK
qt1:    FCALL   fn_config       ; never returns...
        FJMP    fail            ; ...if not armed

url:     NC_URL
t_okeys: DB     "5:ADD 0:DEL 9:OK", 0
t_url:   DB     "URL, 1:QUIT", 0
t_line:  DB     "SEND, 1:BACK", 0
t_conn:  DB     "CONNECTING", 0
t_keys:  DB     "5:TYPE  0:QUIT  ", 0
t_gone:  DB     "CLOSED  0:QUIT  ", 0

        S2CLAIM
```

osk.inc asm/osk.inc

```
; osk.inc -- the on-screen keyboard netcat uses:
; CONFIG's own, copied unchanged from fujinet-config
; 86c763d between the >>> and <<< lines
; (tools/vendorcheck.py holds them to it). The
; program supplies st_edit, ed_ok, ed_esc and the
; RAM: V_EDIT, V_EDLEN, V_EDMAX, V_OSK, V_KOLD and
; V_KTO, with RB.1 = HI(V_APP) throughout.

; >>> fujinet-config 86c763d studio2/src/cfgdefs.inc: the macros
; D = var.
LDV     MACRO   var
        LDI     LO(var)
        PLO     RB
        LDN     RB
        ENDM

; var = D. Leaves D, and RB on var. Uses RD.1.
STV     MACRO   var
        PHI     RD
        LDI     LO(var)
        PLO     RB
        GHI     RD
        STR     RB
        ENDM

; var = constant.
SETV    MACRO   var, val
        LDI     LO(var)
        PLO     RB
        LDI     val
        STR     RB
        ENDM

; RB = var.
PTV     MACRO   var
        LDI     LO(var)
        PLO     RB
        ENDM

; The 2-byte var = adr, high byte first (go_ind).
SETW    MACRO   var, adr
        LDI     LO(var)
        PLO     RB
        LDI     HI(adr)
        STR     RB
        INC     RB
```

```
        LDI     LO(adr)
        STR     RB
        ENDM

; Every state starts with an empty stack: R2.1 never leaves $09.
STATE   MACRO
        LDI     LO(STACK)
        PLO     R2
        ENDM

; One kdisp table entry.
KEYTO   MACRO   k, adr
        DB      k
        DW      adr
        ENDM

; FIT that also keeps out of $x000-$x3FF: `size` bytes start on the next
; page if they would cross one, and at $x400 if that page is $x000-$x3FF.
RFIT    MACRO   size
        IF      ((($) & 0FFH) + (size)) > 100H
        ORG     (($) + 0FFH) & 0FF00H
        ENDIF
        IF      (($) & 0C00H) == 0
        ORG     (($) & 0F000H) + 0400H
        ENDIF
        ENDM
; <<<

; >>> fujinet-config 86c763d studio2/src/ui.inc: keys and dispatch
; key_wait -- D = the next key, 0-9, from either pad. A held key repeats
; only for 2/4/6/8: a held select must not fire twice.
; key_tmo  -- the same, but D = $FF once D seconds pass without one.
        RFIT    67
key_tmo: FCALL  fn_frames
        LDI     1
        BR      kw0
key_wait:
        LDI     0
kw0:    STV     V_KTO
kw1:    LDV     V_KLAST
        STV     V_KOLD
        FCALL   in_get
        BNZ     kw2
        LDV     V_KTO
        BZ      kw1
        FCALL   fn_tick
        BDF     kw1
        LDI     0FFH
        FRET
kw2:    STR     R2
        LDV     V_KOLD
        XOR
        BNZ     kw3                     ; a fresh press
        LDN     R2
        ANI     0FH
        BZ      kw1
        SHR
        BDF     kw1                     ; an odd key: no repeat
kw3:    LDI     2
        PLO     RD                      ; a click
        LDN     R2
        ANI     0FH
        FRET

; kdisp -- FCALL kdisp with D = a key, then KEYTO entries and DB $FF: go to
; the first entry's address whose key is D (with D kept), else return past
; the table. The jump is a return with R6 swapped, so the stack balances.
        RFIT    27
kdisp:  PLO     R9
kd1:    LDA     R6
        XRI     0FFH
        BZ      kd3                     ; the end of the table
        XRI     0FFH
        STR     R2
        GLO     R9
        XOR
        BZ      kd2
        INC     R6
        INC     R6
        BR      kd1
kd2:    LDA     R6
        PHI     R9
        LDN     R6
        PLO     R6
        GHI     R9
        PHI     R6
        GLO     R9
        FRET
kd3:    GLO     R9
        FRET
; <<<

; >>> fujinet-config 86c763d studio2/src/edit.inc: moves, add, delete
        RFIT    34
ed_in:  FCALL   key_wait
        FCALL   kdisp
        KEYTO   2, ed_up
        KEYTO   8, ed_dn
        KEYTO   4, ed_lt
        KEYTO   6, ed_rt
        KEYTO   5, ed_add
        KEYTO   0, ed_del
        KEYTO   9, ed_ok
        KEYTO   1, ed_esc
        DB      0FFH
        FJMP    ed_in

; Moves wrap at the edges. The old cell is repainted plain first.
        RFIT    117
ed_up:  LDV     V_OSK
        FCALL   ed_cell
        LDV     V_OSK
        SMI     16
        BDF     ed_mv
        ADI     96
        BR      ed_mv
ed_dn:  LDV     V_OSK
        FCALL   ed_cell
        LDV     V_OSK
        ADI     16
        STR     R2
        SMI     96
        BDF     ed_mv
        LDN     R2
        BR      ed_mv
ed_lt:  LDV     V_OSK
        FCALL   ed_cell
        LDV     V_OSK
        ANI     0FH
        BZ      edl1
        LDV     V_OSK
        SMI     1
        BR      ed_mv
edl1:   LDV     V_OSK
        ADI     15
        BR      ed_mv
ed_rt:  LDV     V_OSK
        FCALL   ed_cell
        LDV     V_OSK
        ANI     0FH
        XRI     0FH
        BZ      edr1
        LDV     V_OSK
        ADI     1
        BR      ed_mv
edr1:   LDV     V_OSK
        SMI     15
ed_mv:  STV     V_OSK
        FCALL   ed_sel
        FCALL   ed_text
        FJMP    ed_in

; ed_add -- type the selected character, if there is room ($7F types nothing).
        RFIT    77
ed_add: LDV     V_OSK
        XRI     7FH-20H
        BZ      ed_bad
        LDV     V_EDLEN
        STR     R2
        LDV     V_EDMAX
        SM
        BZ      ed_bad                  ; full
        LDN     R2
        ADI     LO(V_EDIT)
        PLO     R8
        LDI     HI(V_EDIT)
        PHI     R8
        LDV     V_OSK
        ADI     20H
        STR     R8
        INC     R8
        LDI     0
        STR     R8
        LDV     V_EDLEN
        ADI     1
        STR     RB
        BR      ed_shw
ed_del: LDV     V_EDLEN
        BZ      ed_bad
        SMI     1
        STR     RB
        ADI     LO(V_EDIT)
        PLO     R8
        LDI     HI(V_EDIT)
        PHI     R8
        LDI     0
        STR     R8
ed_shw: FCALL   ed_text
        FJMP    ed_in
ed_bad: LDI     8
        PLO     RD
        FJMP    ed_in
; <<<

; >>> fujinet-config 86c763d studio2/src/edit.inc: the grid and the text
; ed_grid -- characters $20-$7F from row 3: the engine wraps each row.
        RFIT    24
ed_grid:
        LDI     30H
        FCALL   d_at
        LDI     20H
        PLO     R8
        LDI     TX_PUTC
        PHI     RF
edg1:   GLO     R8
        PLO     RF
        LDN     RF
        INC     R8
        GLO     R8
        XRI     80H
        BNZ     edg1
        LDI     FN_PARK
        PHI     RF
        FRET

; ed_sel  -- paint cell D inverse: the keyboard's cursor.
; ed_cell -- paint cell D plain.
        RFIT    27
ed_sel: PLO     R9
        LDI     TOP_INV_ON
        FCALL   d_op
        GLO     R9
        FCALL   ed_cell
        LDI     TOP_INV_OFF
        FJMP    d_op
ed_cell:
        PLO     R9
        ADI     30H
        FCALL   d_at
        GLO     R9
        ADI     20H
        FJMP    d_putc

; ed_text -- row 1: the text's last 14 characters, then the selected key as
; an inverse block cursor (what 5 would type), short of the row's end so
; CLREOL never wraps onto row 2.
        RFIT    54
ed_text:
        LDI     10H
        FCALL   d_at
        LDV     V_EDLEN
        SMI     15
        BNF     edt1
        LDV     V_EDLEN
        ADI     LO(V_EDIT)-14
        BR      edt2
edt1:   LDI     LO(V_EDIT)
edt2:   PLO     R8
        LDI     HI(V_EDIT)
        PHI     R8
        FCALL   d_puts
        LDI     TOP_INV_ON
        FCALL   d_op
        LDV     V_OSK
        ADI     20H
        FCALL   d_putc
        LDI     TOP_INV_OFF
        FCALL   d_op
        LDI     TOP_CLREOL
        FJMP    d_op
; <<<
```

APPENDIX D

Game and CONFIG Listings

The files the game and CONFIG chapters discuss, at the commits in the Parts List. The rest of each program is in its repository, and in `listings/`.

Battleship

tiles.inc battleship/src/tiles.inc

```
; tiles.inc -- the 16 mosaic tiles (the cart's TOP_TILE): four scanlines of
; two pixels, bit 1 the left. One tile is one square of a board, 2 pixels by 4
; scanlines, and a text cell holds 2 x 3 of them (board.inc). On a TV a pixel
; is about three scanlines wide, so a square is a little wider than tall.
;
; A board is a lit field and the marks are dark: black is a ship -- a hit on
; an opponent, or our own hull -- a hatched square is a hit on our hull, and a
; dark dot is a miss. Lit, the four boards stand out as four panels; dark,
; with only their marks lit, they could not be told from the black around them.
;
; A square's tile is its gamefield value ORed with MAP's mark, so the numbers
; are not free: 0-2 are the wire's 0 unknown / 1 hit / 2 miss, T_HULL (4) and
; T_CUR (8) add on top of them, and the rest fill the holes.

T_SEA   EQU     0               ; open water, or not yet shot at
T_HIT   EQU     1
T_MISS  EQU     2
T_PIP   EQU     3               ; a fleet pip: the ship is afloat
T_HULL  EQU     4               ; a hull (ours, or the winner's revealed)
T_HHIT  EQU     5               ; a hull that has been hit
T_PIPX  EQU     7               ; a fleet pip: sunk
T_CUR   EQU     8               ; the aiming cursor over 0-2 (8-10): blinks
T_PLC   EQU     11              ; the ship being placed: blinks
T_BLANK EQU     12              ; nothing: the pip row's gaps, the row below
T_BAD   EQU     15              ; the ship being placed, on another: blinks

; Scanline rows, top to bottom: "##" is 3, "#." 2, ".#" 1, ".." 0.
tiles:  DB      3, 3, 3, 3              ; 0  T_SEA    ##  ##  ##  ##
        DB      0, 0, 0, 0              ; 1  T_HIT    ..  ..  ..  ..
        DB      3, 1, 1, 3              ; 2  T_MISS   ##  .#  .#  ##
        DB      0, 3, 3, 0              ; 3  T_PIP    ..  ##  ##  ..
        DB      0, 0, 0, 0              ; 4  T_HULL   ..  ..  ..  ..
        DB      2, 1, 2, 1              ; 5  T_HHIT   #.  .#  #.  .#
        DB      0, 0, 0, 0              ; 6  (a miss on a hull cannot be)
        DB      0, 0, 2, 0              ; 7  T_PIPX   ..  ..  #.  ..
        DB      3, 3, 3, 3              ; 8-10, 11: tblink's
        DB      0, 0, 0, 0
        DB      3, 1, 1, 3
        DB      0, 0, 0, 0
        DB      0, 0, 0, 0              ; 12 T_BLANK
        DB      0, 0, 0, 0              ; 13, 14 unused
        DB      0, 0, 0, 0
        DB      2, 1, 2, 1              ; 15: tblink's

; The blinking tiles, each "on" then "off": the cursor inverts what it is
; over; a ship being placed flickers dark / hatched, and where it overlaps
; another it buzzes between two stripes.
blinks: DB      T_CUR+T_SEA
        DB      0, 0, 0, 0,  3, 3, 3, 3
        DB      T_CUR+T_HIT
        DB      3, 3, 3, 3,  0, 0, 0, 0
        DB      T_CUR+T_MISS
        DB      0, 2, 2, 0,  3, 1, 1, 3
        DB      T_PLC
        DB      0, 0, 0, 0,  2, 1, 2, 1
        DB      T_BAD
        DB      3, 0, 3, 0,  0, 3, 0, 3
NBLINK  EQU     5
BLINKF  EQU     12              ; frames a phase

; tinit -- define all sixteen tiles.
        FIT     40
tinit:  LDR     RB, tiles
        LDI     0
        PLO     RC
        GLO     RC
ti1:    HOT     TX_PARAM
        LDI     4
        PHI     RC
ti2:    LDA     RB
        HOT     TX_PARAM
        GHI     RC
        SMI     1
        PHI     RC
        BNZ     ti2
        LDI     TOP_TILE
        HOT     TX_OP
        INC     RC
        GLO     RC
        XRI     16
        BNZ     ti1
        FRET

; tblink -- every BLINKF frames, flip V_BLNK and redefine the blinking
; tiles; the cart redraws just the cells that hold them.
        FIT     80
tblink: GETV    V_BLT
        STR     R2
        GLO     RE
        SM
        SMI     BLINKF
        BNF     tbl9                    ; not yet
        GLO     RE
        PUTV    V_BLT
        GETV    V_BLNK
        XRI     1
        STR     RA
        LDR     RB, blinks
        LDI     NBLINK
        PLO     RC
tbl1:   LDA     RB                      ; the tile
        HOT     TX_PARAM
        GETV    V_BLNK
        BNZ     tbl2
        INC     RB                      ; phase 0 is "off": skip "on"
        INC     RB
        INC     RB
        INC     RB
tbl2:   LDI     4
        PHI     RC
tbl3:   LDA     RB
        HOT     TX_PARAM
        GHI     RC
        SMI     1
        PHI     RC
        BNZ     tbl3
        GETV    V_BLNK
        BZ      tbl4
        INC     RB                      ; phase 1 is "on": skip "off"
        INC     RB
        INC     RB
        INC     RB
tbl4:   LDI     TOP_TILE
        HOT     TX_OP
        DEC     RC
        GLO     RC
        BNZ     tbl1
tbl9:   FRET
```

board.inc battleship/src/board.inc

```
; board.inc -- the four boards, in the cart's mosaic cells.
;
; A board is 5 x 4 text cells, each a mosaic of 2 x 3 squares (TOP_MOSAIC),
; so 10 x 12 squares:
;
;   square row 0      five fleet pips, one a cell (shipsLeft: T_PIP / T_PIPX)
;   square rows 1-10  the field, lit (tiles.inc)
;   square row 11     T_BLANK: the gap above the next name row
;
; A square's tile is its gamefield byte (0 unknown, 1 hit, 2 miss) ORed with
; its MAP byte (0, T_HULL, T_CUR, or placement's T_PLC / T_BAD). A board with
; no gamefield yet (placement) reads ZEROS for it, and a board with no marks
; reads ZEROS for MAP, so one loop draws every case. A text row is fifteen
; tile pairs (5 cells x 3 thirds) sent as one TOP_MOSAIC: four ops a board.
;
; The seats sit where the C ports put them: us bottom-left, then clockwise
; from top-left. Each board has its name row above it.
;
;   col  0-4   5  6-10  11  12-15
;   row 0  [seat 1 name]  [seat 2 name]   panel (game.inc)
;   1-4    [seat 1 board] [seat 2 board]
;   5      [YOU]          [seat 3 name]
;   6-9    [our board]    [seat 3 board]

; ---- the data page: page-aligned, so a table is reached by its low byte ----
ZEROS:  DB      100 DUP (0)             ; no gamefield, or no marks
ONES:   DB      NSHIPS DUP (1)          ; no shipsLeft: all afloat
mul10:  DB      0, 10, 20, 30, 40, 50, 60, 70, 80, 90
trow:   DB      0, 0, 1, 1, 1, 2, 2, 2, 3, 3    ; square row -> text row
sizes:  DB      5, 4, 3, 3, 2           ; the fleet (gameLogic.go SHIP_SIZES)
orgrc:  DB      60H, 10H, 16H, 66H      ; each seat's first board cell, $rc
dxt:    DB      0, 9, 0, 1, 9, 0, 1, 9, 0, 1    ; aiming: each digit's step,
dyt:    DB      0, 9, 9, 9, 0, 0, 0, 1, 1, 1    ; -1 as 9, the keypad's layout
        IF      HI($) <> HI(ZEROS)
        ERROR   "board.inc's tables cross a page"
        ENDIF

; tbyte -- D = the byte at table + D, for the data page's tables. Clobbers R8.
tbyte:  PLO     R8
        LDI     HI(ZEROS)
        PHI     R8
        LDN     R8
        FRET

; ---- the composer ------------------------------------------------------------

; bsetup -- V_BGF and V_BPIP for seat V_BSEAT: its gamefield and shipsLeft in
; the reply, or ZEROS and ONES while there are none (placement, status 1).
; V_BMH is the caller's.
        FIT     80
bsetup: GETV    V_ST
        SMI     ST_START
        BDF     bsu1                    ; 10-13 and 99 carry gamefields
        VAR     V_BGF
        LDI     HI(ZEROS)
        STR     RA
        INC     RA
        LDI     LO(ZEROS)
        STR     RA
        INC     RA                      ; V_BPIP
        LDI     HI(ONES)
        STR     RA
        INC     RA
        LDI     LO(ONES)
```

```
        STR     RA
        FRET
bsu1:   GETV    V_BSEAT
        FCALL   prec
        LDI     PGF
        FCALL   radd
        VAR     V_BGF
        GHI     RB
        STR     RA
        INC     RA
        GLO     RB
        STR     RA
        LDI     PLEFT-PGF
        FCALL   radd
        VAR     V_BPIP
        GHI     RB
        STR     RA
        INC     RA
        GLO     RB
        STR     RA
        FRET

; bdrow -- text row V_BTR (0-3) of seat V_BSEAT's board. The parameters go
; into V_BUF cell by cell (a cell's top, middle and bottom thirds), built a
; third at a time: RB walks V_BUF + third in steps of three.
        FIT     200
bdrow:  SETV    V_BK, 0
bdk:    LDI     HI(V_BUF)
        PHI     RB
        GETV    V_BK
        ADI     LO(V_BUF)
        PLO     RB
        GETV    V_BTR                   ; the square row: 3 * row + third
        STR     R2
        SHL
        ADD
        STR     R2
        GETV    V_BK
        ADD
        BZ      bdpip                   ; 0: the pips
        SMI     11
        BZ      bdpad                   ; 11: the gap
        ADI     10                      ; 1-10: field row y = that - 1
        ADI     LO(mul10)
        FCALL   tbyte
        PLO     R9                      ; R9 = MAP (or ZEROS) + y * 10
        GETV    V_BMH
        PHI     R9
        VAR     V_BGF+1                 ; R8 = the gamefield + y * 10
        GLO     R9
        SEX     RA
        ADD
        PLO     R8
        DEC     RA
        LDI     0
        ADC
        PHI     R8
        SEX     R2
        LDI     5
        PHI     RC
bdc:    LDA     R8                      ; left square: high nibble
        ANI     3
        STR     R2
        LDA     R9
        OR
        SHL
        SHL
        SHL
        SHL
        STR     RB
        LDA     R8                      ; right square: low nibble
        ANI     3
        STR     R2
        LDA     R9
        OR
        SEX     RB
        OR
        SEX     R2
        STR     RB
        INC     RB
        INC     RB
        INC     RB
        GHI     RC
        SMI     1
        PHI     RC
        BNZ     bdc
        BR      bdnext
bdpip:  VAR     V_BPIP                  ; a pip in each cell's left square
        LDA     RA
        PHI     R8
        LDN     RA
        PLO     R8
        LDI     NSHIPS
        PHI     RC
bdp1:   LDA     R8
        BZ      bdp2
        LDI     T_PIP*16+T_BLANK
        BR      bdp3
bdp2:   LDI     T_PIPX*16+T_BLANK
bdp3:   STR     RB
        INC     RB
        INC     RB
        INC     RB
        GHI     RC
        SMI     1
        PHI     RC
        BNZ     bdp1
        BR      bdnext
bdpad:  LDI     5
        PHI     RC
bdz1:   LDI     T_BLANK*16+T_BLANK
        STR     RB
        INC     RB
        INC     RB
        INC     RB
        GHI     RC
        SMI     1
        PHI     RC
        BNZ     bdz1
bdnext: GETV    V_BK
        ADI     1
        STR     RA
        XRI     3
        BNZ     bdk
        GETV    V_BSEAT                 ; the cursor: the seat's first cell,
        ADI     LO(orgrc)               ; down by the row
        FCALL   tbyte
        STR     R2
        GETV    V_BTR
        SHL
        SHL
        SHL
        SHL
        ADD
        HOT     TX_CURSOR
        LDR     RB, V_BUF
        LDI     TX_PARAM
        PHI     RF
        LDI     15
        PLO     RC
bde1:   LDA     RB
        PLO     RF
        LDN     RF
        DEC     RC
        GLO     RC
        BNZ     bde1
        LDI     FN_PARK
        PHI     RF
        LDI     TOP_MOSAIC
        HOT     TX_OP
        FRET

; bdraw -- the whole of seat V_BSEAT's board; V_BMH is the caller's. The
; keypads are looked at between rows: a board is ~20 ms of drawing.
        FIT     32
bdraw:  FCALL   bsetup
        SETV    V_BTR, 0
bdr1:   FCALL   bdrow
        FCALL   kpoll
        GETV    V_BTR
        ADI     1
        STR     RA
        XRI     4
        BNZ     bdr1
        FRET

; bblank -- seat V_BSEAT's name row and board as blanks: an empty seat.
        FIT     40
bblank: GETV    V_BSEAT
        ADI     LO(orgrc)
        FCALL   tbyte
        SMI     10H                     ; the name row
        PLO     RC
        LDI     5
        PHI     RC
bbl1:   GLO     RC
        FCALL   d_at
        LDI     5
        FCALL   blanks
        GLO     RC
        ADI     10H
        PLO     RC
        GHI     RC
        SMI     1
        PHI     RC
        BNZ     bbl1
        FRET

; ---- MAP ---------------------------------------------------------------

; mclear -- MAP all zero.
        FIT     16
mclear: LDR     R8, MAP+99
        SEX     R8
mcl1:   LDI     0
        STXD
        GLO     R8
        XRI     0FFH
        BNZ     mcl1
        SEX     R2
        FRET

; mstart -- the ship V_MX, V_MY, V_MD, V_ML: R8 = MAP + its first square,
; R9.0 = its step (1 across, 10 down), RC.0 = its length, and DF = 1; DF = 0
; if any of it is off the board.
        FIT     72
mstart: GETV    V_MX
        SMI     10
        BDF     msbad
        GETV    V_MY
        SMI     10
        BDF     msbad
        GETV    V_MD
        BZ      ms1
        GETV    V_MY
        BR      ms2
ms1:    GETV    V_MX
ms2:    STR     R2
        GETV    V_ML
        PLO     RC
        ADD
        SDI     10                      ; 10 - (start + length): no borrow, fits
        BNF     ms9
        GETV    V_MY
        ADI     LO(mul10)
        FCALL   tbyte
        STR     R2
        GETV    V_MX
        ADD
        PLO     R8
        LDI     HI(MAP)
        PHI     R8
        GETV    V_MD
        BZ      ms3
        LDI     10
        BR      ms4
ms3:    LDI     1
ms4:    PLO     R9
        SMI     0                       ; DF = 1
ms9:    FRET
msbad:  LDI     0
        ADI     0                       ; DF = 0
        FRET

; mfits -- DF = 1 if the ship (mstart's) is on the board over empty squares.
        FIT     40
mfits:  FCALL   mstart
        BNF     mf9
mf1:    LDN     R8
        BNZ     mf8
        GLO     R9
        STR     R2
        GLO     R8
        ADD
        PLO     R8
        DEC     RC
        GLO     RC
        BNZ     mf1
        SMI     0                       ; D = 0: DF = 1
        FRET
mf8:    ADI     0                       ; D != 0 and no carry: DF = 0
mf9:    FRET

; mmark -- the ship's squares (mstart's) = V_MT; where V_MT is T_PLC and a
; square is already marked, T_BAD instead, and V_MBAD = 1. DF as mstart.
        FIT     72
mmark:  FCALL   mstart
        BNF     mm9
mm1:    LDN     R8
        BZ      mm2
```

```
335         GETV    V_MT
336         XRI     T_PLC
337         BNZ     mm2
338         SETV    V_MBAD, 1
339         LDI     T_BAD
340         BR      mm3
341 mm2:    GETV    V_MT
342 mm3:    STR     R8
343         GLO     R9
344         STR     R2
345         GLO     R8
346         ADD
347         PLO     R8
348         DEC     RC
349         GLO     RC
350         BNZ     mm1
351         SMI     0                       ; DF = 1
352 mm9:    FRET
353
354 ; mdec -- V_MX, V_MY, V_MD from D = pos + 100 * dir, the wire's
    placement.
355 ; DF = 0 if D is past 199.
356         FIT     64
357 mdec:   PLO     R9
358         SETV    V_MD, 0
359         GLO     R9
360         SMI     100
361         BNF     md1
362         PLO     R9
363         SETV    V_MD, 1
364         GLO     R9
365         SMI     100
366         BDF     mdbad
367 md1:    LDI     0                       ; R9.1 = pos / 10, R9.0
    = pos % 10
368         PHI     R9
369 md2:    GLO     R9
370         SMI     10
371         BNF     md3
372         PLO     R9
373         GHI     R9
374         ADI     1
375         PHI     R9
376         BR      md2
377 md3:    GLO     R9
378         PUTV    V_MX
379         GHI     R9
380         PUTV    V_MY
381         SMI     0                       ; DF = 1
382         FRET
383 mdbad:  LDI     0
384         ADI     0                       ; DF = 0
385         FRET
386
387 ; mships -- MAP gets the five wire placements at RB as T_HULL.
    RB is left
388 ; past them.
389         FIT     56
390 mships: SETV    V_CTR, 0
391 msh1:   LDA     RB
392         FCALL   mdec
393         BNF     msh2
394         GETV    V_CTR
395         ADI     LO(sizes)
396         FCALL   tbyte
397         PUTV    V_ML
398         SETV    V_MT, T_HULL
399         FCALL   mmark
400 msh2:   GETV    V_CTR
401         ADI     1
402         STR     RA
403         XRI     NSHIPS
404         BNZ     msh1
405         FRET
406
407 ; mcur -- MAP all zero but the aiming cursor's square, T_CUR.
408         FIT     24
409 mcur:   FCALL   mclear
410         FCALL   cpos
411         PLO     R8
412         LDI     HI(MAP)
413         PHI     R8
414         LDI     T_CUR
415         STR     R8
416         FRET
417
418 ; cpos -- D = the cursor's square, V_CY * 10 + V_CX.
419         FIT     24
420 cpos:   GETV    V_CY
421         ADI     LO(mul10)
422         FCALL   tbyte
423         STR     R2
424         GETV    V_CX
425         ADD
426         FRET
427
```

net.inc battleship/src/net.inc

```
1  ; net.inc -- the N: device round trip, and the name appkey. (5
   Card Stud's,
2  ; as is.)
3  ;
4  ; A 1802 transcription of the Arcadia and Channel F sequence
   (which is the
5  ; Astrocade's, which is the Odyssey 2's). The rules it carries
   forward, each
6  ; learned the hard way on an earlier console:
7  ;
8  ;  * The CLOSE belongs at the START of the next request, never
   after the
9  ;    READ: every transaction repaints the reply window, a CLOSE
   reply is
10 ;    empty, and the screens render straight out of the window
   between polls.
11 ;  * The reply length belongs to the latest transaction, so it
   is taken
12 ;    immediately after the READ.
13 ;  * The adapter reports STATUS as soon as SOME of the response
   is in: poll
14 ;    until two readings agree AND at least a known minimum has
   arrived
15 ;    (Channel F's sixth clause), bounded, rather than guessing
   a delay.
16 ;  * STATUS byte 3 is nDevStatus_t and checking it is not
   optional: the GET
17 ;    is deferred until the first STATUS, and an HTTP error page
   still has a
18 ;    readable body.
19 ;  * Validate the reply length against what playerCount implies
   before
20 ;    believing a byte of it.
21 ;
22 ; Routines return D = 0 on success, nonzero on failure, and may
   clobber R8,
23 ; R9, RB, RC and RA.0.
24
25 ; nbeg -- begin an N1: transaction: R9.0 = command, D =
   parameter count.
26 ; The timeout outlasts the cart's own 90 s, so the cart always
   answers.
27 nbeg:   PHI     R8
28         LDI     NDEV
29         PHI     R9
30         GHI     R8
31         FCALL   fn_beg
32         SETV    V_TMO, NETTMO
33         FRET
34
35 ; ngo -- fujilib's fn_go and fn_ack in one, scanning the
   keypads while it
36 ; waits. D = 0 on an ACK; FNEWAIT, FNENAK or the cart's FN_ERR
   otherwise.
37 ; The sequence number is the cart's ACKSEQ + 1, never a counter
   in RAM.
38         FIT     96
39 ngo:    PEEK    FN_ACKSEQ
40         ADI     1
41         BNZ     ngo1
42         LDI     1                       ; 0 is "never used"
43 ngo1:   PUTV    V_SEQ
44         LDI     FN_REG_SEQ
45         PLO     R8
46         GETV    V_SEQ
47         FCALL   fn_rw                   ; this pair launches it
48         GETV    V_TMO
49         FCALL   fn_frames
50 ngo2:   PEEK    FN_ACKSEQ               ; published last, after
   the reply
51         STR     R2
52         GETV    V_SEQ
53         XOR
54         BZ      ngo3
55         FCALL   kpoll
56         FCALL   fn_tick
57         BDF     ngo2
58         LDI     FNEWAIT
59         FRET
60 ngo3:   PEEK    FN_ERR
61         BNZ     ngo9
62         PEEK    FN_REPLYCMD
63         XRI     06H
64         BZ      ngo9
65         LDI     FNENAK
66 ngo9:   FRET
67
68 ; nclose -- close the previous request. The first one has
   nothing to close
69 ; and earns a NAK; either way is fine.
70 nclose: LDI     NCCLOSE
71         PLO     R9
72         LDI     0
73         FCALL   nbeg
74         FCALL   ngo
75         LDI     0
76         FRET
77
78 ; nopen -- OPEN the request V_REQ names as an HTTP GET, no
   translation.
79 nopen:  LDI     NCOPEN
80         PLO     R9
81         LDI     2
82         FCALL   nbeg
83         LDI     NMHTTPG
84         FCALL   fn_pb
85         LDI     NTRNONE
86         FCALL   fn_pb
87         FCALL   bldurl                  ; the devicespec is the
   payload
88         FJMP    ngo
89
90 ; nstatus -- one STATUS: V_AVL = bytes waiting.
91         FIT     48
92 nstatus:
93         LDI     NCSTAT
94         PLO     R9
95         LDI     0
96         FCALL   nbeg
97         FCALL   ngo
98         BNZ     nst9
99         PEEK    FN_REPLY+NSDEVST
100        XRI     1
101        BNZ     nst9                    ; anything but SUCCESS
102        VAR     V_AVL
103        PEEK    FN_REPLY
104        STR     RA
105        INC     RA
106        PEEK    FN_REPLY+1
107        STR     RA
108        LDI     0
109 nst9:  FRET
110
111 ; nsettle -- STATUS until V_AVL >= V_MIN and two readings
   agree, bounded.
112        FIT     112
113 nsettle:
114        SETV    V_TRY, 60
115        VAR     V_PRV
116        LDI     0
117        STR     RA
118        INC     RA
119        STR     RA
120 nse1:  FCALL   nstatus
121        BNZ     nse9
122        LDI     HI(V_AVL)
123        PHI     RB
124        LDI     LO(V_AVL)
125        PLO     RB
126        VAR     V_MIN                   ; enough yet? avail -
   min, unsigned
127        SEX     RA
128        LDA     RB
129        SM
130        INC     RA
131        LDN     RB
132        SMB
133        SEX     R2
134        BNF     nse2
135        DEC     RB                      ; and the same as the
   reading before?
136        VAR     V_PRV
137        SEX     RA
138        LDA     RB
139        XOR
140        BNZ     nse2x
141        INC     RA
142        LDN     RB
143        XOR
144 nse2x: SEX     R2
145        BZ      nse9                    ; D = 0: settled
146 nse2:  VAR     V_AVL                   ; remember this reading
147        LDA     RA
148        PLO     R8
149        LDN     RA
150        PHI     R8
151        VAR     V_PRV
152        GLO     R8
153        STR     RA
154        INC     RA
155        GHI     R8
156        STR     RA
157        LDI     6                       ; 100 ms, not a handful
   of cycles
158        FCALL   wframes
159        VAR     V_TRY
160        LDN     RA
161        SMI     1
162        STR     RA
163        BNZ     nse1
```

```
        LDI     FE_STAT
nse9:   FRET

; nread -- READ min(V_AVL, V_CAP), then take the reply length.
        FIT     80
nread:  LDI     NCREAD
        PLO     R9
        LDI     1
        FCALL   nbeg
        LDI     HI(V_AVL)
        PHI     RB
        LDI     LO(V_AVL)
        PLO     RB
        VAR     V_CAP                   ; avail - cap: borrow means avail
        SEX     RA
        LDA     RB
        SM
        INC     RA
        LDN     RB
        SMB
        SEX     R2
        BNF     nrd1
        VAR     V_CAP
        LDA     RA
        PLO     R8
        LDN     RA
        PHI     R8
        BR      nrd2
nrd1:   DEC     RB
        LDA     RB
        PLO     R8
        LDN     RB
        PHI     R8
nrd2:   FCALL   fn_pw
        FCALL   ngo
        BNZ     nrd9
        VAR     V_RXL                   ; now: the next transaction replaces it
        PEEK    FN_RXLEN
        STR     RA
        INC     RA
        PEEK    FN_RXLEN+1
        STR     RA
        LDI     0
nrd9:   FRET

; nfetch -- the whole cycle for V_REQ: close the last request, open this
; one, let STATUS settle on V_MIN, read up to V_CAP. D = 0 or an FE_ code.
        FIT     40
nfetch: FCALL   nclose
        FCALL   nopen
        BNZ     nfo
        FCALL   nsettle
        BNZ     nfs
        FCALL   nread
        BNZ     nfr
        FRET
nfo:    LDI     FE_OPEN
        FRET
nfs:    LDI     FE_STAT
        FRET
nfr:    LDI     FE_READ
        FRET

; nleave -- tell the server we left V_TABLE. /leave?bin=1 answers no body
; at all, so this is OPEN and the one STATUS that runs the GET; no READ.
nleave: SETV    V_REQ, REQ_LEAVE
        FCALL   nclose
        FCALL   nopen
        FCALL   nstatus
        FCALL   nclose
        SETV    V_SEAT, 0
        FRET

; setlim -- V_MIN = R8, V_CAP = R9.
setlim: VAR     V_MIN
        GLO     R8
        STR     RA
        INC     RA
        GHI     R8
        STR     RA
        VAR     V_CAP
        GLO     R9
        STR     RA
        INC     RA
        GHI     R9
        STR     RA
        FRET

; vlen -- D = 0 if V_RXL == R8 + R9.0 * D (the count a header promised).
        FIT     40
vlen:   PLO     RC
vl1:    GLO     RC
        BZ      vl2
        GLO     R8
        STR     R2
        GLO     R9
        ADD
        PLO     R8
        GHI     R8
        ADCI    0
        PHI     R8
        DEC     RC
        BR      vl1
vl2:    VAR     V_RXL
        SEX     RA
        GLO     R8
        XOR
        BNZ     vl3
        INC     RA
        GHI     R8
        XOR
vl3:    SEX     R2
        FRET

; ---- the name appkey ----

; akopen -- OPEN_APPKEY for the shared username, mode D (AKREAD/AKWRITE).
; The payload is fujiDevice.h's packed struct appkey, all SIX bytes:
; creator (LE), app, key, mode, reserved. Five leaves the firmware waiting
; for a byte that never comes (the Odyssey 2 port's note).
akopen: PLO     RC
        LDI     FNDEVF
        PHI     R9
        LDI     FNCAKOP
        PLO     R9
        LDI     0
        FCALL   fn_beg
        LDI     1                       ; creator 1 (LE)
        FCALL   fn_txb
        LDI     0
        FCALL   fn_txb
        LDI     1                       ; app 1
        FCALL   fn_txb
        LDI     0                       ; key 0: the username
        FCALL   fn_txb
        GLO     RC
        FCALL   fn_txb
        LDI     0
        FCALL   fn_txb
        FJMP    ngo

; akread -- READ_APPKEY: the window holds u16 length, then the value.
akread: LDI     FNDEVF
        PHI     R9
        LDI     FNCAKRD
        PLO     R9
        LDI     0
        FCALL   fn_beg
        FJMP    ngo

; akwrite -- WRITE_APPKEY with V_NAME (its characters, no NUL).
akwrite:
        LDI     FNDEVF
        PHI     R9
        LDI     FNCAKWR
        PLO     R9
        LDI     0
        FCALL   fn_beg
        LDR     R8, V_NAME
        FCALL   txs
        FJMP    ngo
```

url.inc battleship/src/url.inc

```
; url.inc -- the devicespec, streamed into the TX stream a byte at a time and
; never assembled in RAM. A /place request is about 110 bytes of the 320 a
; transaction carries. The table id and the name are percent-encoded: the
; name may come from another console's appkey, and a space or an '&' in it
; would otherwise end the query early.

; txs -- append the NUL-terminated string at R8.
        FIT     16
txs:    LDI     HI(FN_TXDATA)
        PHI     RF
txs1:   LDA     R8
        BZ      txs9
        PLO     RF
        LDN     RF
        BR      txs1
txs9:   LDI     FN_PARK
        PHI     RF
        FRET

; txenc -- append the string at R8, every byte but A-Z a-z 0-9 - _ . ~ as %XX.
        FIT     80
txenc:  LDA     R8
        BZ      txe9
        PLO     R9
        XRI     '-'
        BZ      txok
        GLO     R9
        XRI     '_'
        BZ      txok
        GLO     R9
        XRI     '.'
        BZ      txok
        GLO     R9
        XRI     '~'
        BZ      txok
        GLO     R9
        SMI     '0'
        BNF     txpct
        SMI     10
        BNF     txok                    ; a digit
        GLO     R9
        ORI     20H
        SMI     'a'
        BNF     txpct
        SMI     26
        BNF     txok                    ; a letter
txpct:  LDI     '%'
        FCALL   fn_txb
        GLO     R9
        SHR
        SHR
        SHR
        SHR
        FCALL   hexc
        FCALL   fn_txb
        GLO     R9
        ANI     0FH
        FCALL   hexc
txok1:  FCALL   fn_txb
        BR      txenc
txok:   GLO     R9
        BR      txok1
txe9:   FRET

; hexc -- D = the hex digit for the nibble in D.
        FIT     12
hexc:   SMI     10
        BDF     hexc1
        ADI     '0'+10
        FRET
hexc1:  ADI     'A'
        FRET

; txdec -- append D (0-255) in decimal, no leading zeros.
        FIT     64
txdec:  PLO     R9
        LDI     0
        PHI     R9                      ; a digit has been sent
        LDI     100
        FCALL   txd1
        LDI     10
        FCALL   txd1
        GLO     R9
        ADI     '0'
        FJMP    fn_txb
txd1:   PLO     RC                      ; one digit: R9.0 / D
        LDI     0
        PHI     RC
txd2:   GLO     RC
        STR     R2
        GLO     R9
        SM
        BNF     txd3
        PLO     R9
        GHI     RC
        ADI     1
        PHI     RC
        BR      txd2
txd3:   GHI     RC
        BNZ     txd4
        GHI     R9
        BZ      txd5                    ; a leading zero
txd4:   LDI     1
        PHI     R9
        GHI     RC
        ADI     '0'
        FCALL   fn_txb
```

```
txd5:   FRET

; bldurl -- the devicespec for V_REQ: N:<endpoint> then
;   tables?bin=1
;   state | ready/1 | ready/0 | place/a,b,c,d,e | attack/p | leave,
;   then ?table=<id>&player=<name>&bin=1&v=2
; v=2 makes activePlayer the winner at game over and adds the winner's
; ships (util.go).
        FIT     112
bldurl: LDR     R8, UENDPT
        FCALL   txs
        GETV    V_REQ
        BNZ     bu1
        LDR     R8, utabls              ; no table, no player
        FJMP    txs
bu1:    SMI     REQ_ATTACK+1            ; past attack: leave
        BDF     bu7
        LDR     RB, upaths-2            ; a request with a fixed path
        GETV    V_REQ
        SHL
        FCALL   radd
        LDA     RB
        PHI     R8
        LDN     RB
        PLO     R8
        FCALL   txs
        GETV    V_REQ
        XRI     REQ_PLACE
        BNZ     bu2
        FCALL   txplace
        BR      bu8
bu2:    GETV    V_REQ
        XRI     REQ_ATTACK
        BNZ     bu8
        GETV    V_SHOT
        FCALL   txdec
        BR      bu8
bu7:    LDR     R8, uleave
        FCALL   txs
bu8:    LDR     R8, uqtab
        FCALL   txs
        LDR     R8, V_TABLE
        FCALL   txenc
        LDR     R8, uqplr
        FCALL   txs
        LDR     R8, V_NAME
        FCALL   txenc
        LDR     R8, uqbin
        FJMP    txs

; txplace -- the five placements, pos + 100 * dir, comma separated.
        FIT     64
txplace:
        SETV    V_CTR, 0
txp1:   GETV    V_CTR                   ; y * 10 + x
        ADI     LO(V_SY)
        PLO     RA
        LDN     RA
        ADI     LO(mul10)
        PLO     R8
        LDI     HI(mul10)
        PHI     R8
        LDN     R8
        STR     R2
        GETV    V_CTR
        ADI     LO(V_SX)
        PLO     RA
        LDN     RA
        ADD
        STR     R2
        GETV    V_CTR                   ; + 100 if down
        ADI     LO(V_SD)
        PLO     RA
        LDN     RA
        BZ      txp2
        LDI     100
txp2:   ADD
        FCALL   txdec
        GETV    V_CTR
        ADI     1
        STR     RA
        XRI     NSHIPS
        BZ      txp9
        LDI     ','
        FCALL   fn_txb
        BR      txp1
txp9:   FRET

upaths: DW      ustate, urdy1, urdy0, uplace, uattk
utabls: DB      "tables?bin=1", 0
ustate: DB      "state", 0
urdy1:  DB      "ready/1", 0
urdy0:  DB      "ready/0", 0
uplace: DB      "place/", 0
uattk:  DB      "attack/", 0
uleave: DB      "leave", 0
uqtab:  DB      "?table=", 0
uqplr:  DB      "&player=", 0
uqbin:  DB      "&bin=1&v=2", 0
```

5 Card Stud

cards.inc `5cardstud/src/cards.inc`

```
; cards.inc -- cards as two cells, [rank][suit], white on black.
;
; Not inverse on a white face: a 3x5 rank's outer strokes then merge into the
; gap beside it and a K loses its stem. The wire's 't' is drawn T. A hole card
; ("?") is a hatched app-glyph pair, the right cell a pixel narrower so two
; backs stay apart.

; glyphs -- teach the cart the app glyphs.
        FIT     32
glyphs: LDR     RB, gtab
gly1:   LDA     RB
        BZ      gly3
        FCALL   d_par                   ; the code
        LDI     12
        PLO     RC
gly2:   LDA     RB
        FCALL   d_par                   ; 12 scanlines, bit 3 the leftmost
        DEC     RC
        GLO     RC
        BNZ     gly2
        LDI     TOP_GLYPH
        FCALL   d_op
        BR      gly1
gly3:   FRET

gtab:   DB      G_BACKL, 0, 0AH, 05H, 0AH, 05H, 0AH, 05H, 0AH, 05H, 0AH, 05H, 0
        DB      G_BACKR, 0, 0AH, 04H, 0AH, 04H, 0AH, 04H, 0AH, 04H, 0AH, 04H, 0
        DB      0

; dhand -- the hand at RB: five cards, ten cells. A NUL rank ends the hand
; (the string is NUL-padded); the rest of the field is blanked.
        FIT     72
dhand:  LDI     5
        PLO     RC
dh1:    LDA     RB
        BZ      dh7
        PLO     R9
        XRI     '?'
        BNZ     dh2
        INC     RB                      ; the suit is '?' too
        LDI     G_BACKL
        FCALL   d_putc
        LDI     G_BACKR
        BR      dh6
dh2:    GLO     R9                      ; a rank: digits stand, letters fold
        SMI     'a'
        BNF     dh3
        GLO     R9
        SMI     20H
        BR      dh4
dh3:    GLO     R9
dh4:    FCALL   d_putc
        LDA     RB                      ; the suit
        PLO     R9
        LDR     R8, suits
dh5:    LDA     R8
        BZ      dh5x                    ; not a suit: '?'
        STR     R2
        GLO     R9
        XOR
        BZ      dh5x
        INC     R8
        BR      dh5
dh5x:   LDN     R8
dh6:    FCALL   d_putc
        DEC     RC
        GLO     RC
        BNZ     dh1
        FRET
dh7:    GLO     RC                      ; two blank cells a card left
        SHL
        FJMP    blanks

suits:  DB      'h', SYM_HEART, 'd', SYM_DIAMOND, 'c', SYM_CLUB, 's', SYM_SPADE
        DB      0, '?'
```

CONFIG

fujicmd.inc `config/src/fujicmd.inc`

```
; fujicmd.inc -- one wrapper per FujiNet command CONFIG sends.
;
; Each returns D = 0 on an ACK, else the FNE* error (FNENAK for a NAK), so a
; caller tests it with BZ. Payload rules the server enforces:
;   - OPEN_DIRECTORY and SET_DEVICE_FULLPATH read exactly 256 path bytes,
;     NUL-padded; all-NUL padding is OPEN_DIRECTORY's empty filter.
;   - SET_SSID is exactly 97 bytes: ssid[33] then password[64].
;   - WRITE_HOST_SLOTS is exactly 8 x 32.
;   - COPY_FILE is "source|destination" at its exact length, with 1-based
;     host slots where every other command's are 0-based.
;   - Never seek to directory position 0: an opened directory is there
;     already, and dir_seek(0) fails on an empty one.

; fc_beg -- FCALL fc_beg; DB command, nparam: begin a FUJI-device command.
        RFIT    9
fc_beg: LDI     FNDEVF
        PHI     R9
        LDA     R6
        PLO     R9
        LDA     R6
        FJMP    fn_beg                  ; its FRET returns past the two bytes

; go_ack -- commit, wait, and check the reply was an ACK.
        RFIT    9
go_ack: FCALL   fn_go
        BNZ     gak1
        FCALL   fn_ack
gak1:   FRET

; c_rhosts -- READ_HOST_SLOTS: 8 x 32 names in the reply.
        RFIT    8
c_rhosts:
        FCALL   fc_beg
        DB      FNCRHST, 0
        FJMP    go_ack

; c_whosts -- WRITE_HOST_SLOTS: the table in the reply (a fresh READ), with
; slot V_HSEL replaced by V_EDIT.
        RFIT    61
c_whosts:
        FCALL   fc_beg
        DB      FNCWHST, 0
        LDI     0
cwh1:   STV     V_I
        STR     R2
        LDV     V_HSEL
        XOR
        BNZ     cwh2
        LDR     R8, V_EDIT
        BR      cwh3
cwh2:   LDN     R2
        SHL
        SHL
        SHL
        SHL
        SHL
        PLO     R8
        LDI     HI(FN_REPLY)
        PHI     R8
cwh3:   LDI     32
        FCALL   tx_fix
        LDV     V_I
        ADI     1
        XRI     8
        BZ      cwh4
        XRI     8
        BR      cwh1
cwh4:   FJMP    go_ack
```

```
; c_mhost -- MOUNT_HOST(D = host slot).
        RFIT    21
c_mhost:
        STV     V_ARG
        FCALL   fc_beg
        DB      FNCMHST, 1
        LDV     V_ARG
        FCALL   fn_pb
        FJMP    go_ack

; c_odir -- OPEN_DIRECTORY(D = host slot) over V_PATH, empty filter.
        RFIT    32
c_odir: STV     V_ARG
        FCALL   fc_beg
        DB      FNCODIR, 1
        LDV     V_ARG
        FCALL   fn_pb
        LDR     R8, V_PATH
        LDI     0
        FCALL   tx_fix
        FJMP    go_ack

; c_spos -- SET_DIRECTORY_POSITION(V_POSL/H).
        RFIT    19
c_spos: FCALL   fc_beg
        DB      FNCSPOS, 1
        LDV     V_POSL
        PLO     R8
        INC     RB
        LDN     RB
        PHI     R8
        FCALL   fn_pw
        FJMP    go_ack

; c_rdblk -- READ_DIR_ENTRY in block mode: one group of NROWS entries (and
; whatever follows that fits) from the position the directory is at.
        RFIT    18
c_rdblk:
        FCALL   fc_beg
        DB      FNCRDIR, 2
        LDI     DIRPAGES
        FCALL   fn_pb
        LDI     DIRMODE
        FCALL   fn_pb
        FJMP    go_ack

; c_cdir -- CLOSE_DIRECTORY.
        RFIT    8
c_cdir: FCALL   fc_beg
        DB      FNCCDIR, 0
        FJMP    go_ack

; c_sdfp -- SET_DEVICE_FULLPATH(device 0, D = host slot, read, V_PATH).
        RFIT    42
c_sdfp: STV     V_ARG
        FCALL   fc_beg
        DB      FNCSDFP, 3
        LDI     0
        FCALL   fn_pb
        LDV     V_ARG
        FCALL   fn_pb
        LDI     FMREAD
        FCALL   fn_pb
        LDR     R8, V_PATH
        LDI     0
        FCALL   tx_fix
        FJMP    go_ack

; c_mimg -- MOUNT_IMAGE(device 0, read): the ESP32 pushes the file to the
; cart before it answers, so the wait is past the cart's own 60 s.
        RFIT    24
c_mimg: FCALL   fc_beg
        DB      FNCMIMG, 2
        LDI     0
        FCALL   fn_pb
        LDI     FMREAD
        FCALL   fn_pb
        SETV    V_TMO, 66
        FJMP    go_ack

; c_adpx -- GET_ADAPTERCONFIG_EXTENDED (AC_* offsets).
        RFIT    8
c_adpx: FCALL   fc_beg
        DB      FNCADPX, 0
        FJMP    go_ack

; c_wst -- GET_WIFISTATUS: reply byte 0 is 3 when connected.
        RFIT    8
c_wst:  FCALL   fc_beg
        DB      FNCWST, 0
        FJMP    go_ack

; c_scan -- SCAN_NETWORKS: reply byte 0 is the count.
        RFIT    8
c_scan: FCALL   fc_beg
        DB      FNCSCAN, 0
        FJMP    go_ack

; c_sres -- GET_SCAN_RESULT(D = index): ssid at reply offset 0.
        RFIT    21
c_sres: STV     V_ARG
        FCALL   fc_beg
        DB      FNCSRES, 1
        LDV     V_ARG
        FCALL   fn_pb
        FJMP    go_ack

; c_ssid -- SET_SSID from V_SSID and V_EDIT (the password), and save it.
        RFIT    35
c_ssid: FCALL   fc_beg
        DB      FNCSSID, 1
        LDI     1
        FCALL   fn_pb
        LDR     R8, V_SSID
        LDI     33
        FCALL   tx_fix
        LDR     R8, V_EDIT
        LDI     64
        FCALL   tx_fix
        FJMP    go_ack

; c_copy -- COPY_FILE(V_SRCH -> V_HSEL): V_PATH (directory and file) to the
; destination's root, under the same name.
        RFIT    51
c_copy: FCALL   fc_beg
        DB      FNCCOPY, 2
        LDV     V_SRCH
        ADI     1
        FCALL   fn_pb
        LDV     V_HSEL
        ADI     1
        FCALL   fn_pb
        LDR     R8, V_PATH
        FCALL   tx_str
        LDI     '|'
        FCALL   fn_txb
        LDI     '/'
        FCALL   fn_txb
        SETV    V_TMO, 66
        FJMP    go_ack
```

boot.inc config/src/boot.inc

```
; boot.inc -- st_boot: mount the file V_PATH names into device slot 0 and
; hand the console to it.
;
; MOUNT_IMAGE makes the ESP32 push the file to the cart before it answers;
; the cart stages it and draws the progress bar in the status strip itself.
; Once BOOTSTATE is READY, fn_boot arms the swap and restarts the BIOS, which
; the cart answers with the new image. It returns only if the swap never arms.

        RFIT    182
st_boot:
        STATE
        FCALL   pr_cls
        DB      "BOOTING", 0
        LDI     20H                     ; the name, while the reply still has it
        FCALL   d_at
        LDI     TOP_MARQUEE
        FCALL   d_op
        LDV     V_CUR
        FCALL   ent_off
        LDI     0
        FCALL   d_rpl
        FCALL   pr_at
        DB      50H, "MOUNTING...", 0
        FCALL   c_cdir                  ; the listing is done with
        LDV     V_HOST
        FCALL   c_mhost
        BZ      bt1
        FCALL   fail
        DB      2
        DW      bt_fail
bt1:    LDV     V_HOST
        FCALL   c_sdfp
        BZ      bt2
        FCALL   fail
        DB      5
        DW      bt_fail
bt2:    FCALL   c_mimg
        BZ      bt3
        FCALL   fail
        DB      6
        DW      bt_fail
bt3:    LDI     10
        FCALL   fn_frames
bt4:    PEEK    FN_BOOTSTATE
        XRI     FN_BOOT_READY
        BZ      bt6
        XRI     FN_BOOT_READY ! FN_BOOT_FAILED ; AS: '!' is XOR ('^' is power)
        BZ      bt5
        FCALL   fn_tick
        BDF     bt4
        LDI     FNEWAIT
        BR      bt7
bt5:    PEEK    FN_BOOTERR
bt7:    FCALL   fail
        DB      7
        DW      bt_fail
bt6:    FCALL   pr_at
        DB      50H, "STARTING...", 0
        FCALL   fn_boot
        FCALL   fail
        DB      15
        DW      bt_fail

; bt_fail -- back to the listing, the file's name off V_PATH again.
bt_fail:
        STATE
        FCALL   path_cut
        FJMP    st_files
```

edit.inc config/src/edit.inc

```
; edit.inc -- st_edit: the on-screen keyboard.
;
; Rows 3-8 hold all of $20-$7F, sixteen to a row, so the cell number is the
; character less $20 and its screen position the character less $10: no
; table, and no CASE key (the font's lowercase is its own, shorter glyph).
; V_EDIT holds the text; V_EDWHAT says where it goes when 9 says done.

        RFIT    132
st_edit:
        STATE
        LDV     V_EDWHAT
        BNZ     ste1
        FCALL   pr_cls
        DB      "NETWORK NAME", 0
        BR      ste3
ste1:   XRI     ED_PASS
        BNZ     ste2
        FCALL   pr_cls
        DB      "PASSWORD", 0
        BR      ste3
ste2:   FCALL   pr_cls
        DB      "HOST NAME", 0
ste3:   FCALL   pr_key
        DB      20H, "5:ADD 0:DEL 9:OK", 0
        FCALL   pr_leg
        DB      "1:CANCEL", 0
        LDR     R8, V_EDIT
ste4:   LDA     R8
        BNZ     ste4
        DEC     R8
        GLO     R8
        SMI     LO(V_EDIT)
        STV     V_EDLEN
        SETV    V_OSK, 'a'-20H
        FCALL   ed_grid
        LDV     V_OSK
        FCALL   ed_sel
        FCALL   ed_text
        FJMP    ed_in

        RFIT    34
ed_in:  FCALL   key_wait
        FCALL   kdisp
        KEYTO   2, ed_up
        KEYTO   8, ed_dn
        KEYTO   4, ed_lt
        KEYTO   6, ed_rt
        KEYTO   5, ed_add
        KEYTO   0, ed_del
```

```
        KEYTO   9, ed_ok
        KEYTO   1, ed_esc
        DB      0FFH
        FJMP    ed_in

; Moves wrap at the edges. The old cell is repainted plain first.
        RFIT    117
ed_up:  LDV     V_OSK
        FCALL   ed_cell
        LDV     V_OSK
        SMI     16
        BDF     ed_mv
        ADI     96
        BR      ed_mv
ed_dn:  LDV     V_OSK
        FCALL   ed_cell
        LDV     V_OSK
        ADI     16
        STR     R2
        SMI     96
        BDF     ed_mv
        LDN     R2
        BR      ed_mv
ed_lt:  LDV     V_OSK
        FCALL   ed_cell
        LDV     V_OSK
        ANI     0FH
        BZ      edl1
        LDV     V_OSK
        SMI     1
        BR      ed_mv
edl1:   LDV     V_OSK
        ADI     15
        BR      ed_mv
ed_rt:  LDV     V_OSK
        FCALL   ed_cell
        LDV     V_OSK
        ANI     0FH
        XRI     0FH
        BZ      edr1
        LDV     V_OSK
        ADI     1
        BR      ed_mv
edr1:   LDV     V_OSK
        SMI     15
ed_mv:  STV     V_OSK
        FCALL   ed_sel
        FCALL   ed_text
        FJMP    ed_in

; ed_add -- type the selected character, if there is room ($7F types nothing).
        RFIT    77
ed_add: LDV     V_OSK
        XRI     7FH-20H
        BZ      ed_bad
        LDV     V_EDLEN
        STR     R2
        LDV     V_EDMAX
        SM
        BZ      ed_bad                  ; full
        LDN     R2
        ADI     LO(V_EDIT)
        PLO     R8
        LDI     HI(V_EDIT)
        PHI     R8
        LDV     V_OSK
        ADI     20H
        STR     R8
        INC     R8
        LDI     0
        STR     R8
        LDV     V_EDLEN
        ADI     1
        STR     RB
        BR      ed_shw
ed_del: LDV     V_EDLEN
        BZ      ed_bad
        SMI     1
        STR     RB
        ADI     LO(V_EDIT)
        PLO     R8
        LDI     HI(V_EDIT)
        PHI     R8
        LDI     0
        STR     R8
ed_shw: FCALL   ed_text
        FJMP    ed_in
ed_bad: LDI     8
        PLO     RD
        FJMP    ed_in

; ed_ok -- the text goes where V_EDWHAT says.
        RFIT    71
ed_ok:  LDV     V_EDWHAT
        BNZ     eok1
        LDR     R8, V_EDIT              ; a network typed by name: now its password
        LDR     R9, V_SSID
        LDI     32
        FCALL   s_copy
        LDR     R8, V_EDIT
        LDI     0
        STR     R8
        SETV    V_EDWHAT, ED_PASS
        SETV    V_EDMAX, 63
        FJMP    st_edit
eok1:   XRI     ED_PASS
        BNZ     eok2
        FJMP    st_conn
eok2:   FJMP    hs_save
ed_esc: LDV     V_EDWHAT
        XRI     ED_HOST
        BNZ     ees1
        FJMP    st_hosts
ees1:   FJMP    st_wifi

; ed_grid -- characters $20-$7F from row 3: the engine wraps each row.
        RFIT    24
ed_grid:
        LDI     30H
        FCALL   d_at
        LDI     20H
        PLO     R8
        LDI     TX_PUTC
        PHI     RF
edg1:   GLO     R8
        PLO     RF
        LDN     RF
        INC     R8
        GLO     R8
        XRI     80H
        BNZ     edg1
        LDI     FN_PARK
        PHI     RF
        FRET

; ed_sel  -- paint cell D inverse: the keyboard's cursor.
; ed_cell -- paint cell D plain.
        RFIT    27
ed_sel: PLO     R9
        LDI     TOP_INV_ON
        FCALL   d_op
        GLO     R9
        FCALL   ed_cell
        LDI     TOP_INV_OFF
        FJMP    d_op
ed_cell:
        PLO     R9
        ADI     30H
        FCALL   d_at
        GLO     R9
        ADI     20H
        FJMP    d_putc

; ed_text -- row 1: the text's last 14 characters, then the selected key as
; an inverse block cursor (what 5 would type), short of the row's end so
; CLREOL never wraps onto row 2.
        RFIT    54
ed_text:
        LDI     10H
        FCALL   d_at
        LDV     V_EDLEN
        SMI     15
        BNF     edt1
        LDV     V_EDLEN
        ADI     LO(V_EDIT)-14
        BR      edt2
edt1:   LDI     LO(V_EDIT)
edt2:   PLO     R8
        LDI     HI(V_EDIT)
        PHI     R8
        FCALL   d_puts
        LDI     TOP_INV_ON
        FCALL   d_op
        LDV     V_OSK
        ADI     20H
        FCALL   d_putc
        LDI     TOP_INV_OFF
        FCALL   d_op
        LDI     TOP_CLREOL
        FJMP    d_op
```

No limit to your fun and action when you add FujiNet programs to your *Studio II* library.

Here's what's currently available:

CONFIG—hosts, WiFi and the SD card, built into every cartridge.

5 Card Stud—poker against the world.

Battleship—four boards, two players and two bots.

netcat—talk to any TCP server, one line at a time.

Write your own

This handbook carries everything the cartridge and the FujiNet answer, with a program for every part of it.

… and more on the way!

See the FujiNet project on GitHub.

github.com/FujiNetWIFI **fujinet.online** **© 2026 the FujiNet Project. CC BY-SA 4.0**

FN-S2-PH (REV.)

FUJINET